

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KÌ
HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

Đề tài:

QUẢN LÝ HỒ SƠ BỆNH ÁN KHOA NGOẠI TRÚ

Nguyễn Thị Hồng Tuyết	22521637
Phan Thị Tường Vi	22521658
Nguyễn Lâm Khôi Nguyên	22520975
Lê Thị Ánh Xuân	22521714

Hồ Chí Minh, tháng 6, năm 2024

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KÌ
HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU
Đề tài

QUẢN LÝ HỒ SƠ BỆNH ÁN KHOA NGOẠI TRÚ

Nguyễn Thị Hồng Tuyết	22521637
Phan Thị Tường Vi	22521658
Nguyễn Lâm Khôi Nguyên	22520975
Lê Thị Ánh Xuân	22521714

GV Lý thuyết: Thái Bảo Trân

GV HDTH: Nguyễn Hồ Duy Tri

Hồ Chí Minh, tháng 6, năm 2024

NHẬN XÉT CỦA GIÁO VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP Hồ Chí Minh, ngày , tháng, năm

Người nhận xét
(ký và ghi rõ họ tên)

MỤC LỤC

Contents

NHẬN XÉT CỦA GIÁO VIÊN	1
DANH MỤC HÌNH ẢNH	4
NỘI DUNG	5
PHẦN I. TỔNG QUAN ĐỒ ÁN	5
1. Phát biểu bài toán	5
1.2. Lí do chọn đề tài	5
2. Xác định và phân tích yêu cầu	6
2.1. Xác định yêu cầu.....	6
2.2. Phân tích yêu cầu.....	7
3. Thiết kế mô hình quan hệ	9
3.1. Mô hình ERD (Thực thể mối kết hợp)	9
3.2. Mô hình quan hệ.....	9
3.3. Bảng thuyết minh quan hệ và thuộc tính.....	10
3.4. Mô tả ràng buộc toàn vẹn	16
PHẦN II. XÂY DỰNG VÀ QUẢN LÝ GIAO TÁC.....	26
1. Qui định hệ thống giao tác.	26
2. Xây dựng và mô tả Trigger trong đồ án môn học	26
3. Xây dựng và mô tả Procedure, Function trong đồ án môn học	51
3.1 Procedure.....	51
3.2 Function	92
PHẦN III. XỬ LÝ ĐỒNG THỜI.....	95
1. Tổng quan	95
2.Mô tả kịch bản gây mất nhất quán dữ liệu, giải pháp đề ra trong đồ án môn học	100
PHẦN IV. THIẾT KẾ GIAO DIỆN	135
1. Thiết kế giao diện.....	135
1.1. Danh sách các giao diện.....	135
1.2. Màn hình giao diện Trang chủ	137
1.3 Màn hình giao diện Đăng nhập	138
1.4 . Màn hình giao diện đăng ký.....	139

1.5. Màn hình giao diện tương tác với bác sĩ.....	140
1.6. Màn hình giao diện tương tác của nhân viên thanh toán.	145
1.7. Màn hình giao diện tương tác với admin.....	148
1.8. Màn hình giao diện tương tác với quản lý kho.....	155
1.9. Màn hình giao diện tương tác với nhân viên tiếp nhận.....	159
PHẦN V. KẾT LUẬN.....	166
1. Bảng phân công công việc	166
2. Môi trường phát triển và môi trường triển khai	167
2.1. Môi trường phát triển	167
2.2. Môi trường triển khai	167
3.1. Kết quả đạt được	167
3.2. Khó khăn	168

DANH MỤC HÌNH ẢNH

Hình 1 Màn hình giao diện trang chủ tổng quan của nhân viên	137
Hình 2 Màn hình đăng nhập của nhân viên	138
Hình 3 Đăng ký	139
Hình 4 Quản lý thông tin khám bệnh.....	140
Hình 5 Kê toa thuốc	142
Hình 6 Thêm chi tiết đơn thuốc	144
Hình 7 Phiếu khám bệnh.....	144
Hình 8 Trang quản lý thông tin biên lai	145
Hình 9 . Thêm biên lai	146
Hình 10 Thông tin chi tiết biên lai	147
Hình 11 Trang chủ admin	148
Hình 12 Trang quản lý thông tin nhân viên	149
Hình 13 Thông tin nhân viên	150
Hình 14 Danh sách tài khoản.....	151
Hình 15 Thông tin tài khoản	152
Hình 16 Thống kê kê quế quán của bệnh nhân.....	153
Hình 17 Thống kê doanh thu qua các năm	154
Hình 18 Thống kê doanh thu theo tháng của năm	155
Hình 19 Quản lý tồn kho thuốc.....	155
Hình 20 Thêm thông tin thuốc	157
Hình 21 Sửa thông tin thuốc	158
Hình 22 Trang chủ nhân viên tiếp nhận.....	159
Hình 23 Trang quản lý thông tin bệnh nhân	160
Hình 24 Thông tin bệnh nhân	162
Hình 25 . Thêm mới phiếu khám bệnh	163
Hình 26 hiếu khám bệnh PDF.....	164
Hình 27 Biên lai PDF.....	165
Hình 28 Phiếu khám bệnh chi tiết PDF	165

NỘI DUNG

PHẦN I. TỔNG QUAN ĐỒ ÁN

1. Phát biểu bài toán

1.1. Tóm tắt nội dung đề tài (bản tiếng Anh)

Technological advancements are heralding a new era of robust digital transformations that transcend the ever-evolving landscape of the medical field. As time progresses, the cumbersome nature of manual, paper-based record-keeping systems consistently proves to be an impediment to hospital administrative operations, resulting in their diminishing prevalence. It becomes glaringly apparent of a predicament as healthcare providers are inundated with the increasing volume of patient data, while simultaneously grappling with the limitations of outdated record management methodologies.

Recognizing the ongoing and evident need for an empirical alternative to the antiquated systems already in place, this project emerges as a concerted effort to address these systematic shortcomings. Our objective is to pursue a comprehensive and integrated technology-driven management system that streamlines administrative workflows, enhances operational efficiency, and improves healthcare delivery effectiveness while prioritizing patient privacy and adhering to regulatory mandates.

Our comprehensive medical record management system is designed to efficiently handle a wide range of patient data. It includes meticulous documentation of:

Diagnoses and prescriptions;

Invoice information;

Records of medication inventory.

Through this integrated approach, our system ensures the smooth operation of healthcare facilities, enabling healthcare professionals to deliver high-quality care while maintaining accurate and organized patient records.

1.2. Lí do chọn đề tài

Tình trạng lưu trữ hồ sơ giấy bệnh nhân tại bệnh viện hiện nay đang đối mặt với nhiều thách thức. Bộ phận quản lý hồ sơ bệnh án của bệnh nhân đã chỉ ra rằng quá trình lưu trữ hồ sơ giấy không chỉ gặp phải vấn đề về không gian lưu trữ mà còn liên quan đến sự tổ chức và quản lý không hiệu quả dẫn đến việc tìm kiếm lâu và lạc hồ sơ.

Bên cạnh đó, hệ thống hồ sơ giấy được viết bằng chữ viết tay có thể có đọc và không rõ ràng, dẫn đến sự hiểu nhầm giữa các bác sĩ khi đọc hồ sơ bệnh án cũng như tạo sự khó hiểu đối với bệnh nhân. Chữ viết tay trên giấy có thể bị bay màu qua thời gian. Dù chất lượng mực viết và giấy có thể tốt, nhưng việc sử dụng thường xuyên cũng có thể làm cho chữ viết bị bay màu theo thời gian.

Việc phân loại và lưu trữ hồ sơ phụ thuộc vào thông tin của mỗi bệnh nhân, dựa vào hệ thống tập tin giấy dẫn đến việc tìm kiếm thông tin mất nhiều thời gian và công sức, khả năng xảy ra các tình huống xấu cao như lạc hồ sơ hay sự mai một của giấy. Bên cạnh đó, không gian lưu trữ cũng là mối lo ngại của nhiều bệnh viện khi số bệnh nhân trong nước ngày càng cao, dẫn đến sự thụt hụt không gian cũng như hỏng hóc trong quá trình lưu trữ.

Để giải quyết vấn đề trên, cần một hệ thống lưu trữ thông minh và hiệu quả phục vụ công tác quản lý hồ sơ bệnh án bệnh nhân của bệnh viện gồm thông tin bệnh nhân, thông tin phiếu khám bệnh, thông tin toa thuốc, chi tiết đơn thuốc và biên lai đóng tiền ứng với mỗi bệnh nhân. Hệ thống được xây dựng phải là một hệ thống quản lý tích hợp, lưu trữ thông tin cá nhân, bệnh án của bệnh nhân. Qua đó hệ thống phải đảm bảo yêu cầu về bảo mật thông tin và xuất giấy bệnh án cho mỗi bệnh nhân để họ theo dõi tình trạng sức khỏe của mình.

2. Xác định và phân tích yêu cầu

2.1. Xác định yêu cầu

Tài khoản: Lưu trữ thông tin nhân viên hay hỗ trợ cho việc phân quyền các chức năng khi đăng nhập vào hệ thống.

Nhân viên: lưu trữ thông tin nhân viên, thêm, xóa, sửa, tìm kiếm nhân viên.

Bệnh nhân: Lưu trữ hồ sơ thông tin, thêm, xóa, sửa thông tin bệnh nhân, hỗ trợ tìm kiếm theo SDT.

Phiếu khám bệnh: Lưu trữ thông tin phiếu khám bệnh, thêm, xóa, sửa phiếu khám bệnh, hỗ trợ tìm kiếm theo tên bệnh nhân.

Phòng khám: Lưu trữ thông tin phòng khám, ngày giờ thời gian hoạt động.

Thuốc: Lưu trữ thông tin thuốc, thêm, xóa, sửa thuốc, hỗ trợ tìm kiếm theo tên thuốc.

Toa thuốc: Lưu trữ thông tin toa thuốc, tổng tiền thuốc, thêm, xóa, sửa toa thuốc.

Chi tiết đơn thuốc: Lưu trữ thông tin thuốc thuộc toa thuốc nào, mỗi chi tiết đơn thuốc thuộc về một toa thuốc và 1 toa thuốc có nhiều thuốc khác nhau.

Biên lai: Lưu trữ thông tin biên lai, tổng tiền khám, thêm, xóa, tìm kiếm theo mã biên lai.

2.2. Phân tích yêu cầu

2.2.1. Yêu cầu chức năng

2.2.1.1. Yêu cầu lưu trữ

Tính năng lưu trữ là một trong những tính năng qua trọng phải có trong các hệ thống, là cơ sở để thực hiện các tính năng khác, để đảm bảo việc quản lý hồ sơ bệnh án khoa ngoại của bệnh viện hiệu quả, tính năng lưu trữ cần lưu những thông tin sau:

Thông tin bệnh nhân: Mã bệnh nhân, tên bệnh nhân, số điện thoại, ngày sinh, địa chỉ, quê quán, ghi chú(tiền sử bệnh).

Thông tin các nhân viên ở các bộ phận: Mã nhân viên, tên nhân viên, ngày sinh, số địa chỉ, giới tính, ngày vào làm , vai trò (bác sĩ, nhân viên tiếp nhận, nhân viên thanh toán).

Thông tin về phiếu thông tin khám bệnh: Mã phiếu khám bệnh, mã bác sĩ, mã bệnh nhân, mã phòng khám, ngày tạo, chẩn đoán.

Thông tin về biên lai hóa đơn: Mã biên lai, mã nhân viên thanh toán, mã toa thuốc , ngày tạo, tổng tiền khám.

Thông tin về thuốc: Tên thuốc, nước sản xuất, đơn giá , hạn sử dụng, số lượng tồn.

Thông tin về toa thuốc: Mã toa thuốc, mã bác sĩ, mã bệnh nhân, tổng tiền thuốc, ngày lập, mã phiếu khám bệnh.

Thông tin về phòng khám: Mã phòng khám, tên phòng khám, tên khoa, thời gian bắt đầu, thời gian kết thúc.

Thông tin về chi tiết đơn thuốc: Mã thuốc, mã toa thuốc, số lượng, ghi chú.

2.2.1.2. Yêu cầu về tính năng

Quản lý tài khoản: Bệnh viện cần quản lý thông tin từng tài khoản và phân quyền cho từng loại tài khoản khi truy cập vào hệ thống. Cần có các chức năng thêm, xóa, sửa, tìm kiếm các thông tin cơ bản của tài khoản.

Quản lý hồ sơ bệnh nhân: Bệnh viện cần quản lý thông tin từng bệnh nhân

Cần có các chức năng thêm, xóa, sửa, tìm kiếm các thông tin cơ bản, xuất danh sách bệnh nhân và thống kê kê quê quán có số lượng bệnh nhân đến khám.

Quản lý thông tin nhân viên: Bệnh viện cần quản lý thông tin từng nhân viên. Cần có các chức năng thêm, xóa, sửa, tìm kiếm theo số điện thoại.

Quản lý thông tin phiếu khám bệnh: Bệnh viện cần quản lý thông tin từng phiếu khám bệnh. Cần có các chức năng thêm, xóa, sửa, tìm kiếm theo tên bệnh nhân và chức năng tạo toa thuốc và chi tiết đơn thuốc.

Quản lý thông tin biên lai: Bệnh viện cần quản lý thông tin từng biên lai thanh toán. Cần có các chức năng thêm, sửa, tìm kiếm theo mã bệnh nhân, và thống kê báo cáo doanh thu theo năm.

Quản lý thông tin thuốc: Bệnh viện cần quản lý thông tin thuốc. Cần có các chức năng thêm, sửa, tìm kiếm các thông tin cơ bản.

Tính năng tính toán: Tính toán tự động các chi phí khám bệnh và mua thuốc dựa trên thông tin nhập vào, bao gồm tiền khám (tiền thuốc + 300000), tiền thuốc (số lượng * đơn giá của loại thuốc).

Tính năng thống kê báo cáo: Cung cấp báo cáo về què quán có số lượng bệnh nhân đến khám bệnh, thống kê về số lượng bệnh mà các bệnh nhân mắc phải, thống kê doanh thu khám bệnh theo năm.

2.2.2. Yêu cầu phi chức năng

Giao diện:

Dễ sử dụng và thân thiện: Giao diện người dùng cần được thiết kế đơn giản, dễ hiểu và dễ sử dụng để người dùng có thể tương tác một cách tự nhiên và không gặp khó khăn khi sử dụng.

Nhiều ngôn ngữ: Hệ thống cần hỗ trợ nhiều ngôn ngữ để phù hợp với nhu cầu sử dụng của người dùng đa dạng về ngôn ngữ và văn hóa.

Tương tác cao: Giao diện cần hỗ trợ tính năng tương tác cao như kéo thả, lướt trang, và giao tiếp thời gian thực để tạo ra trải nghiệm người dùng mượt mà và thuận tiện.

Phù hợp về văn hóa và chính trị: Giao diện cần được thiết kế sao cho phù hợp với văn hóa và giá trị chính trị của cộng đồng sử dụng.

Tính bảo mật:

Phân quyền cụ thể: Hệ thống cần có khả năng phân quyền cụ thể để quản lý truy cập thông tin theo vai trò của từng người dùng, đảm bảo chỉ những người được ủy quyền mới có thể truy cập vào các chức năng và dữ liệu tương ứng.

Chặt chẽ: Hệ thống cần có các biện pháp bảo mật mạnh mẽ như mã hóa dữ liệu, xác thực hai yếu tố, và giám sát hoạt động truy cập để đảm bảo an toàn thông tin.

Tính tương thích:

Chạy được trên nhiều hệ điều hành: Hệ thống cần được phát triển sao cho có khả năng chạy trên nhiều hệ điều hành khác nhau như Windows, macOS, và Linux để đáp ứng nhu cầu sử dụng đa dạng của người dùng.

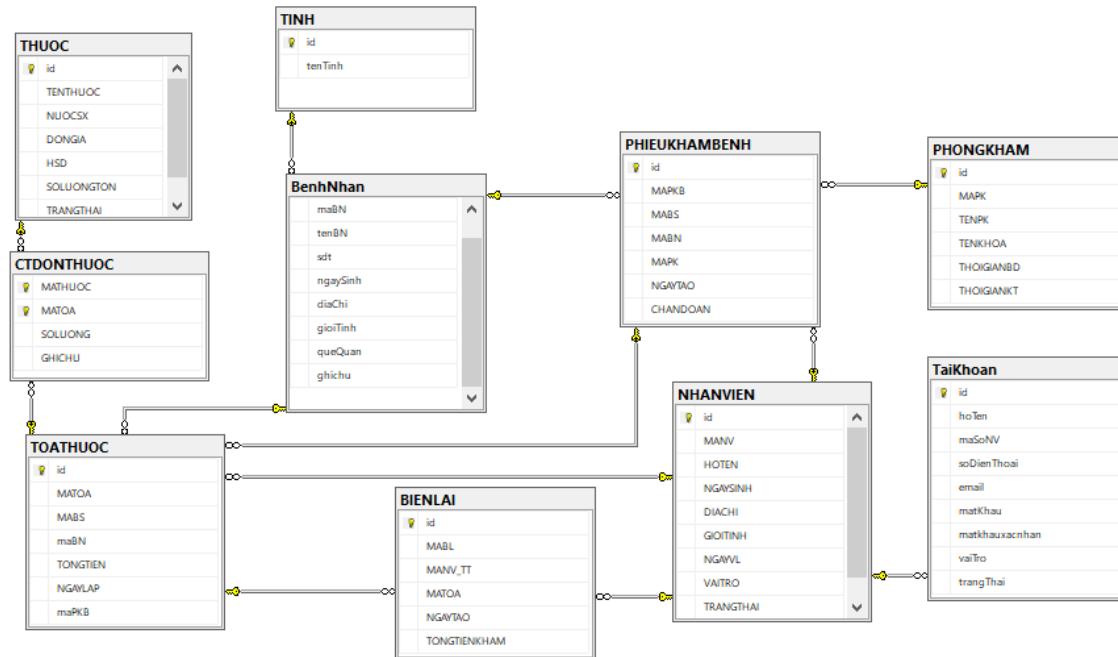
Hiệu suất:

Đảm bảo ổn định: Hệ thống cần được thiết kế và tối ưu hóa để đảm bảo hoạt động ổn định và mượt mà ngay cả khi có nhiều người dùng đăng nhập và sử dụng cùng lúc.

Điều này đảm bảo trải nghiệm người dùng luôn được duy trì ở mức cao nhất mà không gặp phải sự gián đoạn hoặc trì trệ.

3. Thiết kế mô hình quan hệ

3.1. Mô hình ERD (Thực thể mối kết hợp)



3.2. Mô hình quan hệ

BệnhNhan(id, maBN, tenBN, sdt, ngaySinh, diaChi, gioiTinh, queQuan, ghiChu).

NhanVien(id, maNV, hoTen, ngaySinh, diaChi, gioiTinh, ngayVL, vaiTro, trangThai).

TaiKhoan(id, hoTen, maSoBN, maSoNV, soDienThoai, email, matKhau, matKhauXacNhan, vaiTro, trangThai).

PhieuKhamBenh(id, maPKB, maBS, maBN, ngayTao, chanDoan).

ToaThuoc(id, maToa, maBS, maBN, tongTien, ngayLap, maPKB).

BienLai(id, maBL, maNV_TT, maToa, ngayTao, tongTienKham).

CTDonThuoc(maThuoc, maToa, soLuong, ghiChu).

Thuoc(id, tenThuoc, nuocSX, donGia, HSD, soLuongTon, trangThai).

Tinh(id, tenTinh).

PhongKham(id, maPK, tenPK, tenKhoa, thoiGianBD, thoiGianKT).

3.3. Bảng thuyết minh quan hệ và thuộc tính

BẢNG THUỘC TÍNH

• Quan hệ BenhNhan

BenhNhan				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	int	Khóa chính, not null	Tự động	Số tự động
maBN	VARCHAR(10)	Not null	Tự động theo dạng 'BN'	Mã bệnh nhân
tenBN	NVARCHAR(50)	Not null		Tên bệnh nhân
sdt	VARCHAR(10)	Not null		Sdt bệnh nhân
ngaySinh	SMALLDATETIME	Not null		Ngày sinh bệnh nhân
diaChi	NVARCHAR(50)	Not null		Địa chỉ bệnh nhân
gioiTinh	NVARCHAR(5)	Not null		Giới tính bệnh nhân
queQuan	INTEGER	Not null		Quê quán bệnh nhân
ghiChu	NVARCHAR(50)	Not null		Ghi chú

• Quan hệ NhanVien

NhanVien				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	int	Khóa chính, not null	Tự động	Số tự động
maNV	VARCHAR(10)	Not null	Tự động theo dạng 'NV'	Mã nhân viên
hoTen	NVARCHAR(50)	Not null		Tên nhân viên
ngaySinh	SMALLDATETIME	Not null		Ngày sinh nhân viên
diaChi	NVARCHAR(50)	Not null		Địa chỉ nhân viên
gioiTinh	NVARCHAR(5)	Not null		Giới tính nhân viên
ngayVL	SMALLDATETIME	Not null		Ngày vào làm
vaiTro	NVARCHAR(20)	Bác sĩ, nhân viên tiếp nhận, nhân viên thanh		Vai trò

		toán, admin, quản lý kho		
trangThai	NVARCHAR(20)	Not null		Trạng thái của nhân viên (Đang tồn tại, nghỉ làm)

• Quan hệ TaiKhoan

TaiKhoan				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	int	Khóa chính, not null	Tự động	Số tự động
hoTen	NVARCHAR(50)	Not null		Tên tài khoản
maSoBN	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng BENHNHAN		Mã bệnh nhân
maSoNV	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng NHANVIEN		Mã nhân viên
soDienThoai	VARCHAR(10)	Not null		Số điện thoại
email	VARCHAR(50)	Not null		Địa chỉ bệnh nhân
matKhau	VARCHAR(50)	Not null		Giới tính bệnh nhân
matKhauXacNhan	VARCHAR(50)	Not null		Quê quán bệnh nhân
vaiTro	NVARCHAR(20)	nhân viên		Ghi chú
trangThai	NVARCHAR(20)	NOT NULL		Trạng thái của tài khoản(Hiệu lực, không có hiệu lực)

• Quan hệ PhieuKhamBenh

PhieuKhamBenh				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	int	Khóa chính, not null	Tự động	Số tự động
maPKB	VARCHAR(10)	Not null	Tự động theo dạng 'PKB'	Mã toa thuốc
maBS	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng NHANVIEN với vaiTro='Bác sĩ'		Mã bác sĩ
maBN	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng BENH NHAN		Mã bệnh nhân
ngayTao	SMALLDATETIME	Not null		Ngày lập phiếu khám bệnh
chanDoan	NVARCHAR(50)	Not null		Chẩn đoán bệnh

• Quan hệ ToaThuoc

ToaThuoc				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	int	Khóa chính, not null	Tự động	Số tự động
maToa	VARCHAR(10)	Not null	Tự động theo	Mã toa thuốc

			dạng 'TOA'	
maBS	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng NHANVIEN với vaiTro='Bác sĩ'		Mã bác sĩ
maBN	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng BENHNHAN		Mã bệnh nhân
tongTien	MONEY	Tự động tính (SUM(soLuong*donGia)) theo toa thuốc		Tổng tiền thuốc
ngayLap	SMALLDATETIME	Not null		Ngày lập toa thuốc
maPKB	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng PHIEUKHAMBENH		Mã phiếu khám bệnh

• Quan hệ BienLai

BienLai				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	INTEGER	Khóa chính, not null	Tự động	Số tự động
maBL	VARCHAR(10)	Not null	Tự động theo dạng 'BL'	Mã biên lai
maNV_TT	VARCHAR(10)	Khóa ngoại tham chiếu đến khóa chính id của bảng NHANVIEN với vai trò 'NV Thanh Toán'		Mã nhân viên thanh toán

maToa	VARCHAR(10)	Not null		Mã toa thuốc
ngayTao	SMALLDATETIME	Not null		Ngày tạo biên lai
tongTienKham	MONEY	Tự động tính (Tổng tiền thuốc + 300)		Tổng tiền khám bệnh

• Quan hệ CTDonThuoc

CTDonThuoc				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>maThuoc</u>	VARCHAR(10)	Khóa chính, not null. Khóa ngoại tham chiếu đến khóa chính id của bảng THUOC		Mã thuốc
<u>maToa</u>	VARCHAR(10)	Khóa chính, not null. Khóa ngoại tham chiếu đến khóa chính id của bảng TOATHUOC		Mã toa thuốc
soLuong	INTEGER	Not null		Số lượng thuốc
ghiChu	NVARCHAR(200)	Not null		Ghi chú

• Quan hệ Thuoc

Thuoc				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	INTEGER	Khóa chính, not null.	Tự động	Số tự động tăng
tenThuoc	VARCHAR(10)	Not null		Tên thuốc
nuocSX	NVARCHAR(50)	Not null		Nước sản xuất

donGia	MONEY	Not null		Đơn giá thuốc
HSD	SMALLDATETIME	Not null		Hạn sử dụng
soLuongTon	INTEGER	Not null		Số lượng tồn
trangThai	NVARCHAR(20)	Not null		Trạng thái của thuốc

• Quan hệ PhongKham

PhongKham				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	INTEGER	Khóa chính, not null.	Tự động	Số tự động tăng
maPK	VARCHAR(10)	Not null		Mã phòng khám
tenPK	NVARCHAR(50)	Not null		Tên áddgh
tenKhoa	NVARCHAR(50)	Not null		Đơn giá thuốc
thoiGianBD	TIME	Not null		Hạn sử dụng
thoiGianKT	TIME	Not null		Số lượng tồn

• Quan hệ Tinh

Tinh				
Thuộc tính	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Ý nghĩa
<u>id</u>	INTEGER	Khóa chính, not null.	Tự động	Số tự động tăng
tenTinh	NVARCHAR(20)	Not null		Tên tình

3.4. Mô tả ràng buộc toàn vẹn

3.4.1. Ràng buộc toàn vẹn có bối cảnh trên 1 quan hệ

- Ràng buộc toàn vẹn khóa chính, unique

Ràng buộc 1

Bối cảnh: BENHNNHAN

Nội dung: Mỗi bệnh nhân chỉ có một mã bệnh nhân duy nhất.

$\forall bn1, bn2 \in BENHNNHAN: bn1.MABN \neq bn2.MABN$

Bảng tầm ảnh hưởng:

RB1	Thêm	Xóa	Sửa
BENHNNHAN	-	-	+(MABN)

Ràng buộc 2

Bối cảnh: NHANVIEN

Nội dung: Mỗi nhân viên chỉ có một mã nhân viên duy nhất.

$\forall nv1, nv2 \in TAIKHOAN: nv1.MANV \neq nv2.MANV$

Bảng tầm ảnh hưởng:

RB2	Thêm	Xóa	Sửa
NHANVIEN	-	-	+(MANV)

Ràng buộc 3

Bối cảnh: TAIKHOAN

Nội dung: Mỗi tài khoản chỉ có một ID duy nhất.

$\forall tk1, tk2 \in TAIKHOAN: tk1.ID \neq tk2.ID$

Bảng tầm ảnh hưởng: Không có tài khoản nào khác được phép có cùng một ID.

RB3	Thêm	Xóa	Sửa
TAIKHOAN	+	-	+(ID)

Ràng buộc 4

Bối cảnh: PHIEUKHAMBENH

Nội dung: Mỗi phiếu khám bệnh chỉ có một mã phiếu khám bệnh duy nhất.

$\forall pkb1, pkb2 \in \text{TAIKHOAN}: pkb1.MAPKB \neq pkb2.MAPKB$

Bảng tầm ảnh hưởng:

RB4	Thêm	Xóa	Sửa
PHIEUKHAMBE NH	+	-	+(MAPKB)

Ràng buộc 5:

Bối cảnh: ToaThuoc

Nội dung: Mỗi toa thuốc chỉ có một mã toa thuốc duy nhất.

Biểu diễn bằng đại số quan hệ: - $\forall tt1, tt2 \in \text{ToaThuoc}: tt1.maToa \neq tt2.maToa \vee tt1, tt2 \in \text{ToaThuoc}: tt1.maToa \neq tt2.maToa$

Bảng tầm ảnh hưởng: ToaThuoc

RB5	Thêm	Xóa	Sửa
TOATHUOC	+	-	+(MATOA)

Ràng buộc 6:

Bối cảnh: BienLai

Nội dung: Mỗi biên lai chỉ có một mã biên lai duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall bl1, bl2 \in \text{BienLai}:$

$bl1.maBL \neq bl2.maBL \vee bl1, bl2 \in \text{BienLai}: bl1.maBL \neq bl2.maBL$

Bảng tầm ảnh hưởng: BienLai

RB6	Thêm	Xóa	Sửa
BIENLAI	+	-	+(MABL)

Ràng buộc 7:

Bối cảnh: CTDonThuoc

Nội dung: Mỗi chi tiết đơn thuốc chỉ có một mã chi tiết đơn thuốc duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall ctdt1, ctdt2 \in CTDonThuoc: ctdt1.maThuoc \neq ctdt2.maThuoc \vee ctdt1, ctdt2 \in CTDonThuoc: ctdt1.maThuoc = ctdt2.maThuoc$

Bảng tầm ảnh hưởng: CTDonThuoc

RB7	Thêm	Xóa	Sửa
PHIEUKHAMBE NH	+	-	+(SOLUONG)

• Ràng buộc liên thuộc tính**Ràng buộc 1:**

Bối cảnh: TaiKhoan

Nội dung: Mỗi tài khoản phải có mã tài khoản duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall tk1, tk2 \in TaiKhoan: tk1.MaTK \neq tk2.MaTK \vee tk1, tk2 \in TaiKhoan: tk1.MaTK = tk2.MaTK$

Bảng tầm ảnh hưởng:

RB1	Thêm	Xóa	Sửa
TAIKHOAN	+	-	+(maSoNV)

Ràng buộc 2:

Bối cảnh: BenhNhan

Nội dung: Mỗi bệnh nhân chỉ có một mã bệnh nhân duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall bn1, bn2 \in BenhNhan: bn1.maBN \neq bn2.maBN \forall bn1, bn2 \in BenhNhan: bn1.maBN \square = bn2.maBN$

Bảng tầm ảnh hưởng:

RB2	Thêm	Xóa	Sửa
TAIKHOAN	+	-	+(masoNV)

Ràng buộc 3:

Bối cảnh: NhanVien

Nội dung: Mỗi nhân viên chỉ có một mã nhân viên duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall nv1, nv2 \in NhanVien: nv1.maNV \neq nv2.maNV \forall nv1, nv2 \in NhanVien: nv1.maNV \square = nv2.maNV$

Bảng tầm ảnh hưởng:

RB3	Thêm	Xóa	Sửa
PHIEUKHAMBE NH	+	-	+(MABN)

Ràng buộc 4:

Bối cảnh: PhieuKhamBenh

Nội dung: Mỗi phiếu khám bệnh chỉ có một mã phiếu khám bệnh duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall pkb1, pkb2 \in PhieuKhamBenh: pkb1.maPKB \neq pkb2.maPKB \forall pkb1, pkb2 \in PhieuKhamBenh: pkb1.maPKB \square = pkb2.maPKB$

Bảng tầm ảnh hưởng:

RB4	Thêm	Xóa	Sửa
PHIEUKHAMBE NH	+	-	+(MABS)

Ràng buộc 5:

Bối cảnh: ToaThuoc

Nội dung: Mỗi toa thuốc chỉ có một mã toa thuốc duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall tt1, tt2 \in ToaThuoc: tt1.maToa \neq tt2.maToa \vee tt1, tt2 \in ToaThuoc: tt1.maToa = tt2.maToa$

Bảng tầm ảnh hưởng:

RB5	Thêm	Xóa	Sửa
TOATHUOC	+	-	+(MABN)

Ràng buộc 6:

Bối cảnh: CTDonThuoc

Nội dung: Mỗi chi tiết đơn thuốc chỉ có một mã chi tiết đơn thuốc duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall ctdt1, ctdt2 \in CTDonThuoc: ctdt1.maCTDT \neq ctdt2.maCTDT \vee ctdt1, ctdt2 \in CTDonThuoc: ctdt1.maCTDT = ctdt2.maCTDT$

Bảng tầm ảnh hưởng:

RB6	Thêm	Xóa	Sửa
TOATHUOC	+	-	+(MABS)

Ràng buộc 7:

Bối cảnh: Thuoc

Nội dung: Mỗi loại thuốc chỉ có một mã thuốc duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall t1, t2 \in Thuoc: t1.id \neq t2.id \forall t1, t2 \in Thuoc: t1.id \square = t2.id$

Bảng tầm ảnh hưởng:

RB6	Thêm	Xóa	Sửa
TOATHUOC	+	-	+(MABS)

Ràng buộc 8:

Bối cảnh: Tỉnh

Nội dung: Mỗi tỉnh chỉ có một mã tỉnh duy nhất.

Biểu diễn bằng đại số quan hệ:

$\forall t1, t2 \in Tinh: t1.id \neq t2.id \forall t1, t2 \in Tinh: t1.id \square = t2.id$

Bảng tầm ảnh hưởng:

RB8	Thêm	Xóa	Sửa
CTDONTHUOC	-	-	+(MATHUOC)

3.4.2. Ràng buộc toàn vẹn có bối cảnh trên nhiều quan hệ

- Ràng buộc toàn vẹn khóa ngoại

Ràng buộc 1: Mỗi tài khoản phải thuộc về một nhân viên

Bối cảnh: TaiKhoan, BenhNhan, NhanVien

Nội dung: $\forall tk \in TaiKhoan, \exists nv \in NhanVien: tk.maNV = nv.id$

Bảng tầm ảnh hưởng:

RB1	Thêm	Xóa	Sửa
TaiKhoan	+	-	+(maNV)
NhanVien	-	+	-

Ràng buộc 2: Mỗi phiếu khám bệnh phải được tạo lập bởi một bác sĩ

Bối cảnh: PhieuKhamBenh, NhanVien

Nội dung: $\forall pkb \in PhieuKhamBenh, \exists bs \in NhanVien: pkb.maBS = bs.id \wedge bs.vaiTro = 'Bác sĩ'$

Bảng tầm ảnh hưởng:

RB2	Thêm	Xóa	Sửa
PhieuKhamBenh	+	-	+(maBS)
NhanVien	+	+	+(vaiTro)

Ràng buộc 3: Mỗi phiếu khám bệnh phải thuộc về một bệnh nhân

Bối cảnh: PhieuKhamBenh, BenhNhan

Nội dung: $\forall pkb \in PhieuKhamBenh, \exists bn \in BenhNhan : pkb.maBN = bn.id$

Bảng tầm ảnh hưởng:

RB3	Thêm	Xóa	Sửa
PhieuKhamBenh	+	-	+(maBN)
BenhNhan	-	+	-

Ràng buộc 4: Mỗi toa thuốc phải được kê bởi một bác sĩ

Bối cảnh: ToaThuoc, NhanVien

Nội dung: $\forall t \in ToaThuoc, \exists bs \in NhanVien : t.maNV = bs.id \wedge bs.vaiTro = 'Bác sĩ'$

Bảng tầm ảnh hưởng:

RB4	Thêm	Xóa	Sửa
ToaThuoc	+	-	+(maBS)
NhanVien	-	+	+(vaiTro)

Ràng buộc 5: Mỗi toa thuốc phải thuộc về một bệnh nhân

Bối cảnh: ToaThuoc, BenhNhan

Nội dung: $\forall t \in ToaThuoc, \exists bn \in BenhNhan : t.maBN = bn.id$

Tầm ảnh hưởng:

RB5	Thêm	Xóa	Sửa
ToaThuoc	+	-	+(maBN)
BenhNhan	-	+	-

Ràng buộc 6: Mỗi toa thuốc phải thuộc về một phiếu khám bệnh

Bối cảnh: ToaThuoc, PhieuKhamBenh

Nội dung: $\forall t \in ToaThuoc, \exists pkb \in PhieuKhamBenh : t.maPKB = pkb.id$

Tầm ảnh hưởng:

RB6	Thêm	Xóa	Sửa
ToaThuoc	+	-	+(maPKB)
PhieuKhamBenh	-	+	-

Ràng buộc 7: Mỗi biên lai phải được tạo lập bởi một nhân viên thanh toán

Bối cảnh: BienLai, NhanVien

Nội dung: $\forall bl \in BienLai, \exists nv_tt \in NhanVien : bl.maNV_TT = nv_tt.id \wedge nv_tt.vaiTro = 'NV Thanh Toán'$

Tầm ảnh hưởng:

RB7	Thêm	Xóa	Sửa
BienLai	+	-	+(maNV TT)
NhanVien	-	+	+(vaiTro)

Ràng buộc 8: Mỗi biên lai phải thuộc về một toa thuốc

Bối cảnh: BienLai, ToaThuoc

Nội dung: $\forall bl \in BienLai, \exists t \in ToaThuoc : bl.maToa = t.id$

Tầm ảnh hưởng:

RB8	Thêm	Xóa	Sửa
BienLai	+	-	+(maToa)
ToaThuoc	-	+	-

Ràng buộc 9: Mỗi CTDonThuoc phải liệt kê loại thuốc

Bối cảnh: CTDonThuoc, Thuoc

Nội dung: $\forall ct \in CTDonThuoc, \exists t \in Thuoc : ct.maThuoc = t.id$

Tầm ảnh hưởng:

RB9	Thêm	Xóa	Sửa
CTDonThuoc	+	-	+(maThuoc)
Thuoc	-	+	-

Ràng buộc 10: Mỗi CTDonThuoc phải thuộc về một toa thuốc

Bối cảnh: CTDonThuoc, ToaThuoc

Nội dung: $\forall ct \in CTDonThuoc, \exists t \in ToaThuoc : ct.maToa = t.id$

Tầm ảnh hưởng:

RB10	Thêm	Xóa	Sửa
CTDonThuoc	+	-	+(maToa)
ToaThuoc	-	+	-

- Ràng buộc liên thuộc tính liên quan hệ

Ràng buộc 11: Ngày lập toa thuốc phải trước hạn sử dụng của thuốc được kê toa

Bối cảnh: ToaThuoc, CTDonThuoc, Thuoc

Nội dung: $\forall toa \in ToaThuoc, \nexists ct \in CTDonThuoc (\exists t \in Thuoc : toa.id = ct.maToa \wedge ct.maThuoc = t.id \wedge t.HSD < toa.ngayLap)$

Bảng tầm ảnh hưởng:

RB11	Thêm	Xóa	Sửa
ToaThuoc	-	- (*)	+(ngayTao)
CTDonThuoc	+	-	+(maThuoc, maToa)
Thuoc	-	- (*)	+(HSD)

Ràng buộc 12: Ngày tạo phiếu khám bệnh phải sau ngày vào làm của bác sĩ

Bối cảnh: PhieuKhamBenh, NhanVien

Nội dung: $\forall pkb \in PhieuKhamBenh, \nexists bs \in NhanVien : pkb.maBS = bs.id \wedge pkb.ngayTao < bs.ngayVL$

Bảng tầm ảnh hưởng:

RB12	Thêm	Xóa	Sửa
PhieuKhamBenh	+	-	+(ngayTao)
NhanVien	-	- (*)	+(ngayVL)

Ràng buộc 13: Ngày tạo biên lai phải trước ngày vào làm của nhân viên thanh toán

Bối cảnh: BienLai, NhanVien

Nội dung: $\forall bl \in BienLai, \nexists nv \in NhanVien : bl.maNV_TT = nv.id \wedge bl.ngayTao < nv.ngayVL$

Bảng tầm ảnh hưởng:

RB13	Thêm	Xóa	Sửa
BienLai	+	-	+(ngayTao)
NhanVien	-	- (*)	+(ngayVL)

Ràng buộc 14: Số lượng tồn của thuốc phải lớn hơn số lượng thuốc được kê toa

Bối cảnh: Thuoc, CTDonThuoc

Nội dung: $\forall th \in Thuoc, \nexists ct \in CTDonThuoc : ct.maThuoc = th.id \wedge ct.soLuong > th.soLuongTon$

Bảng tầm ảnh hưởng:

RB14	Thêm	Xóa	Sửa
Thuoc	-	- (*)	+(HSD)
CTDonThuoc	+	-	+(soLuongTon)

- Ràng buộc do thuộc tính tổng hợp

Ràng buộc 15: Tổng tiền thuốc của một toa thuốc sẽ bằng tổng giá trị (soLuong * donGia) của các CT đơn thuốc, trong đó donGia là thuộc tính trong bảng Thuoc ứng với các CT đơn thuốc của toa thuốc đó.

Bối cảnh: ToaThuoc, CTDonThuoc, Thuoc

Nội dung: $\forall toa \in ToaThuoc, toa.tongTienThuoc =$

$$\sum_{(ct \in CTDonThuoc, th \in Thuoc : toa.id = ct.maToa \wedge ct.maThuoc = th.id)} (soLuong * donGia)$$

Bảng tầm ảnh hưởng:

RB15	Thêm	Xóa	Sửa
ToaThuoc	-	- (*)	-
CTDonThuoc	+	+	-
Thuoc	-	- (*)	-

PHẦN II. XÂY DỰNG VÀ QUẢN LÝ GIAO TÁC

1. Qui định hệ thống giao tác.

Yêu Cầu	Mục Tiêu	Tác Dụng
Tăng tự động MaBN trong bảng BenhNhan	Đảm bảo mỗi bệnh nhân mới thêm vào hệ thống có một mã bệnh nhân (MaBN) duy nhất, tăng dần.	Tránh trùng lặp mã bệnh nhân, dễ dàng quản lý và tra cứu bệnh nhân.
Tăng tự động MaNV trong bảng NhanVien	Tạo mã nhân viên (MaNV) duy nhất cho mỗi nhân viên mới thêm vào hệ thống.	Quản lý nhân sự hiệu quả, đảm bảo mỗi nhân viên có mã riêng biệt.
Tự động tăng MaPKB trong bảng PhieuKhamBenh	Đảm bảo mỗi phiếu khám bệnh mới có mã phiếu (MaPKB) tăng dần và duy nhất.	Dễ dàng theo dõi và quản lý các phiếu khám bệnh.
Tự động cập nhật thông tin trong hồ sơ bệnh án	Đảm bảo thông tin hồ sơ bệnh án luôn được cập nhật khi có thay đổi.	Đảm bảo tính toàn vẹn và chính xác của hồ sơ bệnh án, hỗ trợ chẩn đoán và điều trị hiệu quả.
Tự động tạo biên lai khi khám bệnh	Tạo biên lai tự động khi bệnh nhân được khám.	Giúp quá trình thanh toán minh bạch và rõ ràng, lưu trữ thông tin thanh toán chính xác.
Tự động cập nhật tổng chi phí của biên lai và hồ sơ bệnh án	Tự động cập nhật chi phí trong biên lai và hồ sơ bệnh án khi có thay đổi.	Đảm bảo tổng chi phí luôn được tính đúng, hỗ trợ quản lý tài chính và kế toán.
Tự động cập nhật trạng thái của hồ sơ bệnh án	Cập nhật trạng thái hồ sơ bệnh án theo quá trình điều trị của bệnh nhân.	Phản ánh chính xác tình trạng hiện tại của bệnh nhân, hỗ trợ quá trình theo dõi và điều trị.
Tự động tạo phiếu toa thuốc khi cần thiết	Tạo toa thuốc tự động khi bác sĩ kê đơn.	Đảm bảo ghi lại chính xác các loại thuốc và liều lượng được kê, hỗ trợ quản lý thuốc và điều trị.
Tự động cập nhật số lượng thuốc trong kho	Cập nhật số lượng thuốc trong kho khi có giao dịch.	Đảm bảo tồn kho chính xác, tránh thiếu hụt thuốc, hỗ trợ quản lý dược phẩm hiệu quả.

2. Xây dựng và mô tả Trigger trong đồ án môn học

STT	TRIGGER	EVENT	TABLE
2.1	trg_CalculateTotalAmount	INSERT, UPDATE, NHANVIEN	CTDONTHUOC
	<u>Mã lệnh T-SQL</u> CREATE TRIGGER trg_CalculateTotalAmount ON CTDONTHUOC		

	<pre> AFTER INSERT, UPDATE, DELETE AS BEGIN DECLARE @MATOA INT; SELECT @MATOA = i.MATOA FROM inserted i; IF @MATOA IS NULL BEGIN SELECT @MATOA = d.MATOA FROM deleted d; END UPDATE TOATHUOC SET TONGTIEN = (SELECT SUM(c.SOLUONG * t.DONGIA) FROM CTDONTHUOC c INNER JOIN THUOC t ON c.MATHUOC = t.id WHERE c.MATOA = @MATOA) WHERE id = @MATOA; END; </pre>
	<u>Mô tả mã lệnh</u> Tạo một biến số nguyên tên là "MATOA": Biến này sẽ lưu trữ giá trị của "MATOA" từ các bản ghi mới thêm vào hoặc cập nhật, hoặc từ các bản ghi bị xóa. Lấy mã toa từ bảng "inserted" hoặc "deleted": Đầu tiên, cố gắng lấy giá trị "MATOA" từ bảng "inserted". Bảng "inserted" chứa các bản ghi mới được thêm vào hoặc các bản ghi sau khi cập nhật. Nếu không có giá trị nào trong "inserted" (nghĩa là thao tác là DELETE), thì lấy giá trị "MATOA" từ bảng "deleted". Bảng "deleted" chứa các bản ghi bị xóa hoặc các bản ghi trước khi cập nhật. Tính tổng tiền của các thuốc trong toa thuốc: Thực hiện câu lệnh UPDATE trên bảng "TOATHUOC" để cập nhật cột "TONGTIEN". "TONGTIEN" sẽ được tính bằng cách: Sử dụng phép nhân giữa "SOLUONG" (số lượng thuốc) từ bảng "CTDONTHUOC" và "DONGIA" (đơn giá thuốc) từ bảng "THUOC". Tổng hợp kết quả của các phép nhân này cho tất cả các bản ghi trong "CTDONTHUOC" có "MATOA" bằng với giá trị của biến "@MATOA". Cập nhật tổng tiền trong bảng "TOATHUOC": Sau khi tính toán, cập nhật giá trị "TONGTIEN" trong bảng "TOATHUOC" cho bản ghi có "id" bằng với giá trị của biến "@MATOA".

2.2	Trigger_TongTienKham	INSERT, UPDATE	BIENLAI
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER Trigger_TongTienKham ON BIENLAI AFTER INSERT, UPDATE AS BEGIN DECLARE @TongTienThuoc MONEY; SELECT @TongTienThuoc = TT.TONGTIEN FROM TOATHUOC TT , INSERTED I WHERE TT.ID = I.MATOA UPDATE BIENLAI SET TONGTIENKHAM = @TongTienThuoc + 300000 WHERE MATOA IN (SELECT MATOA FROM inserted); END; GO </pre>		
	<u>Mô tả mã lệnh</u> Tạo trigger Trigger_TongTienKham trên bảng BIENLAI: Trigger này sẽ được thực thi tự động sau các thao tác INSERT và UPDATE trên bảng BIENLAI. Khai báo một biến kiểu tiền tệ tên là @TongTienThuoc: Biến này sẽ lưu trữ tổng tiền thuốc từ bảng TOATHUOC. Lấy tổng tiền thuốc từ bảng TOATHUOC: Sử dụng câu lệnh SELECT để gán giá trị của cột TONGTIEN trong bảng TOATHUOC vào biến @TongTienThuoc. Thực hiện phép nối giữa bảng TOATHUOC và bảng inserted để lấy giá trị TONGTIEN cho bản ghi vừa được chèn hoặc cập nhật. Cập nhật tổng tiền khám trong bảng BIENLAI: Thực hiện câu lệnh UPDATE để cập nhật cột TONGTIENKHAM trong bảng BIENLAI. Giá trị TONGTIENKHAM sẽ bằng tổng tiền thuốc (@TongTienThuoc) cộng với một khoản phí cố định là 300,000. Điều kiện WHERE MATOA IN (SELECT MATOA FROM inserted) đảm bảo rằng chỉ các bản ghi trong BIENLAI có MATOA nằm trong các bản ghi vừa được chèn hoặc cập nhật sẽ được cập nhật.		
2.3	T_QuyentaoBienLai	INSERT, UPDATE	BIENLAI
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER T_QuyentaoBienLai </pre>		

	<pre> ON BIENLAI AFTER INSERT, UPDATE AS BEGIN IF EXISTS (SELECT 1 FROM inserted i JOIN NHANVIEN nv ON i.MANV_TT = nv.ID WHERE nv.VAITRO != N'NV Thanh Toán') BEGIN PRINT (N'Biên lai chỉ được tạo bởi nhân viên có vai trò NV Thanh Toán'); ROLLBACK TRANSACTION; RETURN END END </pre>		
	<u>Mô tả mã lệnh</u> Tạo trigger T_QuyềnTạoBiênLai trên bảng BIENLAI: Trigger này sẽ được thực thi tự động sau các thao tác INSERT và UPDATE trên bảng BIENLAI. Kiểm tra xem các giá trị mới thêm vào hoặc cập nhật có phù hợp không: Sử dụng câu lệnh IF EXISTS để kiểm tra nếu có bất kỳ bản ghi nào trong bảng inserted mà không thỏa mãn điều kiện. Kiểm tra điều kiện: SELECT 1 FROM inserted i JOIN NHANVIEN nv ON i.MANV_TT = nv.ID WHERE nv.VAITRO != N'NV Thanh Toán': Thực hiện phép nối giữa bảng inserted và bảng NHANVIEN để kiểm tra vai trò (VAITRO) của nhân viên tạo biên lai (MANV_TT). Điều kiện WHERE nv.VAITRO != N'NV Thanh Toán' đảm bảo rằng chỉ các bản ghi mà vai trò của nhân viên không phải là 'NV Thanh Toán' mới được chọn. Nếu điều kiện đúng (có bản ghi không hợp lệ): PRINT (N'Biên lai chỉ được tạo bởi nhân viên có vai trò NV Thanh Toán');: In ra thông báo rằng "Biên lai chỉ được tạo bởi nhân viên có vai trò NV Thanh Toán". ROLLBACK TRANSACTION;: Hủy bỏ giao dịch hiện tại, nghĩa là các thay đổi do thao tác INSERT hoặc UPDATE sẽ không được lưu vào cơ sở dữ liệu. RETURN: Kết thúc việc thực thi trigger.		
2.4	TRIG_TONGTIENTHUOC	INSERT, UPDATE	TOATHUOC
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER TRIG_TONGTIENTHUOC ON TOATHUOC </pre>		

	<pre> FOR INSERT, UPDATE AS BEGIN DECLARE @TONGTIENTHUOC MONEY; DECLARE cur CURSOR FOR SELECT TT.TONGTIEN FROM TOATHUOC TT JOIN INSERTED I ON TT.ID = I.ID; OPEN cur; FETCH NEXT FROM cur INTO @TONGTIENTHUOC; WHILE @@FETCH_STATUS = 0 BEGIN IF @TONGTIENTHUOC <= 0 BEGIN PRINT N'Tổng tiền thuốc phải lớn hơn 0'; ROLLBACK TRANSACTION; RETURN; END FETCH NEXT FROM cur INTO @TONGTIENTHUOC; END CLOSE cur; DEALLOCATE cur; PRINT 'Thành công'; END; </pre>
	<u>Mô tả mã lệnh</u> Tạo trigger TRIG_TONGTIENTHUOC trên bảng TOATHUOC: Trigger này sẽ được thực thi tự động sau các thao tác INSERT và UPDATE trên bảng TOATHUOC. Khai báo biến kiểu tiền tệ tên là @TONGTIENTHUOC: Biến này sẽ lưu trữ giá trị của tổng tiền thuốc từ các bản ghi mới được chèn hoặc cập nhật. Tạo cursor để duyệt qua các bản ghi mới thêm vào hoặc cập nhật: DECLARE cur CURSOR FOR SELECT TT.TONGTIEN FROM TOATHUOC TT JOIN INSERTED I ON TT.ID = I.ID; Cursor này sẽ lấy các giá trị TONGTIEN từ bảng TOATHUOC cho các bản ghi mới được chèn hoặc cập nhật. Mở cursor: OPEN cur; Mở cursor để bắt đầu duyệt qua các bản ghi. Lấy giá trị đầu tiên từ cursor:

	FETCH NEXT FROM cur INTO @TONGTIENHUOC;; Lấy giá trị TONGTIEN từ bản ghi đầu tiên và gán cho biến @TONGTIENHUOC. Sử dụng vòng lặp WHILE để duyệt qua từng bản ghi lấy được trong cursor: WHILE @@FETCH_STATUS = 0: Tiếp tục vòng lặp cho đến khi không còn bản ghi nào. Kiểm tra điều kiện: IF @TONGTIENHUOC <= 0: Kiểm tra nếu tổng tiền thuộc nhỏ hơn hoặc bằng 0. BEGIN PRINT N'Tổng tiền thuộc phải lớn hơn 0'; ROLLBACK TRANSACTION; RETURN; END; In ra thông báo rằng "Tổng tiền thuộc phải lớn hơn 0". Hủy bỏ giao dịch hiện tại, nghĩa là các thay đổi do thao tác INSERT hoặc UPDATE sẽ không được lưu vào cơ sở dữ liệu. Kết thúc việc thực thi trigger để ngăn chặn xử lý tiếp tục. Lấy giá trị tiếp theo từ cursor: FETCH NEXT FROM cur INTO @TONGTIENHUOC;; Lấy giá trị TONGTIEN từ bản ghi tiếp theo. Đóng và giải phóng cursor: CLOSE cur;; Đóng cursor sau khi hoàn thành duyệt qua các bản ghi. DEALLOCATE cur;; Giải phóng tài nguyên đã cấp phát cho cursor. In thông báo thành công: PRINT 'Thành công'; In ra thông báo rằng quá trình kiểm tra đã thành công.		
2.5	TRIG_TONGTIENKHAM	INSERT, UPDATE	BIENLAI
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER TRIG_TONGTIENKHAM ON BIENLAI FOR INSERT, UPDATE AS BEGIN DECLARE @TONGTIENKHAM MONEY; DECLARE cur CURSOR FOR SELECT BL.TONGTIENKHAM FROM BIENLAI BL JOIN INSERTED I ON BL.ID = I.ID; OPEN cur; FETCH NEXT FROM cur INTO @TONGTIENKHAM; WHILE @@FETCH_STATUS = 0 BEGIN IF @TONGTIENKHAM <= 0 BEGIN PRINT N'Tổng tiền khám phải lớn hơn 0'; </pre>		

	<pre> ROLLBACK TRANSACTION; RETURN; END FETCH NEXT FROM cur INTO @TONGTIENKHAM; END CLOSE cur; DEALLOCATE cur; PRINT 'Thành công'; END; </pre> <u>Mô tả mã lệnh</u> Trigger TRIG_TONGTIENKHAM trên bảng BIENLAI: Kích hoạt sau thao tác INSERT và UPDATE. Khai báo biến: @TONGTIENKHAM MONEY: Lưu trữ giá trị tổng tiền khám. Tạo và mở cursor: Lấy giá trị TONGTIENKHAM từ các bản ghi mới chèn hoặc cập nhật. Duyệt qua các bản ghi với cursor: Kiểm tra nếu @TONGTIENKHAM <= 0: In thông báo lỗi: "Tổng tiền khám phải lớn hơn 0". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Dừng xử lý tiếp: RETURN. Đóng và giải phóng cursor: CLOSE cur; DEALLOCATE cur; In thông báo thành công: PRINT 'Thành công';		
2.6	TRIG_SOLUONGTHUOC	INSERT, UPDATE	CTDONTHUOC
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER TRIG_SOLUONGTHUOC ON CTDONTHUOC FOR INSERT, UPDATE AS BEGIN DECLARE @SOLUONGTHUOC INT; DECLARE cur CURSOR FOR SELECT CT.SOLUONG </pre>		

	FROM CTDONTHUOC CT JOIN INSERTED I ON CT.MATOA = I.MATOA OPEN cur; FETCH NEXT FROM cur INTO @SOLUONGTHUOC; WHILE @@FETCH_STATUS = 0 BEGIN IF @SOLUONGTHUOC <= 0 BEGIN PRINT N'Số lượng thuốc phải lớn hơn 0'; ROLLBACK TRANSACTION; RETURN; END FETCH NEXT FROM cur INTO @SOLUONGTHUOC; END CLOSE cur; DEALLOCATE cur; PRINT 'Thành công'; END;		
	<u>Mô tả mã lệnh</u> Trigger TRIG_SOLUONGTHUOC trên bảng CTDONTHUOC: Kích hoạt sau thao tác INSERT và UPDATE. Khai báo biến: @SOLUONGTHUOC INT: Lưu trữ giá trị số lượng thuốc. Tạo và mở cursor: Lấy giá trị SOLUONG từ các bản ghi mới chèn hoặc cập nhật. Duyệt qua các bản ghi với cursor: Kiểm tra nếu @SOLUONGTHUOC <= 0: In thông báo lỗi: "Số lượng thuốc phải lớn hơn 0". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Dừng xử lý tiếp: RETURN. Đóng và giải phóng cursor: CLOSE cur; DEALLOCATE cur; In thông báo thành công: PRINT 'Thành công';		
2.7	TRIG SOLUONGTON		
	<u>Mã lệnh T-SQL</u> CREATE TRIGGER TRIG SOLUONGTON		

	<pre> ON CTDONTHUOC FOR INSERT, UPDATE AS BEGIN DECLARE @SOLUONGTON INT, @SOLUONG INT, @MATHUOC INT; DECLARE cur CURSOR FOR SELECT I.MATHUOC, I.SOLUONG, T.SOLUONGTON FROM INSERTED I JOIN THUOC T ON I.MATHUOC = T.ID; OPEN cur; FETCH NEXT FROM cur INTO @MATHUOC, @SOLUONG, @SOLUONGTON; WHILE @@FETCH_STATUS = 0 BEGIN IF @SOLUONGTON < @SOLUONG BEGIN PRINT N'Số lượng tồn không đủ'; ROLLBACK TRANSACTION; CLOSE cur; DEALLOCATE cur; RETURN; END ELSE BEGIN UPDATE THUOC SET SOLUONGTON = SOLUONGTON - @SOLUONG WHERE ID = @MATHUOC; END FETCH NEXT FROM cur INTO @MATHUOC, @SOLUONG, @SOLUONGTON; END CLOSE cur; DEALLOCATE cur; PRINT 'Thành công'; END; </pre>
	<u>Mô tả mã lệnh</u>

	Trigger TRIG_SOLUONGTON trên bảng CTDONTHUOC: Kích hoạt sau thao tác INSERT và UPDATE. Khai báo biến: @SOLUONGTON INT: Lưu trữ số lượng tồn hiện tại. @SOLUONG INT: Lưu trữ số lượng yêu cầu. @MATHUOC INT: Lưu trữ mã thuốc. Tạo và mở cursor: Lấy các giá trị MATHUOC, SOLUONG từ các bản ghi mới chèn hoặc cập nhật và số lượng tồn (SOLUONGTON) từ bảng THUOC. Duyệt qua các bản ghi với cursor: Kiểm tra nếu @SOLUONGTON nhỏ hơn @SOLUONG: In thông báo lỗi: "Số lượng tồn không đủ". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Đóng và giải phóng cursor: CLOSE cur; DEALLOCATE cur;. Dừng xử lý tiếp: RETURN. Nếu số lượng tồn đủ: Cập nhật số lượng tồn trong bảng THUOC: SOLUONGTON = SOLUONGTON - @SOLUONG. Đóng và giải phóng cursor: CLOSE cur; DEALLOCATE cur; In thông báo thành công: PRINT 'Thành công';		
2.8	<u>TRIG HSD</u>	<u>INSERT, UPDATE</u>	<u>CTDONTHOC</u>
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER TRIG_HSD ON CTDONTHUOC FOR INSERT, UPDATE AS BEGIN DECLARE @MATHUOC INT, @NGAYTOATHUOC DATE, @HSD DATE; DECLARE cur CURSOR FOR SELECT I.MATHUOC, T.HSD, TT.NGAYLAP FROM INSERTED I JOIN THUOC T ON I.MATHUOC = T.ID JOIN TOATHUOC TT ON I.MATOA = TT.ID; OPEN cur; FETCH NEXT FROM cur INTO @MATHUOC, @HSD, @NGAYTOATHUOC; WHILE @@FETCH_STATUS = 0 BEGIN </pre>		

	<pre> IF @HSD <= @NGAYTOATHUOC BEGIN PRINT N'Hạn sử dụng của thuốc không hợp lệ'; ROLLBACK TRANSACTION; CLOSE cur; DEALLOCATE cur; RETURN; END FETCH NEXT FROM cur INTO @MATHUOC, @HSD, @NGAYTOATHUOC; END CLOSE cur; DEALLOCATE cur; PRINT 'Thành công'; END; </pre>		
	<u>Mô tả mã lệnh</u> Trigger TRIG_HSD trên bảng CTDONTHUOC: Kích hoạt sau thao tác INSERT và UPDATE. Khai báo biến: @MATHUOC INT: Lưu trữ mã thuốc. @NGAYTOATHUOC DATE: Lưu trữ ngày lập toa thuốc. @HSD DATE: Lưu trữ hạn sử dụng của thuốc. Tạo và mở cursor: Lấy các giá trị MATHUOC, HSD, và NGAYLAP từ các bản ghi mới chèn hoặc cập nhật. Duyệt qua các bản ghi với cursor: Kiểm tra nếu @HSD nhỏ hơn hoặc bằng @NGAYTOATHUOC: In thông báo lỗi: "Hạn sử dụng của thuốc không hợp lệ". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Đóng và giải phóng cursor: CLOSE cur; DEALLOCATE cur;. Dừng xử lý tiếp: RETURN. Đóng và giải phóng cursor: CLOSE cur; DEALLOCATE cur; In thông báo thành công: PRINT 'Thành công';		
2.9	trg_NhanVien_Update_TaiKhoan	UPDATE	NHANVIEN
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER trg_NhanVien_Update_TaiKhoan ON NhanVien </pre>		

	<pre> AFTER UPDATE AS BEGIN DISABLE TRIGGER trg_TaiKhoan_Update_NhanVien ON TaiKhoan; BEGIN TRY UPDATE tk SET tk.hoTen = i.HOTEN FROM TaiKhoan tk INNER JOIN inserted i ON tk.maSoNV= i.id END TRY BEGIN CATCH PRINT 'Error occurred in trg_NhanVien_Update_TaiKhoan'; ROLLBACK TRANSACTION; END CATCH; ENABLE TRIGGER trg_TaiKhoan_Update_NhanVien ON TaiKhoan; END; </pre>		
	<u>Mô tả mã lệnh</u> Trigger trg_NhanVien_Update_TaiKhoan trên bảng NhanVien: Kích hoạt sau thao tác UPDATE. Tắt trigger: Tắt trigger trg_TaiKhoan_Update_NhanVien trên bảng TaiKhoan để tránh vòng lặp. Bắt đầu khối TRY-CATCH: Khởi TRY: Cập nhật thông tin tài khoản (hoTen) trong bảng TaiKhoan với hoTen từ bảng inserted cho những tài khoản có maSoNV khớp với id của nhân viên (NhanVien). Khởi CATCH: In thông báo lỗi: "Error occurred in trg_NhanVien_Update_TaiKhoan". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Bật lại trigger: Bật lại trigger trg_TaiKhoan_Update_NhanVien trên bảng TaiKhoan.		
2.10	trg_TaiKhoan_Update_NhanVien	UPDATE	TAIKHOAN
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER trg_TaiKhoan_Update_NhanVien ON TaiKhoan AFTER UPDATE AS BEGIN -- Tắt trigger trg_NhanVien_Update_TaiKhoan để tránh vòng lặp DISABLE TRIGGER trg_NhanVien_Update_TaiKhoan ON NhanVien; </pre>		

	<pre> BEGIN TRY -- Cập nhật thông tin bệnh nhân UPDATE NHANVIEN SET HOTEN = i.hoTen FROM NHANVIEN nv INNER JOIN inserted i ON nv.id = i.maSoNV; END TRY BEGIN CATCH -- Xử lý lỗi nếu cần thiết PRINT 'Error occurred in trg_TaiKhoan_Update_NhanVien'; ROLLBACK TRANSACTION END CATCH; -- Bật lại trigger trg_NhanVien_Update_TaiKhoan ENABLE TRIGGER trg_NhanVien_Update_TaiKhoan ON NhanVien; END; </pre>		
	<u>Mô tả mã lệnh</u> Trigger trg_TaiKhoan_Update_NhanVien trên bảng TaiKhoan: Kích hoạt sau thao tác UPDATE. Tắt trigger: Tắt trigger trg_NhanVien_Update_TaiKhoan trên bảng NhanVien để tránh vòng lặp. Bắt đầu khối TRY-CATCH: Khởi TRY: Cập nhật thông tin nhân viên (HOTEN) trong bảng NHANVIEN với hoTen từ bảng inserted cho những nhân viên có id khớp với maSoNV trong bảng TaiKhoan. Khởi CATCH: In thông báo lỗi: "Error occurred in trg_TaiKhoan_Update_NhanVien". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Bật lại trigger: Bật lại trigger trg_NhanVien_Update_TaiKhoan trên bảng NhanVien.		
2.11	trg_BenhNhan_Update_TaiKhoan	UPDATE	BENHNHAN
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER trg_BenhNhan_Update_TaiKhoan ON BenhNhan AFTER UPDATE AS BEGIN DISABLE TRIGGER trg_TaiKhoan_Update_BenhNhan ON TaiKhoan; BEGIN TRANSACTION </pre>		

	<pre>BEGIN TRY -- Chỉ cập nhật những tài khoản có maSoBN khớp với id của bệnh nhân UPDATE TaiKhoan SET hoTen = i.tenBN, soDienThoai = i.sdt FROM TaiKhoan tk INNER JOIN inserted i ON tk.maSoBN = i.id; COMMIT TRANSACTION; END TRY BEGIN CATCH PRINT 'Error occurred in trg_BenhNhan_Update_TaiKhoan'; ROLLBACK TRANSACTION END CATCH; ENABLE TRIGGER trg_TaiKhoan_Update_BenhNhan ON TaiKhoan; END;</pre>		
	<u>Mô tả mã lệnh</u> Trigger trg_BenhNhan_Update_TaiKhoan trên bảng BenhNhan: Kích hoạt sau thao tác UPDATE. Tắt trigger: Tắt trigger trg_TaiKhoan_Update_BenhNhan trên bảng TaiKhoan để tránh vòng lặp. Bắt đầu giao dịch: Khởi TRY: Cập nhật thông tin tài khoản (hoTen, soDienThoai) trong bảng TaiKhoan với thông tin từ bảng inserted cho những tài khoản có maSoBN khớp với id của bệnh nhân (BenhNhan). Hoàn thành giao dịch: COMMIT TRANSACTION. Khởi CATCH: In thông báo lỗi: "Error occurred in trg_BenhNhan_Update_TaiKhoan". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Bật lại trigger: Bật lại trigger trg_TaiKhoan_Update_BenhNhan trên bảng TaiKhoan.		
2.12	<u>trg_TaiKhoan_Update_BenhNhan</u>	<u>UPDATE</u>	<u>TAIKHOAN</u>
	<u>Mã lệnh T-SQL</u> <pre>CREATE TRIGGER trg_TaiKhoan_Update_BenhNhan ON TaiKhoan AFTER UPDATE AS BEGIN --BEGIN TRANSACTION; DISABLE TRIGGER trg_BenhNhan_Update_TaiKhoan ON BenhNhan;</pre>		

	<pre> BEGIN TRY UPDATE BenhNhan SET tenBN = i.hoTen FROM BenhNhan bn INNER JOIN inserted i ON bn.id = i.maSoBN; END TRY BEGIN CATCH -- Xử lý lỗi nếu cần thiết PRINT 'Error occurred in trg_TaiKhoan_Update_NhanVien'; ROLLBACK TRANSACTION END CATCH; -- Bật lại trigger trg_NhanVien_Update_TaiKhoan ENABLE TRIGGER trg_BenhNhan_Update_TaiKhoan ON BenhNhan; END; </pre>		
	<u>Mô tả mã lệnh</u> Trigger trg_TaiKhoan_Update_BenhNhan trên bảng TaiKhoan: Kích hoạt sau thao tác UPDATE. Tắt trigger: Tắt trigger trg_BenhNhan_Update_TaiKhoan trên bảng BenhNhan để tránh vòng lặp. Bắt đầu khối TRY-CATCH: Khởi TRY: Cập nhật thông tin bệnh nhân (tenBN) trong bảng BenhNhan với hoTen từ bảng inserted cho những bệnh nhân có id khớp với maSoBN trong bảng TaiKhoan. Khởi CATCH: In thông báo lỗi: "Error occurred in trg_TaiKhoan_Update_NhanVien". Hủy bỏ giao dịch: ROLLBACK TRANSACTION. Bật lại trigger: Bật lại trigger trg_BenhNhan_Update_TaiKhoan trên bảng BenhNhan.		
2.13	<u>TRG_HANSĐ_THUOC</u>	<u>INSERT, UPDATE</u>	<u>CTDONTHUOC</u>
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER TRG_HANSĐ_THUOC ON CTDONTHUOC FOR INSERT, UPDATE AS BEGIN BEGIN TRY BEGIN TRANSACTION; </pre>		

	<pre> DECLARE @MATHUOC INT, @MATOA INT, @HSD SMALLDATETIME, @NGAYLAP SMALLDATETIME; SELECT @MATOA = MATOA, @MATHUOC = MATHUOC FROM INSERTED; SELECT @HSD = HSD FROM THUOC WHERE ID = @MATHUOC; SELECT @NGAYLAP = NGAYLAP FROM TOATHUOC WHERE ID = @MATOA; IF (@NGAYLAP > @HSD) BEGIN ROLLBACK TRANSACTION; THROW 51001, N'Thuốc đã hết hạn sử dụng, chọn thuốc khác', 1; END; COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(); END CATCH END GO </pre>
	<u>Mô tả mã lệnh</u> Trigger TRG_HANSĐ_THUOC trên bảng CTDONTHUOC: Kích hoạt sau thao tác INSERT hoặc UPDATE. Khối TRY-CATCH: Khối TRY: Bắt đầu giao dịch: BEGIN TRANSACTION. Khai báo biến: @MATHUOC: Lưu trữ mã thuốc. @MATOA: Lưu trữ mã toa. @HSD: Lưu trữ hạn sử dụng của thuốc. @NGAYLAP: Lưu trữ ngày lập toa thuốc. Lấy dữ liệu từ bảng INSERTED:

	Gán giá trị MATOA và MATHUOC từ bảng INSERTED vào các biến tương ứng. Lấy hạn sử dụng của thuốc từ bảng THUOC: Gán giá trị HSD từ bảng THUOC vào biến @HSD dựa trên @MATHUOC. Lấy ngày lập từ bảng TOATHUOC: Gán giá trị NGAYLAP từ bảng TOATHUOC vào biến @NGAYLAP dựa trên @MATOA. Kiểm tra hạn sử dụng: Nếu NGAYLAP lớn hơn HSD, hủy bỏ giao dịch và ném lỗi với thông báo: "Thuốc đã hết hạn sử dụng, chọn thuốc khác". Hoàn thành giao dịch: COMMIT TRANSACTION. Khởi CATCH: Nếu có lỗi, hủy bỏ giao dịch nếu đang có giao dịch mở (IF @@TRANCOUNT > 0). In thông báo lỗi: "Lỗi xảy ra: " kèm theo thông báo lỗi chi tiết từ ERROR_MESSAGE().
2.14	
	<u>Mã lệnh T-SQL</u> <pre> CREATE TRIGGER TRG_MABS ON TOATHUOC FOR INSERT, UPDATE AS BEGIN BEGIN TRY BEGIN TRANSACTION; DECLARE @MABS INT, @VAITRO NVARCHAR(20); SELECT @MABS = MABS FROM INSERTED; SELECT @VAITRO = VAITRO FROM NHANVIEN WHERE ID = @MABS; IF (@VAITRO <> N'Bác sĩ') BEGIN ROLLBACK TRANSACTION; THROW 51002, N'Toa thuốc chỉ được tạo bởi bác sĩ', 1; END; COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 </pre>

	ROLLBACK TRANSACTION; PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(); END CATCH END GO		
	<u>Mô tả mã lệnh</u> Trigger TRG_MABS trên bảng TOATHUOC: Kích hoạt sau thao tác INSERT hoặc UPDATE. Khởi TRY-CATCH: Khởi TRY: Bắt đầu giao dịch: BEGIN TRANSACTION. Khai báo biến: @MABS: Lưu trữ mã bác sĩ. @VAITRO: Lưu trữ vai trò của nhân viên. Lấy dữ liệu từ bảng INSERTED: Gán giá trị MABS từ bảng INSERTED vào biến @MABS. Lấy vai trò của bác sĩ từ bảng NHANVIEN: Gán giá trị VAITRO từ bảng NHANVIEN vào biến @VAITRO dựa trên @MABS. Kiểm tra vai trò: Nếu vai trò (VAITRO) không phải là "Bác sĩ", hủy bỏ giao dịch và ném lỗi với thông báo: "Toa thuốc chỉ được tạo bởi bác sĩ". Hoàn thành giao dịch: COMMIT TRANSACTION. Khởi CATCH: Nếu có lỗi, hủy bỏ giao dịch nếu đang có giao dịch mở (IF @@TRANCOUNT > 0). In thông báo lỗi: "Lỗi xảy ra: " kèm theo thông báo lỗi chi tiết từ ERROR_MESSAGE().		
2.15	TRIG_UPDATE_THUOC	INSERT	CTDONTHUOC
	<u>Mã lệnh T-SQL</u> CREATE TRIGGER TRIG_UPDATE_THUOC ON CTDONTHUOC AFTER INSERT AS		

	<pre> BEGIN SET NOCOUNT ON; DECLARE @MATHUOC INT, @SOLUONG INT; SELECT @MATHUOC = i.MATHUOC, @SOLUONG = i.SOLUONG FROM inserted i; UPDATE THUOC SET SOLUONGTON = SOLUONGTON - @SOLUONG WHERE id = @MATHUOC; IF EXISTS (SELECT 1 FROM THUOC WHERE id = @MATHUOC AND SOLUONGTON <= 0) BEGIN UPDATE THUOC SET tinhTrang = N'Đã hết thuốc' WHERE id = @MATHUOC; END END; </pre>
	<u>Mô tả mã lệnh:</u> Kích hoạt sau thao tác INSERT. Ngăn chặn thông báo đếm số hàng: SET NOCOUNT ON. Khai báo biến: @MATHUOC: Lưu trữ mã thuốc. @SOLUONG: Lưu trữ số lượng thuốc. Lấy dữ liệu từ bảng INSERTED: Gán giá trị MATHUOC và SOLUONG từ bảng INSERTED vào các biến @MATHUOC và @SOLUONG. Cập nhật bảng THUOC: Giảm số lượng tồn kho (SOLUONGTON) của thuốc dựa trên @SOLUONG. Kiểm tra số lượng tồn kho: Nếu số lượng tồn kho (SOLUONGTON) nhỏ hơn hoặc bằng 0, cập nhật trạng thái của thuốc thành "Đã hết thuốc".
2.16	<pre> TRG_NGAYTAO_PKB INSERT, UPDATE PHIEUKHAMBENH </pre>
	<u>Mã lệnh T-SQL:</u> <pre> CREATE OR ALTER TRIGGER TRG_NGAYTAO_PKB </pre>

	<pre> ON PHIEUKHAMBENH FOR INSERT, UPDATE AS BEGIN BEGIN TRY BEGIN TRANSACTION; DECLARE @ngaytao smalldatetime, @mabs int, @ngayvl smalldatetime; SELECT @ngaytao = NGAYTAO, @mabs = MABS FROM inserted; SELECT @ngayvl = NGAYVL FROM NHANVIEN WHERE id = @mabs; IF (@ngaytao < @ngayvl) BEGIN ROLLBACK TRANSACTION; THROW 51000, N'NGAYTAO phải lớn hơn NGAYVL của bác sĩ', 1; END; COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(); END CATCH; END; GO </pre>
	Mô tả mã lệnh: Trigger TRG_NGAYTAO_PKB trên bảng PHIEUKHAMBENH Kích hoạt sau thao tác INSERT hoặc UPDATE Khởi TRY-CATCH: Khởi TRY: BEGIN TRANSACTION để đảm bảo tính toàn vẹn dữ liệu Khai báo biến: @ngaytao smalldatetime, lưu trữ ngày tạo phiếu khám bệnh @mabs, int, lưu trữ mã bác sĩ @ngayvl, smalldatetime, lưu trữ ngày vào làm của bác sĩ Lấy dữ liệu từ bảng INSERTED:

	Nạp vào @ngaytao và @mabs giá trị NGAYTAO và MABS từ bảng INSERTED Lấy dữ liệu từ bảng NHANVIEN: Nạp vào @ngayvl giá trị NGAYVL từ bảng NHANVIEN sao cho ID = @mabs Thực hiện kiểm tra điều kiện @ngaytao < @ngayvl: Nếu cho kết quả đúng, thực hiện ROLLBACK và thông báo lỗi. Nếu không, COMMIT để hoàn tất giao tác. Khối CATCH: Chuyển sang khối CATCH khi có phát hiện lỗi Nếu phát hiện có giao dịch đang mở (bằng cách kiểm tra giá trị biến hệ thống @@TRANCOUNT - nếu có ít nhất 1 giá trị sẽ dương) thì thực hiện ROLLBACK Thông báo lỗi ra màn hình
2.17	TRG_TONGTIEN_TOATHUOC INSERT, DELETE CTDONTHUOC
	<u>Mã lệnh T-SQL:</u> CREATE OR ALTER TRIGGER TRG_TONGTIEN_TOATHUOC ON CTDONTHUOC AFTER INSERT, DELETE -- Thêm DELETE vào danh sách các loại hoạt động AS BEGIN BEGIN TRY BEGIN TRANSACTION; DECLARE @MATOA INT, @MATHUOC INT, @SOLUONG INT, @DONGIA MONEY, @TONGTIEN MONEY; IF EXISTS (SELECT * FROM inserted) BEGIN SELECT @MATOA = MATOA, @MATHUOC = MATHUOC, @SOLUONG = SOLUONG FROM inserted; SELECT @DONGIA = DONGIA FROM THUOC WHERE id = @MATHUOC; SELECT @TONGTIEN = TONGTIEN FROM TOATHUOC WHERE id = @MATOA; IF (@TONGTIEN IS NULL) SET @TONGTIEN = 0; UPDATE TOATHUOC


```

SET TONGTIEN = @TONGTIEN + (@SOLUONG * @DONGIA)
WHERE id = @MATOA;
END

IF EXISTS (SELECT * FROM deleted)
BEGIN
    SELECT @MATOA = MATOA, @MATHUOC = MATHUOC, @SOLUONG
= SOLUONG
    FROM deleted;

    SELECT @DONGIA = DONGIA
    FROM THUOC
    WHERE id = @MATHUOC;

    SELECT @TONGTIEN = TONGTIEN
    FROM TOATHUOC
    WHERE id = @MATOA;

    IF (@TONGTIEN IS NOT NULL)
    BEGIN
        UPDATE TOATHUOC
        SET TONGTIEN = @TONGTIEN - (@SOLUONG * @DONGIA)
        WHERE id = @MATOA;
    END
END

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE();
END CATCH
END
GO

```

Mô tả mã lệnh:

Trigger TRG_TONGTIEN_TOATHUOC trên bảng CTDONTHUOC

Kích hoạt sau thao tác INSERT hoặc DELETE

Khởi TRY-CATCH:

Khởi TRY:

BEGIN TRANSACTION để đảm bảo tính toàn vẹn của dữ liệu khi thực hiện cập nhật

Khai báo biến:

@MATOA, kiểu dữ liệu int, lưu trữ mã toa thuốc

@MATHUOC, kiểu dữ liệu int, lưu trữ mã thuốc

@SOLUONG, kiểu dữ liệu int, lưu trữ số lượng thuốc trong CT đơn thuốc

	@DONGIA, kiểu dữ liệu money, lưu trữ đơn giá thuốc @TONGTIEN, kiểu dữ liệu money, lưu trữ tổng tiền thuốc của một toa Nếu đang thực hiện INSERT: Nạp vào @MATOA, @MATHUOC và @SOLUONG giá trị MATOA, MATHUOC, SOLUONG từ bảng ghi CTDONTHUOC đang được thêm vào (INSERTED) Nạp vào @DONGIA giá trị DONGIA từ bảng THUOC có @MATHUOC Nạp vào @TONGTIEN giá trị TONGTIEN từ bảng TOATHUOC. Nếu TONGTIEN NULL thì SET TONGTIEN = 0 Thực hiện tính giá trị của CT đơn thuốc bằng tích của số lượng và đơn giá của thuốc. Cập nhật TOATHUOC, SET giá trị của TONGTIEN = @TONGTIEN + @SOLUONG * @DONGIA Nếu đang thực hiện DELETE: Nạp vào @MATOA, @MATHUOC và @SOLUONG giá trị MATOA, MATHUOC, SOLUONG từ bảng ghi CTDONTHUOC đang được xóa (DELETED) Nạp vào @DONGIA giá trị DONGIA từ bảng THUOC có @MATHUOC Nếu @TONGTIEN khác NULL, nạp vào @TONGTIEN giá trị TONGTIEN từ bảng TOATHUOC. Thực hiện tính giá trị của CT đơn thuốc bằng tích của số lượng và đơn giá của thuốc. Cập nhật TOATHUOC, SET giá trị của TONGTIEN = @TONGTIEN - @SOLUONG * @DONGIA COMMIT để hoàn tất giao tác Khởi CATCH: Chuyển sang khối CATCH khi có phát hiện lỗi Nếu phát hiện có giao dịch đang mở (bằng cách kiểm tra giá trị biến hệ thống @@TRANCOUNT - nếu có ít nhất 1 giá trị sẽ dương) thì thực hiện ROLLBACK Thông báo lỗi ra màn hình
2.18	TRG_NGAYTAO_BIENLAI INSERT, UPDATE BIENLAI
	<u>Mã lệnh T-SQL:</u> CREATE OR ALTER TRIGGER TRG_NGAYTAO_BIENLAI ON BIENLAI FOR INSERT, UPDATE AS BEGIN BEGIN TRY BEGIN TRANSACTION; DECLARE @MANV_TT INT, @NGAYTAO DATETIME, @NGAYVL SMALLDATETIME; SELECT @MANV_TT = MANV_TT, @NGAYTAO = NGAYTAO FROM INSERTED

```

IF NOT EXISTS (SELECT * FROM NHANVIEN WHERE ID = @MANV_TT AND
VAITRO = N'NV Thanh toán')
BEGIN
ROLLBACK TRANSACTION;
THROW 51007, N'Không tồn tại nhân viên với ID trên', 1;
END
ELSE
BEGIN
SELECT @NGAYVL = NGAYVL
FROM NHANVIEN
WHERE ID = @MANV_TT;

IF(@NGAYTAO < @NGAYVL)
BEGIN
ROLLBACK TRANSACTION;
THROW 51008, N'NGAYTAO biên lai phải lớn hơn NGAYVL của NV_TT', 1;
END
END
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0
ROLLBACK TRANSACTION;
PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE();
END CATCH
END
GO

```

Mô tả mã lệnh:

Trigger TRG_NGAYTAO_BIENLAI trên bảng BIENLAI

Kích hoạt sau khi INSERT

Khởi lệnh TRY-CATCH:

Khởi lệnh TRY:

Khai báo biến:

@MANV_TT, kiểu dữ liệu int, lưu trữ mã nhân viên thanh toán

@NGAYTAO, kiểu dữ liệu smalldatetime, lưu trữ ngày tạo biên lai

@NGAYVL kiểu dữ liệu smalldatetime, lưu trữ ngày vào làm của nhân viên thanh toán

Nạp vào @MANV_TT, @NGAYTAO giá trị MANV_TT và NGAYTAO từ bảng ghi đang được thêm vào (INSERTED).

Nếu không tồn tại MANV_TT trong NHANVIEN thì ROLLBACK và thông báo lỗi

Nạp vào @NGAYVL giá trị NGAYVL của bảng ghi NHANVIEN có @MANV_TT

Thực hiện kiểm tra điều kiện:

Nếu @NGAYTAO < @NGAYVL, thực hiện ROLLBACK và thông báo lỗi

COMMIT khi hoàn tất giao tác

Khởi CATCH:

Chuyển sang khối CATCH khi có phát hiện lỗi

	Nếu phát hiện có giao dịch đang mở (bằng cách kiểm tra giá trị biến hệ thống @@TRANCOUNT - nếu có ít nhất 1 giá trị sẽ dương) thì thực hiện ROLLBACK Thông báo lỗi ra màn hình
2.19	TRG_SET_TRANGTHAI INSERT NHANVIEN
	Mã lệnh T-SQL: <pre>CREATE TRIGGER TRG_SET_TRANGTHAI ON NHANVIEN AFTER INSERT AS BEGIN -- Set NOCOUNT to ON to prevent extra result sets from interfering with SELECT statements. SET NOCOUNT ON; -- Update the TRANGTHAI column to 'Đang tồn tại' for newly inserted rows UPDATE NHANVIEN SET TRANGTHAI = N'Đang tồn tại' WHERE id IN (SELECT id FROM INSERTED); END;</pre> Mô tả mã lệnh: Trigger TRG_SET_TRANGTHAI: Kích hoạt sau INSERT SET NOCOUNT ON: Thiết lập NOCOUNT thành ON để ngăn các kết quả phụ xuất phát từ các câu lệnh SELECT can thiệp vào. UPDATE NHANVIEN SET TRANGTHAI = N'Đang tồn tại' WHERE id IN (SELECT id FROM INSERTED): Cập nhật cột TRANGTHAI của bảng NHANVIEN cho các hàng mới được chèn vào với giá trị 'Đang tồn tại'.
2.20	Trg_UpdateThuocOnInsert INSERT CTDONTHUOC
	Mã lệnh T-SQL: <pre>CREATE TRIGGER trg_UpdateThuocOnInsert ON CTDONTHUOC AFTER INSERT AS BEGIN SET NOCOUNT ON; DECLARE @MATHUOC INT, @SOLUONG INT; SELECT @MATHUOC = i.MATHUOC, @SOLUONG = i.SOLUONG FROM inserted i; UPDATE THUOC SET SOLUONGTON = SOLUONGTON - @SOLUONG WHERE id = @MATHUOC; IF EXISTS (</pre>

<pre> SELECT 1 FROM THUOC WHERE id = @MATHUOC AND SOLUONGTON <= 0) BEGIN UPDATE THUOC SET TRANGTHAI = N'Đã hết thuốc' WHERE id = @MATHUOC; END END; </pre> Mô tả mã lệnh: Trigger trg_UpdateThuocOnInsert trên bảng CTDonThuoc được kích hoạt sau INSERT Phương thức hoạt động: Khai báo biến: @MATHUOC: lưu trữ mã thuốc; @SOLUONG: lưu trữ số lượng của dòng được chèn mới. Gán giá trị của cột MATHUOC và SOLUONG của dòng mới được chèn vào biến cục bộ tương ứng. Cập nhật cột SOLUONGTON của bảng THUOC bằng cách giảm số lượng tồn đi theo số lượng của thuốc được chèn mới. Cập nhật được thực hiện dựa trên ID của thuốc được chèn. Kiểm tra nếu số lượng tồn của thuốc đó sau khi giảm bằng hoặc nhỏ hơn 0. BEGIN: Bắt đầu khối mã được thực thi nếu điều kiện trong IF là đúng. Cập nhật cột TRANGTHAI của bảng THUOC thành 'Đã hết thuốc' cho thuốc có ID tương ứng với @MATHUOC, nếu số lượng tồn của thuốc đó sau khi giảm bằng hoặc nhỏ hơn 0.
--

3. Xây dựng và mô tả Procedure, Function trong đồ án môn học

3.1 Procedure

STT	Procedure
3.1.1	<u>Tên procedure:</u> InsertBenhNhan <u>Mã lệnh T-SQL:</u> <pre> CREATE PROCEDURE InsertBenhNhan @tenBN NVARCHAR(50), @sdt VARCHAR(10), @ngaySinh SMALLDATETIME, @diaChi NVARCHAR(50), @gioiTinh NVARCHAR(5), @queQuan NVARCHAR(20), </pre>

	<pre> @ghichu NVARCHAR(50) = NULL AS BEGIN SET NOCOUNT ON; BEGIN TRY BEGIN TRANSACTION; INSERT INTO BenhNhan (tenBN, sdt, ngaySinh, diaChi, gioiTinh, queQuan, ghichu) VALUES (@tenBN, @sdt, @ngaySinh, @diaChi, @gioiTinh, @queQuan, @ghichu); PRINT N'Thêm bệnh nhân thành công'; COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT N'Thêm bệnh nhân không thành công'; END CATCH; END; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh trên tạo ra một thủ tục đặt tên là InsertBenhNhan, dùng để thêm một bản ghi bệnh nhân mới vào bảng BenhNhan. Các thông số đầu vào của thủ tục gồm: tên bệnh nhân, số điện thoại, ngày sinh, địa chỉ, giới tính, quê quán, và ghi chú. Dùng lệnh SET NOCOUNT ON để dừng việc gửi thông báo đến máy khách về số hàng bị ảnh hưởng bởi câu lệnh T-SQL Khối lệnh BEGIN TRY ... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác chèn dữ liệu. BEGIN TRANSACTION: Bắt đầu một giao dịch cơ sở dữ liệu. INSERT INTO: Thêm một bản ghi bệnh nhân mới vào bảng BenhNhan từ các tham số đã cung cấp. PRINT N'Thêm bệnh nhân thành công': Thực hiện in thông báo trên console khi thêm bệnh nhân mới thành công. COMMIT TRANSACTION: Thực hiện lưu vĩnh viễn dữ liệu nếu thêm thành công. Khối lệnh BEGIN CATCH ... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY.

	Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. IF @@TRANCOUNT >0: Có ít nhất một giao dịch đang mở, và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK. ROLLBACK TRANSACTION: Hoàn tác tất cả thay đổi được thực hiện kể từ lúc giao dịch bắt đầu và đưa cơ sở dữ liệu về trạng thái ban đầu. PRINT N'Thêm bệnh nhân không thành công': Sau khi thực hiện rollback thì in thông báo trên console khi thêm bệnh nhân mới không thành công.
3.1.2	<u>Tên procedure:</u> InsertNhanVien <u>Mã lệnh T-SQL:</u> <pre> CREATE PROCEDURE InsertNhanVien @HOTEN NVARCHAR(50), @NGAYSINH SMALLDATETIME, @DIACHI NVARCHAR(50), @GIOITINH NVARCHAR(5), @NGAYVL SMALLDATETIME, @VAITRO NVARCHAR(20), @TRANGTHAI NVARCHAR(20) AS BEGIN SET NOCOUNT ON; BEGIN TRY BEGIN TRANSACTION; INSERT INTO NHANVIEN (HOTEN, NGAYSINH, DIACHI, GIOITINH, NGAYVL, VAITRO,TRANGTHAI) VALUES (@HOTEN, @NGAYSINH, @DIACHI, @GIOITINH, @NGAYVL, @VAITRO,N'DANG TON TẠI'); PRINT N'Thêm nhân viên thành công'; COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT N'Thêm nhân viên không thành công'; END CATCH; END; </pre>

	GO
	<u>Mô tả mã lệnh:</u> Mã lệnh trên tạo ra một thủ tục đặt tên là InsertNhanVien, dùng để thêm một bản ghi nhân viên mới vào bảng NHANVIEN. Các thông số đầu vào của thủ tục gồm: họ tên nhân viên, ngày sinh, địa chỉ, giới tính, ngày vào làm, vai trò và trạng thái. Vai trò bao gồm: bác sỹ, bệnh nhân, nhân viên tiếp nhận, và nhân viên thanh toán. Dùng lệnh SET NOCOUNT ON để dừng việc gửi thông báo đến máy khách về số hàng bị ảnh hưởng bởi câu lệnh T-SQL Khởi lệnh BEGIN TRY ... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác chèn dữ liệu. BEGIN TRANSACTION: Bắt đầu một giao dịch cơ sở dữ liệu. INSERT INTO: Thêm một bản ghi nhân viên mới vào bảng NHANVIEN từ các tham số đã cung cấp. PRINT N'Thêm nhân viên thành công': Thực hiện in thông báo trên console khi thêm nhân viên mới thành công. COMMIT TRANSACTION: Thực hiện lưu vĩnh viễn dữ liệu nếu thêm thành công. Khởi lệnh BEGIN CATCH ... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY. Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. IF @@TRANCOUNT >0: Có ít nhất một giao dịch đang mở, và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK. ROLLBACK TRANSACTION: Hoàn tác tất cả thay đổi được thực hiện kể từ lúc giao dịch bắt đầu và đưa cơ sở dữ liệu về trạng thái ban đầu. PRINT N'Thêm nhân viên không thành công': Sau khi thực hiện rollback thì in thông báo trên console khi thêm nhân viên mới không thành công.
3.1.3	<u>Tên procedure:</u> InsertPhieuKhamBenh <u>Mã lệnh T-SQL:</u> CREATE PROCEDURE InsertPhieuKhamBenh @MABS INT,

	<pre> @MABN INT, @MAPK INT, @NGAYTAO SMALLDATETIME, @CHANDOAN NVARCHAR(50) AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION; BEGIN TRY SELECT @MABS = id FROM NHANVIEN WHERE id = @MABS AND VAITRO = 'Bác sỹ'; SELECT @MABN = id FROM BenhNhan WHERE id = @MABN; IF @MABS IS NULL OR @MABN IS NULL BEGIN THROW 51000, N'Mã bác sỹ và mã bệnh nhân không tồn tại', 1; END; INSERT INTO PHIEUKHAMBENH (MABS, MABN, MAPK, NGAYTAO, CHANDOAN) VALUES (@MABS, @MABN, @MAPK, @NGAYTAO, @CHANDOAN); COMMIT TRANSACTION; PRINT N'Thêm phiếu khám bệnh thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT N'Thêm phiếu khám bệnh thất bại'; END CATCH; END; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh trên tạo ra một thủ tục đặt tên là InsertPhieuKhamBenh, dùng để tạo ra một phiếu khám bệnh và thêm vào bảng PHIEUKHAMBENH.

	Các thông số đầu vào của thủ tục gồm: mã bác sĩ, mã bệnh nhân, mã phiếu khám, ngày tạo, và chuẩn đoán. Dùng lệnh SET NOCOUNT ON để dừng việc gửi thông báo đến máy khách về số hàng bị ảnh hưởng bởi câu lệnh T-SQL Khởi lệnh BEGIN TRY ... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác chèn dữ liệu. BEGIN TRANSACTION: Bắt đầu một giao dịch cơ sở dữ liệu. Dùng câu lệnh SELECT để truy vấn mã bác sĩ và mã bệnh nhân trong cơ sở dữ liệu. Kiểm tra xem @MABS có tồn tại trong bảng NHANVIEN với VaiTro là bác sĩ. Kiểm tra xem @MABN có tồn tại trong bảng BenhNhan. IF @MABS IS NULL OR @MABN IS NULL: Nếu một trong hai giá trị bác sĩ và bệnh nhân không tồn tại, thực hiện THROW ném ra lỗi và kết thúc thủ tục. INSERT INTO: Thêm một bản ghi phiếu khám bệnh mới vào bảng PHIEUKHAMBENH từ các tham số đã cung cấp. PRINT N'Thêm phiếu khám bệnh thành công': Thực hiện in thông báo trên console khi thêm phiếu khám bệnh mới thành công. COMMIT TRANSACTION: Thực hiện lưu vĩnh viễn dữ liệu nếu thêm thành công. Khởi lệnh BEGIN CATCH ... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY. Khối catch sẽ bắt được lỗi THROW từ khối try và thực hiện rollback. Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. IF @@TRANCOUNT >0: Có ít nhất một giao dịch đang mở, và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK. ROLLBACK TRANSACTION: Hoàn tác tất cả thay đổi được thực hiện kể từ lúc giao dịch bắt đầu và đưa cơ sở dữ liệu về trạng thái ban đầu. PRINT N'Thêm phiếu khám bệnh không thành công': Sau khi thực hiện rollback thì in thông báo trên console khi thêm phiếu khám bệnh mới không thành công.
3.1.4	<u>Tên procedure:</u> InsertPhongKham <u>Mã lệnh T-SQL:</u> <pre>CREATE PROCEDURE ThemPhongKham @TENPK NVARCHAR(50),</pre>

	<pre> @TENKHOA NVARCHAR(50), @THOIGIANBD TIME, @THOIGIANKT TIME AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION; BEGIN TRY IF @THOIGIANBD >= @THOIGIANKT BEGIN RAISERROR ('Thời gian bắt đầu phải nhỏ hơn thời gian kết thúc', 16,1) RETURN; END INSERT INTO PHONGKHAM (TENPK, TENKHOA, THOIGIANBD, THOIGIANKT) VALUES (@TENPK, @TENKHOA, @THOIGIANBD, @THOIGIANKT) COMMIT TRANSACTION PRINT N'Thêm phòng khám thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT >0 ROLLBACK TRANSACTION PRINT N'Thêm phòng khám thất bại. ' + ERROR_MESSAGE(); END CATCH END; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh trên tạo ra một thủ tục đặt tên là InsertPhongKham, dùng để tạo ra một phòng khám và thêm vào bảng PHONGKHAM. Các thông số đầu vào của thủ tục gồm: tên phòng khám, tên khoa, thời gian bắt đầu và thời gian kết thúc. Dùng lệnh SET NOCOUNT ON để dừng việc gửi thông báo đến máy khách về số hàng bị ảnh hưởng bởi câu lệnh T-SQL Khởi lệnh BEGIN TRY ... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác chèn dữ liệu. IF @THOIGIANBD >= @THOIGIANKT: Kiểm tra thời gian bắt đầu làm việc của phòng khám có lớn hơn

	thời gian kết thúc làm việc hay không. Nếu nhỏ hơn thì ném ra lỗi RAISERROR, lỗi này được xử lý trong khối catch. INSERT INTO: Thêm một bản ghi phòng khám mới vào bảng PHONGKHAM từ các tham số đã cung cấp. PRINT N'Thêm phòng khám thành công': Thực hiện in thông báo trên console khi thêm phòng khám thành công. COMMIT TRANSACTION: Thực hiện lưu vĩnh viễn dữ liệu nếu thêm thành công. Khối lệnh BEGIN CATCH ... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY. Khối catch sẽ bắt được lỗi RAISERROR từ khối try và thực hiện rollback. Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. IF @@TRANCOUNT >0: Có ít nhất một giao dịch đang mở, và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK. ROLLBACK TRANSACTION: Hoàn tác tất cả thay đổi được thực hiện kể từ lúc giao dịch bắt đầu và đưa cơ sở dữ liệu về trạng thái ban đầu. PRINT N'Thêm phòng khám không thành công' + ERROR_MESSAGE (): Sau khi thực hiện rollback thì in thông báo trên console kèm theo thông báo lỗi bắt được từ hệ thống khi thêm phòng khám mới không thành công.
3.1.5	<u>Tên procedure:</u> ThemToaThuoc <u>Mã lệnh T-SQL:</u> <pre> CREATE PROCEDURE ThemToaThuoc @MABS INT, @maBN INT, @TONGTIEN MONEY, @NGAYLAP DATETIME = NULL AS BEGIN SET NOCOUNT ON; IF @NGAYLAP IS NULL BEGIN SET @NGAYLAP = GETDATE() END BEGIN TRANSACTION ThemToaThuocTransaction </pre>

	<pre> BEGIN TRY IF NOT EXISTS (SELECT 1 FROM NHANVIEN WHERE @MABS = id AND VAITRO = N'Bác sĩ') BEGIN RAISERROR ('Mã bác sỹ không tồn tại', 16,1); RETURN; END; IF NOT EXISTS (SELECT 1 FROM BENHNHAN WHERE @MABN = id) BEGIN RAISERROR ('Mã bệnh nhân không tồn tại', 16,1); RETURN; END; IF EXISTS (SELECT 1 FROM TOATHUOC WHERE MABS = @MABS AND maBN = @maBN) BEGIN RAISERROR ('Toa thuốc đã tồn tại.', 16, 1); RETURN; END; INSERT INTO TOATHUOC (MABS, maBN, TONGTIEN, NGAYLAP) VALUES (@MABS, @maBN, @TONGTIEN, @NGAYLAP); COMMIT TRANSACTION ThemToaThuoc PRINT N'Thêm toa thuốc thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION ThemToaThuocTransaction PRINT N'Thêm toa thuốc thất bại. ' + ERROR_MESSAGE(); END CATCH END; </pre>
	<u>Mô tả mã lệnh:</u>

	Mã lệnh trên tạo ra một thủ tục đặt tên là InsertToaThuoc, dùng để tạo ra một toa thuốc và thêm vào bảng TOATHUOC. Các thông số đầu vào của thủ tục gồm: mã bác sĩ, mã bệnh nhân, tổng tiền thuốc, và ngày lập toa thuốc. Dùng lệnh SET NOCOUNT ON để dừng việc gửi thông báo đến máy khách về số hàng bị ảnh hưởng bởi câu lệnh T-SQL Nếu giá trị của @NGAYLAP là NULL, thì nó sẽ được thiết lập là ngày hiện tại lấy từ hệ thống (GETDATE()). Khởi lệnh BEGIN TRY ... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác chèn dữ liệu. Dùng lệnh SELECT để truy vấn mã bác sĩ và mã bệnh nhân trong cơ sở dữ liệu. Nếu mã bác sĩ và mã bệnh nhân không tồn tại thì ném ra thông báo lỗi và kết thúc Dùng lệnh IF EXISTS kiểm tra xem toa thuốc đã tồn tại cho cặp mã bác sĩ và mã bệnh nhân đã cho hay không. Nếu tồn tại, ném ra một lỗi. INSERT INTO: Thêm một bản ghi toa thuốc mới vào bảng TOATHUOC từ các tham số đã cung cấp. PRINT N'Thêm toa thuốc thành công': Thực hiện in thông báo trên console khi thêm toa thuốc thành công. COMMIT TRANSACTION: Thực hiện lưu vĩnh viễn dữ liệu nếu thêm thành công. Khởi lệnh BEGIN CATCH ... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY. Khối catch sẽ bắt được lỗi RAISERROR từ khối try và thực hiện rollback. Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. IF @@TRANCOUNT >0: Có ít nhất một giao dịch đang mở, và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK. ROLLBACK TRANSACTION: Hoàn tác tất cả thay đổi được thực hiện kể từ lúc giao dịch bắt đầu và đưa cơ sở dữ liệu về trạng thái ban đầu. PRINT N'Thêm toa thuốc không thành công' + ERROR_MESSAGE (): Sau khi thực hiện rollback thì in thông báo trên console kèm theo thông báo lỗi bắt được từ hệ thống khi thêm toa thuốc mới không thành công.
3.1.6	<u>Tên procedure:</u> ThemCTDonThuoc <u>Mã lệnh T-SQL:</u> CREATE PROCEDURE ThemCTDonThuoc

	<pre> @MATHUOC INT, @MATOA INT, @SOLUONG INT AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION; BEGIN TRY IF NOT EXISTS (SELECT 1 FROM THUOC WHERE id = @MATHUOC) BEGIN RAISERROR('Mã thuốc không tồn tại trong bảng THUOC.', 16, 1); RETURN; END; IF NOT EXISTS (SELECT 1 FROM TOATHUOC WHERE id = @MATOA) BEGIN RAISERROR('Mã toa thuốc không tồn tại trong bảng TOATHUOC.', 16, 1); RETURN; END; IF EXISTS (SELECT 1 FROM CTDONTHUOC WHERE MATHUOC = @MATHUOC AND MATOA = @MATOA AND SOLUONG = @SOLUONG) BEGIN RAISERROR ('Chi tiết đơn thuốc đã tồn tại.', 16, 1); RETURN; END; -- Chèn dữ liệu vào bảng CTDONTHUOC INSERT INTO CTDONTHUOC (MATHUOC, MATOA, SOLUONG) VALUES (@MATHUOC, @MATOA, @sOLUONG); -- Commit transaction nếu mọi thứ thành công COMMIT TRANSACTION; PRINT N'Thêm chi tiết đơn thuốc thành công.'; END TRY BEGIN CATCH -- Rollback transaction nếu có lỗi IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; </pre>
--	---

	<pre>-- In thông báo lỗi chi tiết PRINT N'Thêm chi tiết đơn thuốc thất bại.' + ERROR_MESSAGE(); END CATCH; END;</pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh trên tạo ra một thủ tục đặt tên là InsertCTDonThuoc, dùng để tạo ra một chi tiết đơn thuốc và thêm vào bảng CTDONTHUOC. Các thông số đầu vào của thủ tục gồm: mã thuốc, mã toa, và số lượng thuốc. Dùng lệnh SET NOCOUNT ON để dừng việc gửi thông báo đến máy khách về số hàng bị ảnh hưởng bởi câu lệnh T-SQL Nếu giá trị của @NGAYLAP là NULL, thì nó sẽ được thiết lập là ngày hiện tại lấy từ hệ thống (GETDATE()). Khởi lệnh BEGIN TRY ... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác chèn dữ liệu. Dùng lệnh IF NOT EXISTS kiểm tra xem mã thuốc và mã toa thuốc có tồn tại trong các bảng THUOC và TOATHUOC không. Nếu không tồn tại, ném ra một lỗi Dùng lệnh IF EXISTS kiểm tra xem chi tiết đơn thuốc đã tồn tại với các thông tin đã cho hay chưa. Nếu tồn tại, ném ra một lỗi. INSERT INTO: Thêm một bản ghi chi tiết đơn thuốc mới vào bảng CTDonThuoc từ các tham số đã cung cấp. PRINT N'Thêm chi tiết đơn thuốc thành công': Thực hiện in thông báo trên console khi thêm chi tiết đơn thuốc thành công. COMMIT TRANSACTION: Thực hiện lưu vĩnh viễn dữ liệu nếu thêm thành công. Khởi lệnh BEGIN CATCH ... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY. Khối catch sẽ bắt được lỗi RAISERROR từ khối try và thực hiện rollback. Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. IF @@TRANCOUNT >0: Có ít nhất một giao dịch đang mở, và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK. ROLLBACK TRANSACTION: Hoàn tác tất cả thay đổi được thực hiện kể từ lúc giao dịch bắt đầu và đưa cơ sở dữ liệu về trạng thái ban đầu.

	PRINT N'Thêm chi tiết đơn thuốc không thành công' + ERROR_MESSAGE (): Sau khi thực hiện rollback thì in thông báo trên console kèm theo thông báo lỗi bắt được từ hệ thống khi thêm chi tiết đơn thuốc mới không thành công.
3.1.7	<u>Tên procedure:</u> ThemThuoc <u>Mã lệnh T-SQL:</u> <pre> CREATE PROCEDURE ThemThuoc @TENTHUOC NVARCHAR(50), @NUOCSX NVARCHAR(50), @DONGIA MONEY, @HSD SMALLDATETIME NULL, @SOLUONGTON INT, @TRANGTHAI NVARCHAR(20) NULL AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION ThemThuocTransaction; BEGIN TRY SET @HSD = COALESCE(@HSD, GETDATE()); -- Sử dụng COALESCE để cung cấp giá trị mặc định cho HSD nếu nó là null INSERT INTO THUOC (TENTHUOC, NUOCSX, DONGIA, HSD, SOLUONGTON) VALUES (@TENTHUOC, @NUOCSX, @DONGIA, @HSD, @SOLUONGTON, N'Còn thuốc'); COMMIT TRANSACTION ThemThuocTransaction; PRINT N'Thêm thuốc thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION ThemThuocTransaction; PRINT N'Thêm thuốc thất bại. ' + ERROR_MESSAGE(); END CATCH; END; </pre>
	<u>Mô tả mã lệnh:</u>

	Mã lệnh trên tạo ra một thủ tục đặt tên là ThemThuoc, dùng để thêm bản ghi thuốc mới và bảng THUOC. Các thông số đầu vào của thủ tục gồm: tên thuốc, nước sản xuất, đơn giá, hạn sử dụng, số lượng tồn, và trạng thái. Dùng lệnh SET NOCOUNT ON để dừng việc gửi thông báo đến máy khách về số hàng bị ảnh hưởng bởi câu lệnh T-SQL Nếu giá trị của @NGAYLAP là NULL, thì nó sẽ được thiết lập là ngày hiện tại lấy từ hệ thống (GETDATE()). Khởi lệnh BEGIN TRY ... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác chèn dữ liệu. Sử dụng hàm COALESCE để cung cấp giá trị mặc định cho @HSD nếu nó là NULL. Trong trường hợp này, nếu @HSD là NULL, thì giá trị của nó sẽ được thiết lập là ngày hiện tại (GETDATE()). INSERT INTO: Thêm một bản ghi thuốc mới vào bảng THUOC từ các tham số đã cung cấp và trạng thái mặc định là “Còn thuốc”. PRINT N'Thêm thuốc thành công': Thực hiện in thông báo trên console khi thêm thuốc thành công. COMMIT TRANSACTION: Thực hiện lưu vĩnh viễn dữ liệu nếu thêm thành công. Khởi lệnh BEGIN CATCH ... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY. Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. IF @@TRANCOUNT >0: Có ít nhất một giao dịch đang mở, và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK. ROLLBACK TRANSACTION: Hoàn tác tất cả thay đổi được thực hiện kể từ lúc giao dịch bắt đầu và đưa cơ sở dữ liệu về trạng thái ban đầu. PRINT N'Thêm thuốc không thành công' + ERROR_MESSAGE (): Sau khi thực hiện rollback thì in thông báo trên console kèm theo thông báo lỗi bắt được từ hệ thống khi thêm bản ghi thuốc mới không thành công.
3.1.8	Tên procedure: XOATOATHUOC Mã lệnh T-SQL: <pre>CREATE PROCEDURE XOATOATHUOC @MATOA INT</pre>

	<pre> AS BEGIN BEGIN TRY BEGIN TRANSACTION; IF NOT EXISTS (SELECT * FROM TOATHUOC WHERE ID = @MATOA) BEGIN ROLLBACK TRANSACTION; THROW 51003, 'Không tồn tại toa thuốc!', 1; END ELSE BEGIN IF EXISTS (SELECT * FROM CTDONTHUOC WHERE MATOA = @MATOA) DELETE FROM CTDONTHUOC WHERE MATOA = @MATOA; IF EXISTS (SELECT * FROM BIENLAI WHERE MATOA = @MATOA) DELETE FROM BIENLAI WHERE MATOA = @MATOA; DELETE FROM TOATHUOC WHERE ID = @MATOA; END COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(); END CATCH END GO </pre>
	Mô tả mã lệnh: Mã lệnh trên tạo ra một thủ tục có tên là XOATOATHUOC, có chức năng xóa toa thuốc và mọi thông tin có liên quan với toa thuốc đó. Các thông số đầu vào của thủ tục bao gồm @MATOA cần xóa. Khởi lệnh BEGIN TRY... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác xóa dữ liệu: Nếu không tìm thấy bản ghi TOATHUOC có ID tương ứng với @MATOA thì thực hiện ROLLBACK TRANSACTION và thông báo TOATHUOC không tồn tại.

	Nếu có bản ghi TOATHUOC với @MATOA đã nhập, thực hiện: Nếu tồn tại một hoặc nhiều bản ghi CTDONTHUOC tương ứng với @MATOA thì lần lượt xóa các bản ghi có liên quan đó. Nếu tồn tại bản ghi BIENLAI tương ứng với @MATOA thì xóa bản ghi BIENLAI có liên quan đó. Xóa bản ghi trong TOATHUOC có ID = @MATOA và COMMIT TRANSACTION. Khởi lệnh BEGIN CATCH... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY: Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. Nếu @@TRANCOUNT > 0 thì có ít nhất một giao dịch đang được mở và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK TRANSACTION. ROLLBACK TRANSACTION: hoàn tác các thay đổi được thực hiện từ lúc bắt đầu thực thi và đưa CSDL về trạng thái ban đầu. PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(): sau khi ROLLBACK thì in ra thông báo lỗi kèm lỗi phát hiện trong lúc thực thi mã lệnh.
3.1.9	Tên procedure: XOABENHNHAN Mã lệnh T-SQL: <pre> CREATE PROCEDURE XOABENHNHAN @MABN INT AS BEGIN SET NOCOUNT ON; BEGIN TRY BEGIN TRANSACTION; IF NOT EXISTS (SELECT * FROM BENHNHAN WHERE ID = @MABN) BEGIN ROLLBACK TRANSACTION; THROW 51004, N'Bệnh nhân không tồn tại', 1; END ELSE BEGIN IF EXISTS (SELECT * FROM PHIEUKHAMBENH WHERE MABN = @MABN) DELETE FROM PHIEUKHAMBENH WHERE MABN = @MABN; </pre>

	<pre> IF EXISTS (SELECT * FROM TOATHUOC WHERE MABN = @MABN) BEGIN DECLARE @MATOA INT; DECLARE CURSOR_TOATHUOC CURSOR FOR SELECT ID FROM TOATHUOC WHERE MABN = @MABN; OPEN CURSOR_TOATHUOC; FETCH NEXT FROM CURSOR_TOATHUOC INTO @MATOA; WHILE @@FETCH_STATUS = 0 BEGIN EXEC XOATOATHUOC @MATOA; FETCH NEXT FROM CURSOR_TOATHUOC INTO @MATOA; END CLOSE CURSOR_TOATHUOC; DEALLOCATE CURSOR_TOATHUOC; END IF EXISTS (SELECT * FROM TAIKHOAN WHERE MASOBN = @MABN) DELETE FROM TAIKHOAN WHERE MASOBN = @MABN; DELETE FROM BENHNNHAN WHERE ID = @MABN; END COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(); END CATCH END GO </pre>
	Mô tả mã lệnh: Mã lệnh trên tạo ra một thủ tục có tên là XOABENHNNHAN, có chức năng xóa bản ghi BENHNNHAN và mọi thông tin có liên quan với bệnh nhân đó. Các thông số đầu vào của thủ tục bao gồm @MABN của bệnh nhân cần xóa. Khởi lệnh BEGIN TRY... END TRY dùng để bắt lỗi trong quá trình thực thi giao tác xóa dữ liệu:

	Nếu không tìm thấy bản ghi với ID tương ứng với @MABN thì thực hiện ROLLBACK và thông báo bệnh nhân không tồn tại. Nếu tìm thấy: Nếu tồn tại các bản ghi PHIEUKHAMBENH với MABN tương ứng thì thực hiện xóa. Nếu tồn tại các bản ghi TOATHUOC với MABN tương ứng: Thực hiện tạo và mở cursor để lấy ID của TOATHUOC liên quan Duyệt qua từng bản ghi với cursor: Thực hiện thủ tục XOATOATHUOC đối với ID tương ứng Đóng và giải phóng cursor Thực hiện xóa bản ghi BENHNNHAN có ID tương ứng @MABN và COMMIT TRANSACTION. Khởi lệnh BEGIN CATCH... END CATCH dùng để xử lý lỗi có thể xảy ra trong khối TRY Sử dụng biến hệ thống @@TRANCOUNT để xác định số lượng giao dịch đang được mở. Nếu @@TRANCOUNT > 0 thì có ít nhất một giao dịch đang được mở và gặp lỗi trong quá trình thực thi thì thực hiện ROLLBACK TRANSACTION. ROLLBACK TRANSACTION: hoàn tác các thay đổi được thực hiện từ lúc bắt đầu thực thi và đưa CSDL về trạng thái ban đầu. PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(): sau khi ROLLBACK thì in ra thông báo lỗi kèm lỗi phát hiện trong lúc thực thi mã lệnh.
3.1.10	Tên Procedure: XOANHANVIEN Mã lệnh T-SQL: <pre> CREATE PROCEDURE XOANHANVIEN @ID INT AS BEGIN BEGIN TRY BEGIN TRANSACTION; IF EXISTS (SELECT * FROM NHANVIEN WHERE ID = @ID) BEGIN UPDATE TAIKHOAN SET TRANGTHAI = N'Ngưng hoạt động' WHERE MASONV = @ID; UPDATE NHANVIEN </pre>

	<pre> SET TRANGTHAI = N'Đã nghỉ việc' WHERE ID = @ID; END ELSE BEGIN THROW 51000, N'Không tìm thấy nhân viên cần xóa', 1; END COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(); END CATCH END </pre>
	Mô tả mã lệnh: Mã lệnh trên mô tả một thủ tục dùng để xóa một nhân viên khỏi hệ thống. Thủ tục này nhận đầu vào là biến @ID của nhân viên. Khối lệnh TRY: BEGIN TRANSACTION dùng để đảm bảo tính toàn vẹn của dữ liệu trong quá trình xóa. Nếu tồn tại @ID trong bảng NHANVIEN, thực hiện: Nếu tồn tại @ID trong bảng TAIKHOAN thì thực hiện cập nhật TRANGTHAI là 'Ngưng hoạt động' Thực hiện cập nhật TRANGTHAI của bản ghi NHANVIEN có @ID là 'Đã nghỉ việc' Nếu không tồn tại @ID trong bảng NHANVIEN thực hiện ném một lỗi với mã 51000 và nội dung 'Không tìm thấy nhân viên cần xóa'. Khi này, chương trình sẽ không thực hiện các lệnh trong IF mà chuyển qua khối lệnh CATCH. COMMIT TRANSACTION để hoàn tất giao tác. Khối lệnh CATCH: Trong trường hợp có lỗi xảy ra thì sẽ chuyển sang khối CATCH. Kiểm tra biến hệ thống @@TRANCOUNT để xem có giao dịch nào còn mở không. Nếu có thực hiện ROLLBACK. In ra màn hình 'Lỗi xảy ra' kèm thông báo lỗi trong trường hợp có lỗi.
3.1.11	Tên procedure: XOATHUOC Mã lệnh T-SQL: <pre> CREATE PROCEDURE XOATHUOC @ID INT </pre>

	<pre> AS BEGIN BEGIN TRY BEGIN TRANSACTION; IF EXISTS (SELECT * FROM THUOC) BEGIN UPDATE THUOC SET TRANGTHAI = N'Thuốc không có sẵn' WHERE ID = @ID; END BEGIN THROW 51001, N'Không tìm thấy thuốc', 1; END COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION PRINT N'Lỗi xảy ra: ' + ERROR_MESSAGE(); END CATCH END </pre>
	Mô tả mã lệnh: Mã lệnh tạo ra một thủ tục dùng để xóa thuốc. Thủ tục nhận đầu vào là @ID thuốc. Khối lệnh TRY: BEGIN TRANSACTION để đảm bảo toàn vẹn dữ liệu khi xóa. Nếu tồn tại @ID trong bảng THUOC: Thực hiện cập nhật TRANGTHAI bảng ghi THUOC có @ID thành 'Thuốc không có sẵn'. Nếu không tồn tại @ID trong bảng THUOC thực hiện ném lỗi mã 51001 với nội dung 'Không tìm thấy thuốc cần xóa'. Khi này, chương trình sẽ chuyển qua khối lệnh CATCH. COMMIT để hoàn tất giao dịch. Khối lệnh CATCH: Dùng @@TRANCOUNT để phát hiện giao dịch đang mở và thực hiện ROLLBACK In ra màn hình thông báo lỗi.
3.1.12	Tên procedure : CapNhatPhieuKhamBenh Mã lệnh T-SQL: <pre> CREATE PROCEDURE CapNhatPhieuKhamBenh @MAPKB VARCHAR(10), @MABS INT = NULL, @MABN INT = NULL, @MAPK INT = NULL, </pre>

	<pre> @NGAYTAO SMALLDATETIME = NULL, @CHANDOAN NVARCHAR(50) = NULL AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION CapNhatPhieuKhamBenhTransaction; BEGIN TRY -- Kiểm tra xem phiếu khám bệnh có tồn tại không IF NOT EXISTS (SELECT 1 FROM PHIEUKHAMBENH WHERE MAPKB = @MAPKB) BEGIN RAISERROR('Phiếu khám bệnh không tồn tại.', 16, 1); RETURN; END; -- Lấy giá trị hiện tại của các trường trong bảng DECLARE @MABS_OLD INT, @MABN_OLD INT, @MAPK_OLD INT, @NGAYTAO_OLD SMALLDATETIME, @CHANDOAN_OLD NVARCHAR(50); SELECT @MABS_OLD = MABS, @MABN_OLD = MABN, @MAPK_OLD = MAPK, @NGAYTAO_OLD = NGAYTAO, @CHANDOAN_OLD = CHANDOAN FROM PHIEUKHAMBENH WHERE MAPKB = @MAPKB; -- Gán giá trị cũ cho các biến nếu các tham số truyền vào là null SET @MABS = COALESCE(@MABS, @MABS_OLD); SET @MABN = COALESCE(@MABN, @MABN_OLD); SET @MAPK = COALESCE(@MAPK, @MAPK_OLD); SET @NGAYTAO = COALESCE(@NGAYTAO, @NGAYTAO_OLD); SET @CHANDOAN = COALESCE(@CHANDOAN, @CHANDOAN_OLD); -- Cập nhật các trường UPDATE PHIEUKHAMBENH </pre>
--	--

	<pre> SET MABS = @MABS, MABN = @MABN, MAPK = @MAPK, NGAYTAO = @NGAYTAO, CHANDOAN = @CHANDOAN WHERE MAPKB = @MAPKB; -- Commit giao dịch nếu mọi thứ thành công COMMIT TRANSACTION CapNhatPhieuKhamBenhTransaction; PRINT N'Cập nhật phiếu khám bệnh thành công'; END TRY BEGIN CATCH -- Rollback giao dịch nếu có lỗi IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION CapNhatPhieuKhamBenhTransaction; PRINT N'Cập nhật phiếu khám bệnh thất bại. ' + ERROR_MESSAGE(); END CATCH; END; GO </pre> Mô tả mã lệnh: Bắt đầu bằng cách thiết lập SET NOCOUNT ON; để không trả về số bản ghi ảnh hưởng. Bắt đầu một giao dịch (BEGIN TRANSACTION) với tên là CapNhatPhieuKhamBenhTransaction. Sử dụng khối BEGIN TRY...END TRY và BEGIN CATCH...END CATCH để xử lý các ngoại lệ. Trong khối TRY, kiểm tra xem phiếu khám bệnh có tồn tại không. Nếu không tồn tại, một lỗi sẽ được ném (RAISERROR) và giao dịch sẽ được rollback. Lấy giá trị hiện tại của các trường trong bảng PHIEUKHAMBENH cho phiếu khám bệnh cần cập nhật. Sử dụng COALESCE để gán giá trị cũ cho các biến đối số nếu các tham số truyền vào là NULL. Sử dụng một câu lệnh UPDATE để cập nhật các trường của phiếu khám bệnh. Commit giao dịch nếu mọi thứ diễn ra thành công. Trong khối CATCH, rollback giao dịch nếu có lỗi và in ra thông báo lỗi.
3.1.13	Tên procedure: CapNhatGiaThuoc

	Mã lệnh T-SQL: <pre> CREATE PROCEDURE CapNhatGiaThuoc @TENTHUOC NVARCHAR(50), @NUOCSX NVARCHAR(50), @DONGIA MONEY AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION CapNhatGiaThuocTransaction; BEGIN TRY IF NOT EXISTS (SELECT 1 FROM THUOC WHERE TENTHUOC = @TENTHUOC AND NUOCSX = @NUOCSX) BEGIN RAISERROR('Loại thuốc không tồn tại.', 16, 1); RETURN; END; UPDATE THUOC SET DONGIA = @DONGIA WHERE TENTHUOC = @TENTHUOC AND NUOCSX = @NUOCSX; COMMIT TRANSACTION CapNhatGiaThuocTransaction; PRINT N'Cập nhật giá thuốc thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION CapNhatGiaThuocTransaction; PRINT N'Cập nhật giá thuốc thất bại. ' + ERROR_MESSAGE(); END CATCH; END; GO </pre> Mô tả mã lệnh □ SET NOCOUNT ON;: Điều này ngăn việc trả về số bản ghi ảnh hưởng sau mỗi lệnh SQL được thực thi, giúp làm giảm lượng dữ liệu truyền từ server về client.
--	---

	□ BEGIN TRANSACTION CapNhatGiaThuocTransaction;; Bắt đầu một giao dịch với tên CapNhatGiaThuocTransaction. □ Khởi TRY-CATCH: Sử dụng để xử lý các lỗi trong quá trình thực thi. Trong khối TRY, đầu tiên kiểm tra xem loại thuốc cần cập nhật giá có tồn tại không thông qua câu lệnh IF NOT EXISTS. Nếu loại thuốc không tồn tại, một lỗi sẽ được ném ra sử dụng RAISERROR và giao dịch sẽ được rollback. Sau đó, một câu lệnh UPDATE sẽ được sử dụng để cập nhật giá mới của loại thuốc. □ Khởi CATCH: Được thực hiện nếu có lỗi xảy ra trong quá trình thực thi lệnh TRY. Nếu có lỗi, giao dịch sẽ được rollback và một thông báo lỗi sẽ được in ra sử dụng PRINT kèm theo thông tin chi tiết về lỗi.
3.1.14	Tên procedure: CapNhatBenhNhan Mã lệnh T-SQL: <pre>CREATE PROCEDURE CapNhatBenhNhan @maBN VARCHAR(10) = NULL, @tenBN NVARCHAR(50) = NULL, @sdt VARCHAR(10) = NULL, @ngaySinh SMALLDATETIME = NULL, @diaChi NVARCHAR(50) = NULL, @gioiTinh NVARCHAR(5) = NULL, @queQuan NVARCHAR(20) = NULL, @ghichu NVARCHAR(50) = NULL AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION CapNhatBenhNhanTransaction; BEGIN TRY -- Kiểm tra xem bệnh nhân có tồn tại không IF NOT EXISTS (SELECT 1 FROM BenhNhan WHERE maBN = @maBN) BEGIN RAISERROR('Bệnh nhân không tồn tại.', 16, 1); RETURN; END; -- Lấy giá trị hiện tại của các trường trong bảng</pre>

	<pre> DECLARE @tenBN_OLD NVARCHAR(50), @sdt_OLD VARCHAR(10), @ngaySinh_OLD SMALLDATETIME, @diaChi_OLD NVARCHAR(50), @gioiTinh_OLD NVARCHAR(5), @queQuan_OLD NVARCHAR(20), @ghichu_OLD NVARCHAR(50); SELECT @tenBN_OLD = tenBN, @sdt_OLD = sdt, @ngaySinh_OLD = ngaySinh, @diaChi_OLD = diaChi, @gioiTinh_OLD = gioiTinh, @queQuan_OLD = queQuan, @ghichu_OLD = ghichu FROM BenhNhan WHERE maBN = @maBN; -- Gán giá trị cũ cho các tham số nếu chúng là null SET @tenBN = COALESCE(@tenBN, @tenBN_OLD); SET @sdt = COALESCE(@sdt, @sdt_OLD); SET @ngaySinh = COALESCE(@ngaySinh, @ngaySinh_OLD); SET @diaChi = COALESCE(@diaChi, @diaChi_OLD); SET @gioiTinh = COALESCE(@gioiTinh, @gioiTinh_OLD); SET @queQuan = COALESCE(@queQuan, @queQuan_OLD); SET @ghichu = COALESCE(@ghichu, @ghichu_OLD); UPDATE BenhNhan SET tenBN = @tenBN, sdt = @sdt, ngaySinh = @ngaySinh, diaChi = @diaChi, gioiTinh = @gioiTinh, queQuan = @queQuan, ghichu = @ghichu WHERE maBN = @maBN; COMMIT TRANSACTION CapNhatBenhNhanTransaction; PRINT N'Cập nhật thông tin bệnh nhân thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 </pre>
--	---

	ROLLBACK TRANSACTION <pre>CapNhatBenhNhanTransaction; PRINT N'Cập nhật thông tin bệnh nhân thất bại. ' + ERROR_MESSAGE(); END CATCH; END;</pre> Mô tả mã lệnh: □ Sử dụng BEGIN TRANSACTION để bắt đầu một giao dịch và TRY...CATCH để xử lý ngoại lệ. Kiểm tra xem bệnh nhân có tồn tại không. Nếu không tồn tại, thông báo lỗi và kết thúc procedure. Lấy giá trị hiện tại của các trường trong bảng BenhNhan. Gán giá trị cũ cho các tham số nếu chúng là NULL, sử dụng COALESCE. Thực hiện UPDATE để cập nhật thông tin bệnh nhân. Kết thúc giao dịch và in ra thông báo thành công nếu không có lỗi. Trong trường hợp có lỗi, sẽ rollback giao dịch và in ra thông báo lỗi. □ Thực hiện cập nhật: Sử dụng lệnh UPDATE để cập nhật thông tin của bệnh nhân trong bảng "BenhNhan" với các giá trị mới được truyền vào qua các tham số. WHERE clause được sử dụng để xác định bệnh nhân cần cập nhật dựa trên mã bệnh nhân (@maBN). □ Kết thúc giao dịch: COMMIT TRANSACTION được sử dụng để xác nhận rằng các thay đổi đã được áp dụng vào cơ sở dữ liệu nếu không có lỗi nào xảy ra. Sau khi thực hiện COMMIT, giao dịch sẽ kết thúc và các thay đổi sẽ trở thành vĩnh viễn. □ Thông báo: Nếu không có lỗi nào xảy ra, một thông báo thành công sẽ được in ra bằng lệnh PRINT. □ Xử lý ngoại lệ: Trong trường hợp có lỗi xảy ra trong quá trình thực thi của giao dịch, nó sẽ được bắt và xử lý trong phần CATCH. Nếu một giao dịch đang chạy khi lỗi xảy ra, ROLLBACK sẽ được sử dụng để hoàn tác các thay đổi đã thực hiện trong giao dịch. Sau đó, một thông báo lỗi cùng với thông điệp lỗi cụ thể sẽ được in ra bằng lệnh PRINT
--	--

3.1.15

Tên procedure: PRO_DANGNHAP

Tham số truyền vào: @EMAIL kiểu dữ liệu VARCHAR(50), @MATKHAU kiểu dữ liệu VARCHAR(50), @VAITRO kiểu dữ liệu NVARCHAR(20)

Mã lệnh T-SQL:

```
CREATE PROCEDURE PRO_DANGNHAP
    @EMAIL VARCHAR(50),
    @MATKHAU VARCHAR(50),
    @VAITRO NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @UserExists INT;
    DECLARE @RoleMatches INT;

    SELECT @UserExists = COUNT(*)
    FROM TaiKhoan
    WHERE email = @EMAIL AND matKhai =
    @MATKHAU;

    IF @UserExists = 1
    BEGIN
        SELECT @RoleMatches = COUNT(*)
        FROM TaiKhoan
        WHERE email = @EMAIL AND matKhai =
        @MATKHAU AND vaiTro = @VAITRO;

        IF @RoleMatches = 1
        BEGIN
            SELECT N'Dăng nhập thành công' AS
            Message, @VAITRO AS Role;
        END
        ELSE
        BEGIN
            SELECT N'Vai trò không đúng' AS Message;
        END
    END
    ELSE
    BEGIN
        SELECT N'Email hoặc mật khẩu không đúng'
        AS Message;
    END
END;
```

	<u>Mô tả mã lệnh</u> Procedure trên có tên là PRO_DANGNHAP, nhiệm vụ của nó là cho phép nhân viên đăng nhập vào hệ thống và phân quyền theo chức năng để đưa nhân viên đến chức năng tương ứng dựa trên email, mật khẩu và vai trò được đưa vào đầu của procedure. Trong thân procedure, các hành động được thực hiện bao gồm: Khai báo mã user tồn tại và vai trò của user. Câu lệnh SELECT @UserExists = COUNT(*) FROM TaiKhoan WHERE email = @EMAIL AND matKhau = @MATKHAU; Kiểm tra xem email và mật khẩu có tồn tại trong bảng TaiKhoan không. Nếu @UserExists=1 tức là tồn tại tài khoản. Tiếp đến là câu lệnh: SELECT @RoleMatches = COUNT(*)FROM TaiKhoan WHERE email = @EMAIL AND matKhau = @MATKHAU AND vaiTro = @VAITRO; để kiểm tra vai trò của tài khoản có khớp với vai trò được cung cấp không. Nếu @RoleMatches= 1 tức là tài khoản tồn tại và khớp với vai trò => Đăng nhập thành công. Ngược lại=>Vai trò không đúng Ngược lại => Tài khoản không tồn tại.
--	--

3.1.16	Tên procedure: PRO_DANGKY Tham số truyền vào: @HOTEN NVARCHAR(50), @MASONV VARCHAR(10), @SDT VARCHAR(10), @EMAIL VARCHAR(50), @MATKHAU VARCHAR(50), @MATKHAUXACNHAN VARCHAR(50), @VAITRO NVARCHAR(20) Mã lệnh T SQL: <pre> create PROCEDURE PRO_DANGKY @HOTEN NVARCHAR(50), @MASONV VARCHAR(10), @SDT VARCHAR(10), @EMAIL VARCHAR(50), @MATKHAU VARCHAR(50), @MATKHAUXACNHAN VARCHAR(50), @VAITRO NVARCHAR(20) AS BEGIN SET NOCOUNT ON; IF @MATKHAU != @MATKHAUXACNHAN BEGIN SELECT N'MẬT KHẨU KHÔNG KHỚP NHAU' AS Message; RETURN; END DECLARE @UserExists INT, @EmployeeExists INT, @ID INT, @Role NVARCHAR(20); IF @VAITRO IN (N'Bác sĩ', N'NV Tiếp Nhận', N'NV Thanh Toán', N'Admin', N'Quản lý kho') BEGIN SELECT @EmployeeExists = COUNT(*) FROM NHANVIEN WHERE MANV = @MASONV; IF @EmployeeExists > 0 BEGIN SELECT @ID = ID, @Role = vaiTro FROM NHANVIEN WHERE MANV = @MASONV; IF @Role != @VAITRO </pre>
--------	---

	<pre>BEGIN SELECT N'Vai trò không khớp với hồ sơ nhân viên' AS Message; RETURN; END INSERT INTO TAIKHOAN (hoTen, maSoNV, soDienThoai, email, matKhai, matKhauxacnhan, vaiTro) VALUES (@HOTEN, @ID, @SDT, @EMAIL, @MATKHAU, @MATKHAUXACNHAN, @VAITRO); SELECT N'Tài khoản nhân viên tạo thành công' AS Message; END END ELSE BEGIN SELECT N'Vai trò không hợp lệ' AS Message; END END;</pre>
--	--

	Mô tả mã lệnh: Procedure trên có tên là PRO_DANGNHAP, nhiệm vụ của nó là cho phép nhân viên đăng nhập vào hệ thống và phân quyền theo chức năng để đưa nhân viên đến chức năng tương ứng dựa trên email, mật khẩu và vai trò được đưa vào đầu của procedure. Trong thân procedure, các hành động được thực hiện bao gồm: Đầu tiên là kiểm tra 2 tham số truyền vào là @MATKHAU, @MATKHAUXACNHAN có khớp nhau không. Nếu không khớp thì thông báo lỗi. Khai báo các biến cùng với các kiểu dữ liệu tương ứng để lưu thông tin được gán vào @UserExists INT, @EmployeeExists INT, @ID INT, @Role NVARCHAR(20). Nếu biến @VAITRO được đưa vào đầu procedure có dữ liệu thuộc 1 trong các dạng (Bác sĩ, NV Tiếp Nhận, NV Thanh Toán, Admin, Quản lý kho. Câu lệnh SELECT @EmployeeExists = COUNT(*) FROM NHANVIEN WHERE MANV = @MASONV; Đếm số lượng nhân viên có MANV = @MASONV và gán vào biến @EmployeeExists. Nếu @EmployeeExists>0 tức là tồn tại nhân viên. Tiếp đến với câu lệnh SELECT @ID = ID, @Role = vaiTro FROM NHANVIEN WHERE MANV = @MASONV; Để lấy ra ID và vai trò tương ứng với @MASONV được truyền vào và gán vào biến @ID, @Role Kiểm tra nếu @Role không khớp với @VAITRO truyền vào thì thông báo vai trò không khớp. Ngược lại thì tạo tài khoản cho nhân viên với vai trò tương ứng. Ngược lại thông báo vai trò không hợp lệ.
3.1.17	Tên procedure : CapNhatTaiKhoan Mã lệnh T-SQL <pre>CREATE PROCEDURE CapNhatTaiKhoan @id INT, @hoTen NVARCHAR(50) = NULL, @maSoBN INT = NULL, @maSoNV INT = NULL, @soDienThoai VARCHAR(10) = NULL, @email VARCHAR(50) = NULL, @matKhai VARCHAR(50) = NULL, @matkhauxacnhan VARCHAR(50) = NULL,</pre>

	<pre> @vaiTro NVARCHAR(20) = NULL AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION CapNhatTaiKhoanTransaction; BEGIN TRY -- Kiểm tra xem tài khoản có tồn tại không IF NOT EXISTS (SELECT 1 FROM TaiKhoan WHERE id = @id) BEGIN RAISERROR('Tài khoản không tồn tại.', 16, 1); RETURN; END; -- Lấy giá trị hiện tại của các trường trong bảng DECLARE @hoTen_OLD NVARCHAR(50), @maSoBN_OLD INT, @maSoNV_OLD INT, @soDienThoai_OLD VARCHAR(10), @email_OLD VARCHAR(50), @matKhau_OLD VARCHAR(50), @matkhauxacnhan_OLD VARCHAR(50), @vaiTro_OLD NVARCHAR(20); SELECT @hoTen_OLD = hoTen, @maSoBN_OLD = maSoBN, @maSoNV_OLD = maSoNV, @soDienThoai_OLD = soDienThoai, @email_OLD = email, @matKhau_OLD = matKhau, @matkhauxacnhan_OLD = matkhauxacnhan, @vaiTro_OLD = vaiTro FROM TaiKhoan WHERE id = @id; -- Gán giá trị cũ cho các tham số nếu chúng là null SET @hoTen = COALESCE(@hoTen, @hoTen_OLD); SET @maSoBN = COALESCE(@maSoBN, @maSoBN_OLD); SET @maSoNV = COALESCE(@maSoNV, @maSoNV_OLD); SET @soDienThoai = COALESCE(@soDienThoai, @soDienThoai_OLD); SET @email = COALESCE(@email, @email_OLD); SET @matKhau = COALESCE(@matKhau, @matKhau_OLD); </pre>
--	---

	<pre> SET @matkhauxacnhan = COALESCE(@matkhauxacnhan, @matkhauxacnhan_OLD); SET @vaiTro = COALESCE(@vaiTro, @vaiTro_OLD); UPDATE TaiKhoan SET hoTen = @hoTen, maSoBN = @maSoBN, maSoNV = @maSoNV, soDienThoai = @soDienThoai, email = @email, matKhau = @matKhau, matkhauxacnhan = @matkhauxacnhan, vaiTro = @vaiTro WHERE id = @id; COMMIT TRANSACTION CapNhatTaiKhoanTransaction; PRINT N'Cập nhật thông tin tài khoản thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION CapNhatTaiKhoanTransaction; PRINT N'Cập nhật thông tin tài khoản thất bại. ' + ERROR_MESSAGE(); END CATCH; END; GO </pre> Mô tả mã lệnh Kiểm tra sự tồn tại của tài khoản: Procedure bắt đầu bằng việc kiểm tra xem tài khoản với id được cung cấp có tồn tại trong bảng TaiKhoan hay không. Điều này giúp đảm bảo rằng chỉ có tài khoản tồn tại mới được cập nhật. Nếu tài khoản không tồn tại, procedure sẽ tạo ra một thông báo lỗi và kết thúc. Lấy giá trị hiện tại của các trường: Sau khi xác nhận sự tồn tại của tài khoản, procedure sẽ lấy giá trị hiện tại của các trường thông tin của tài khoản đó trong bảng TaiKhoan. Điều này cho phép so sánh giữa các giá trị cũ và mới để quyết định xem liệu cần cập nhật hay không.
--	--

	Gán giá trị cũ cho các tham số nếu chúng là null: Trong trường hợp các tham số đầu vào (như @hoTen, @maSoBN, ...) được gửi vào procedure với giá trị NULL, procedure sẽ sử dụng các giá trị hiện tại của các trường tương ứng lấy từ bước trước đó. Điều này giúp bảo toàn dữ liệu và tránh việc ghi đè các giá trị quan trọng bằng NULL. Cập nhật thông tin tài khoản: Sau khi đã chuẩn bị đầy đủ thông tin, procedure tiến hành cập nhật thông tin tài khoản trong bảng TaiKhoan với các giá trị mới hoặc cũ tùy thuộc vào các tham số được truyền vào. Việc này đảm bảo rằng thông tin tài khoản sẽ được cập nhật chính xác theo yêu cầu của người dùng. Quản lý giao dịch (Transaction Management): Trước khi bắt đầu thực hiện bất kỳ thay đổi nào trong cơ sở dữ liệu, procedure bắt đầu một giao dịch mới (BEGIN TRANSACTION) với tên là CapNhatTaiKhoanTransaction. Nếu quá trình cập nhật thông tin diễn ra thành công, procedure sẽ commit giao dịch (COMMIT TRANSACTION), ghi nhận các thay đổi vào cơ sở dữ liệu. Nếu có bất kỳ lỗi nào xảy ra trong quá trình cập nhật, procedure sẽ rollback giao dịch (ROLLBACK TRANSACTION), hủy bỏ mọi thay đổi đã thực hiện và in ra thông báo lỗi. Thông báo kết quả: Nếu quá trình cập nhật thành công, procedure sẽ in ra một thông báo xác nhận thành công. Nếu có lỗi xảy ra, procedure sẽ in ra một thông báo lỗi kèm theo thông điệp cụ thể về lỗi đó.
3.1.18	Tên procedure : CapNhatThuoc Mô tả mã lệnh T-SQL CREATE PROCEDURE CapNhatThuoc @id INT, @tenThuoc NVARCHAR(50) = NULL, @nuocSX NVARCHAR(50) = NULL, @donGia MONEY = NULL, @HSD SMALLDATETIME = NULL, @soLuongTon INT = NULL, @trangThai NVARCHAR(20) = NULL AS BEGIN SET NOCOUNT ON;

	<pre> BEGIN TRANSACTION CapNhatThuocTransaction; BEGIN TRY -- Kiểm tra xem thuốc có tồn tại không IF NOT EXISTS (SELECT 1 FROM Thuoc WHERE id = @id) BEGIN RAISERROR('Thuốc không tồn tại.', 16, 1); RETURN; END; -- Lấy giá trị hiện tại của các trường trong bảng DECLARE @tenThuoc_OLD NVARCHAR(50), @nuocSX_OLD NVARCHAR(50), @donGia_OLD MONEY, @HSD_OLD SMALLDATETIME, @soLuongTon_OLD INT, @trangThai_OLD NVARCHAR(20); SELECT @tenThuoc_OLD = tenThuoc, @nuocSX_OLD = nuocSX, @donGia_OLD = donGia, @HSD_OLD = HSD, @soLuongTon_OLD = soLuongTon, @trangThai_OLD = trangThai FROM Thuoc WHERE id = @id; -- Gán giá trị cũ cho các tham số nếu chúng là null SET @tenThuoc = COALESCE(@tenThuoc, @tenThuoc_OLD); SET @nuocSX = COALESCE(@nuocSX, @nuocSX_OLD); SET @donGia = COALESCE(@donGia, @donGia_OLD); SET @HSD = COALESCE(@HSD, @HSD_OLD); SET @soLuongTon = COALESCE(@soLuongTon, @soLuongTon_OLD); SET @trangThai = COALESCE(@trangThai, @trangThai_OLD); UPDATE Thuoc SET tenThuoc = @tenThuoc, nuocSX = @nuocSX, donGia = @donGia, HSD = @HSD, soLuongTon = @soLuongTon, trangThai = @trangThai </pre>
--	---

	<pre> WHERE id = @id; COMMIT TRANSACTION CapNhatThuocTransaction; PRINT N'Cập nhật thông tin thuốc thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION CapNhatThuocTransaction; PRINT N'Cập nhật thông tin thuốc thất bại. ' + ERROR_MESSAGE(); END CATCH; END; GO --test EXEC CapNhatThuoc @id = 1, @tenThuoc = 'Paracetamol', @nuocSX = 'Vietnam', @donGia = 50000, @HSD = '2025-12-31', @soLuongTon = 200, @trangThai = 'Available'; </pre> Mô tả mã lệnh Kiểm tra sự tồn tại của thuốc: Procedure bắt đầu bằng việc kiểm tra xem loại thuốc với id được cung cấp có tồn tại trong bảng Thuoc hay không. Nếu không tồn tại, nó sẽ tạo ra một thông báo lỗi và kết thúc procedure. Lấy giá trị hiện tại của các trường: Sau khi xác nhận sự tồn tại của thuốc, procedure lấy giá trị hiện tại của các trường thông tin của thuốc đó trong bảng Thuoc. Điều này cho phép so sánh giữa các giá trị cũ và mới để quyết định xem liệu cần cập nhật hay không. Gán giá trị cũ cho các tham số nếu chúng là null: Trong trường hợp các tham số đầu vào (như @tenThuoc, @nuocSX, ...) được gửi vào procedure với giá trị NULL, procedure sẽ sử dụng các giá trị hiện tại của các trường tương ứng lấy từ bước trước đó. Điều này giúp bảo toàn dữ liệu và tránh việc ghi đè các giá trị quan trọng bằng NULL. Cập nhật thông tin thuốc:
--	--

	Sau khi đã chuẩn bị đầy đủ thông tin, procedure tiến hành cập nhật thông tin của loại thuốc trong bảng Thuoc với các giá trị mới hoặc cũ tùy thuộc vào các tham số được truyền vào. Việc này đảm bảo rằng thông tin của thuốc sẽ được cập nhật chính xác theo yêu cầu của người dùng. Quản lý giao dịch (Transaction Management): Trước khi bắt đầu thực hiện bất kỳ thay đổi nào trong cơ sở dữ liệu, procedure bắt đầu một giao dịch mới (BEGIN TRANSACTION) với tên là CapNhatThuocTransaction. Nếu quá trình cập nhật thông tin diễn ra thành công, procedure sẽ commit giao dịch (COMMIT TRANSACTION), ghi nhận các thay đổi vào cơ sở dữ liệu. Nếu có bất kỳ lỗi nào xảy ra trong quá trình cập nhật, procedure sẽ rollback giao dịch (ROLLBACK TRANSACTION), hủy bỏ mọi thay đổi đã thực hiện và in ra thông báo lỗi.
3.1.19	Tên procedure : CapNhatNhanVien Mã lệnh T-SQL <pre> CREATE PROCEDURE CapNhatNhanVien @id INT, @hoTen NVARCHAR(50) = NULL, @ngaySinh SMALLDATETIME = NULL, @diaChi NVARCHAR(50) = NULL, @gioiTinh NVARCHAR(5) = NULL, @ngayVL SMALLDATETIME = NULL, @vaiTro NVARCHAR(20) = NULL AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION CapNhatNhanVienTransaction; BEGIN TRY -- Kiểm tra xem nhân viên có tồn tại không IF NOT EXISTS (SELECT 1 FROM NhanVien WHERE id = @id) BEGIN RAISERROR('Nhân viên không tồn tại.', 16, 1); RETURN; END; -- Lấy giá trị hiện tại của các trường trong bảng </pre>

	<pre> DECLARE @hoTen_OLD NVARCHAR(50), @ngaySinh_OLD SMALLDATETIME, @diaChi_OLD NVARCHAR(50), @gioiTinh_OLD NVARCHAR(5), @ngayVL_OLD SMALLDATETIME, @vaiTro_OLD NVARCHAR(20); SELECT @hoTen_OLD = hoTen, @ngaySinh_OLD = ngaySinh, @diaChi_OLD = diaChi, @gioiTinh_OLD = gioiTinh, @ngayVL_OLD = ngayVL, @vaiTro_OLD = vaiTro FROM NhanVien WHERE id = @id; -- Gán giá trị cũ cho các tham số nếu chúng là null SET @hoTen = COALESCE(@hoTen, @hoTen_OLD); SET @ngaySinh = COALESCE(@ngaySinh, @ngaySinh_OLD); SET @diaChi = COALESCE(@diaChi, @diaChi_OLD); SET @gioiTinh = COALESCE(@gioiTinh, @gioiTinh_OLD); SET @ngayVL = COALESCE(@ngayVL, @ngayVL_OLD); SET @vaiTro = COALESCE(@vaiTro, @vaiTro_OLD); UPDATE NhanVien SET hoTen = @hoTen, ngaySinh = @ngaySinh, diaChi = @diaChi, gioiTinh = @gioiTinh, ngayVL = @ngayVL, vaiTro = @vaiTro WHERE id = @id; COMMIT TRANSACTION CapNhatNhanVienTransaction; PRINT N'Cập nhật thông tin nhân viên thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION CapNhatNhanVienTransaction; </pre>
--	--

	<pre> PRINT N'Cập nhật thông tin nhân viên thất bại. ' + ERROR_MESSAGE(); END CATCH; END; GO --test EXEC CapNhatNhanVien @id = 1, @hoTen = 'Nguyen Van B', @ngaySinh = '1985-05-20', @diaChi = '123 Main St', @gioiTinh = 'Nam', @ngayVL = '2010-09-15', @vaiTro = 'Bac Si'; </pre> Mô tả mã lệnh: Kiểm tra sự tồn tại của nhân viên: Procedure bắt đầu bằng việc kiểm tra xem nhân viên với id được cung cấp có tồn tại trong bảng NhanVien hay không. Nếu không tồn tại, nó sẽ tạo ra một thông báo lỗi và kết thúc procedure. Lấy giá trị hiện tại của các trường: Sau khi xác nhận sự tồn tại của nhân viên, procedure lấy giá trị hiện tại của các trường thông tin của nhân viên đó trong bảng NhanVien. Điều này cho phép so sánh giữa các giá trị cũ và mới để quyết định xem liệu cần cập nhật hay không. Gán giá trị cũ cho các tham số nếu chúng là null: Trong trường hợp các tham số đầu vào (như @hoTen, @ngaySinh, ...) được gửi vào procedure với giá trị NULL, procedure sẽ sử dụng các giá trị hiện tại của các trường tương ứng lấy từ bước trước đó. Điều này giúp bảo toàn dữ liệu và tránh việc ghi đè các giá trị quan trọng bằng NULL. Cập nhật thông tin nhân viên: Sau khi đã chuẩn bị đầy đủ thông tin, procedure tiến hành cập nhật thông tin của nhân viên trong bảng NhanVien với các giá trị mới hoặc cũ tùy thuộc vào các tham số được truyền vào. Việc này đảm bảo rằng thông tin của nhân viên sẽ được cập nhật chính xác theo yêu cầu của người dùng. Quản lý giao dịch (Transaction Management): Trước khi bắt đầu thực hiện bất kỳ thay đổi nào trong cơ sở dữ liệu, procedure bắt đầu một giao dịch mới
--	---

	(BEGIN TRANSACTION) với tên là CapNhatNhanVienTransaction. Nếu quá trình cập nhật thông tin diễn ra thành công, procedure sẽ commit giao dịch (COMMIT TRANSACTION), ghi nhận các thay đổi vào cơ sở dữ liệu. Nếu có bất kỳ lỗi nào xảy ra trong quá trình cập nhật, procedure sẽ rollback giao dịch (ROLLBACK TRANSACTION), hủy bỏ mọi thay đổi đã thực hiện và in ra thông báo lỗi.
3.1.20	Tên procedure : CapNhatToaThuoc Mã lệnh T-SQL <pre> CREATE PROCEDURE CapNhatToaThuoc @id INT, @maBN INT = NULL, @mabs INT = NULL, @tongTien MONEY = NULL, @ngayLap DATETIME = NULL AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION CapNhatToaThuocTransaction; BEGIN TRY -- Kiểm tra xem toa thuốc có tồn tại không IF NOT EXISTS (SELECT 1 FROM ToaThuoc WHERE id = @id) BEGIN RAISERROR('Toa thuốc không tồn tại.', 16, 1); RETURN; END; -- Lấy giá trị hiện tại của các trường trong bảng DECLARE @maBN_OLD INT, @mabs_OLD INT, @tongTien_OLD MONEY, @ngayLap_OLD DATETIME; SELECT @maBN_OLD = maBN, @mabs_OLD = mabs, @tongTien_OLD = tongTien, @ngayLap_OLD = ngayLap FROM ToaThuoc WHERE id = @id; -- Gán giá trị cũ cho các tham số nếu chúng là null SET @maBN = COALESCE(@maBN, @maBN_OLD); </pre>

	<pre> SET @mabs = COALESCE(@mabs, @mabs_OLD); SET @tongTien = COALESCE(@tongTien, @mongTien_OLD); SET @ngayLap = COALESCE(@ngayLap, @ngayLap_OLD); UPDATE ToaThuoc SET maBN = @maBN, mabs = @mabs, tongTien = @tongTien, ngayLap = @ngayLap WHERE id = @id; COMMIT TRANSACTION CapNhatToaThuocTransaction; PRINT N'Cập nhật thông tin toa thuốc thành công'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION CapNhatToaThuocTransaction; PRINT N'Cập nhật thông tin toa thuốc thất bại. ' + ERROR_MESSAGE(); END CATCH; END; GO </pre> Mô tả mã lệnh Kiểm tra sự tồn tại của toa thuốc: Procedure bắt đầu bằng việc kiểm tra xem toa thuốc với id được cung cấp có tồn tại trong bảng ToaThuoc hay không. Nếu không tồn tại, nó sẽ tạo ra một thông báo lỗi và kết thúc procedure. Lấy giá trị hiện tại của các trường: Sau khi xác nhận sự tồn tại của toa thuốc, procedure lấy giá trị hiện tại của các trường thông tin của toa thuốc đó trong bảng ToaThuoc. Điều này cho phép so sánh giữa các giá trị cũ và mới để quyết định xem liệu cần cập nhật hay không. Gán giá trị cũ cho các tham số nếu chúng là null: Trong trường hợp các tham số đầu vào (như @maBN, @mabs, ...) được gửi vào procedure với giá trị NULL,
--	---

	procedure sẽ sử dụng các giá trị hiện tại của các trường tương ứng lấy từ bước trước đó. Điều này giúp bảo toàn dữ liệu và tránh việc ghi đè các giá trị quan trọng bằng NULL. Cập nhật thông tin toa thuốc: Sau khi đã chuẩn bị đầy đủ thông tin, procedure tiến hành cập nhật thông tin của toa thuốc trong bảng ToaThuoc với các giá trị mới hoặc cũ tùy thuộc vào các tham số được truyền vào. Việc này đảm bảo rằng thông tin của toa thuốc sẽ được cập nhật chính xác theo yêu cầu của người dùng. Quản lý giao dịch (Transaction Management): Trước khi bắt đầu thực hiện bất kỳ thay đổi nào trong cơ sở dữ liệu, procedure bắt đầu một giao dịch mới (BEGIN TRANSACTION) với tên là CapNhatToaThuocTransaction. Nếu quá trình cập nhật thông tin diễn ra thành công, procedure sẽ commit giao dịch (COMMIT TRANSACTION), ghi nhận các thay đổi vào cơ sở dữ liệu. Nếu có bất kỳ lỗi nào xảy ra trong quá trình cập nhật, procedure sẽ rollback giao dịch (ROLLBACK TRANSACTION), hủy bỏ mọi thay đổi đã thực hiện và in ra thông báo lỗi.

3.2 Function

STT	Function
3.2.1	Tên function: dbo.DoanhThu Tham số truyền vào: @ngay DATE Mã lệnh PL/SQL: <pre> CREATE FUNCTION dbo.DoanhThu (@ngay DATE) RETURNS MONEY AS BEGIN DECLARE @tongTienKham MONEY; SELECT @tongTienKham = SUM(tongTienKham) FROM BienLai WHERE CAST(ngayTao AS DATE) = @ngay; </pre>

	<pre> RETURN ISNULL(@tongTienKham, 0); END; </pre> <u>Mô tả mã lệnh:</u> Function ở trên có tên là dbo.DoanhThu, có tham số truyền vào là một biến @ngay có kiểu dữ liệu là DATE và các hành động được thực hiện bao gồm: Khởi tạo biến @tongTienKham kiểu dữ liệu là MONEY để lưu trữ tổng tiền khám theo ngày. Sử dụng câu lệnh SELECT @tongTienKham = SUM(tongTienKham) FROM BienLai WHERE CAST(ngayTao AS DATE) = @ngay; để lấy ra giá trị của tổng các biên lai trong ngày và gán vào biến @tongTienKham. Hàm CAST được dùng để chuyển đổi giá trị của cột ngayTao từ kiểu dữ liệu SMALLDATETIME sang kiểu dữ liệu DATE. Cuối cùng là trả về giá trị của biến @tongTienKham. Nếu @tongTienKham = null thì trả về 0. Function này không thực hiện bất kì theo tác nào trên CSDL, vì vậy không cần thực hiện commit.
3.2.2	<u>Tên function:</u> dbo.SoLuongKhamTrongNgay <u>Tham số truyền vào:</u> @ngay kiểu dữ liệu DATE <u>Mã lệnh PL/SQL:</u> CREATE FUNCTION dbo.SoLuongKhamTrongNgay (@ngay DATE) RETURNS int AS BEGIN DECLARE @SoLuongBenhNhan int; SELECT @SoLuongBenhNhan = count(*) FROM BienLai WHERE CAST(ngayTao AS DATE) = @ngay; RETURN ISNULL(@SoLuongBenhNhan, 0); END; <u>Mô tả mã lệnh:</u> Function ở trên có tên là: dbo.SoLuongKhamTrongNgay, có tham số truyền vào là một biến @ngay có kiểu dữ liệu là DATE và các hành động được thực hiện bao gồm: Khởi tạo biến @ SoLuongBenhNhan kiểu dữ liệu là Int để lưu số lượng khám bệnh trong ngày. Sử dụng câu SELECT @SoLuongBenhNhan = count(*) FROM BienLai WHERE CAST(ngayTao AS DATE) = @ngay; Để đếm số lượng các biên lai được tạo ra trong ngày và gán vào biến @ SoLuongBenhNhan.

	Hàm CAST được dùng để chuyển đổi giá trị của cột ngayTao từ kiểu dữ liệu SMALLDATETIME sang kiểu dữ liệu DATE. Cuối cùng là trả về giá trị của biến @SoLuongBenhNhan. Nếu @SoLuongBenhNhan = null thì trả về 0. Function này không thực hiện bất kì theo tác nào trên CSDL, vì vậy không cần thực hiện commit.
3.2.3	Tên function: TONGLANKHAM Tham số truyền vào: @MABN với kiểu dữ liệu INT Mã lệnh T-SQL: <pre>CREATE OR ALTER FUNCTION TONGLANKHAM (@MABN INT) RETURNS INT AS BEGIN DECLARE @TONGLANKHAM INT = 0; SELECT @TONGLANKHAM = COUNT(*) FROM PHIEUKHAMBENH WHERE MABN = @MABN; RETURN @TONGLANKHAM; END GO</pre> Mô tả mã lệnh: Function có tên TONGLANKHAM với đầu vào là @MABN với kiểu dữ liệu INT và các hành động sau đó: Khai báo biến @TONGLANKHAM: kiểu dữ liệu INT, đặt giá trị ban đầu là 0, lưu trữ giá trị tổng lần khám Thực hiện lệnh SELECT: giá trị @TONGLANKHAM sẽ bằng COUNT tất cả các bản ghi có @MABN. Trả về giá trị @TONGLANKHAM
3.2.4	Tên function: TONGBIENLAI Tham số truyền vào: @NGAY với kiểu dữ liệu SMALLDATETIME Mã lệnh T-SQL: <pre>CREATE OR ALTER FUNCTION TONGBIENLAI (@NGAY SMALLDATETIME) RETURNS INT AS BEGIN DECLARE @TONGBIENLAI INT; SELECT @TONGBIENLAI = COUNT(*)</pre>

	FROM PHIEUKHAMBENH WHERE NGAYTAO = @NGAY; RETURN @TONGBIENLAI; END GO Mô tả mã lệnh: Tên function là TONGBIENLAI với tham số truyền vào là @NGAY với kiểu dữ liệu SMALLDATETIME, hoạt động như sau: Khởi tạo biến @TONGBIENLAI với kiểu dữ liệu INT để lưu trữ tổng số biên lai. Thực hiện SELECT: Giá trị của @TONGBIENLAI được thiết lập lại bằng COUNT các bản ghi có NGAYTAO = @NGAY Trả về giá trị @TONGBIENLAI

PHẦN III. XỬ LÝ ĐỒNG THỜI

1. Tổng quan

a. Giao tác trong Oracle

Một giao tác là một đơn vị thao tác luận lý bao gồm một hoặc nhiều câu lệnh SQL, được thực thi bởi một người dùng đơn. Trong Oracle, một giao tác bắt đầu bằng việc thực thi câu lệnh SQL đầu tiên của người dùng. Và kết thúc khi một trong những điều sau xảy ra:

- o Người dùng sử dụng lệnh COMMIT hoặc ROLLBACK mà không một điểm đánh dấu SAVEPOINT.
- o Người dùng chạy một câu lệnh DDL chẳng hạn như CREATE, DROP, RENAME, hay ALTER.
- o Người dùng ngắt kết nối với Oracle. Giao tác hiện tại sẽ được commit.
 - o Xử lý của người dùng bị ngắt một cách bất thường. Giao tác hiện tại sẽ bị rollback.

Sau khi một giao tác kết thúc, giao tác tiếp theo sẽ bắt đầu với câu lệnh SQL kế tiếp. Các câu lệnh kiểm soát giao tác:

- o Ghi nhận vĩnh viễn những thay đổi được thực hiện trong giao tác (COMMIT).

- o Quay ngược lại những thay đổi của giao tác, tính từ lúc giao tác bắt đầu hoặc từ một điểm savepoint (ROLLBACK).

- o Đặt một điểm mà có thể rollback (SAVEPOINT)

- o Thiết đặt thuộc tính cho giao tác (SET TRANSACTION)

- SET TRANSACTION ISOLATION LEVEL {READ COMMITTED | SERIALIZABLE};

- SET TRANSACTION READ {ONLY|WRITE};

- o Điều chỉnh khi nào Oracle tiến hành commit (SET AUTOCOMMIT)

- SET AUTOCOMMIT ON: commit ngay những thay đổi xuống cơ sở dữ liệu sau khi Oracle thực thi thành công các câu lệnh INSERT, UPDATE hoặc DELETE

- SET AUTOCOMMIT OFF (mặc định): ngăn không để Oracle commit tự động, người dùng phải commit thủ công.

b. Vấn đề trong môi trường truy xuất đồng thời

Mất dữ liệu cập nhật (Lost Update): Tình trạng này xảy ra khi có nhiều hơn một giao tác cùng thực hiện cập nhật trên 1 đơn vị dữ liệu. Khi đó, tác dụng của giao tác cập nhật thực hiện sau sẽ đè lên tác dụng của thao tác cập nhật trước.

Đọc dữ liệu chưa commit (Uncommitted data, Dirty read): Xảy ra khi một giao tác thực hiện đọc trên một đơn vị dữ liệu mà đơn vị dữ liệu này đang bị cập nhật bởi một giao tác khác nhưng việc cập nhật chưa được xác nhận.

Giao tác đọc không thể lặp lại (Non-repeatable read): Tình trạng này xảy ra khi một giao tác T1 vừa thực hiện xong thao tác đọc trên một đơn vị dữ liệu (nhưng chưa commit) thì giao tác khác (t2) lại thay đổi (ghi) trên đơn vị dữ liệu này. Điều này làm cho lần đọc sau đó của T1 không còn nhìn thấy dữ liệu ban đầu nữa.

Bóng ma (Phantom read): Là tình trạng mà một giao tác đang thao tác trên một tập dữ liệu nhưng giao tác khác lại chèn thêm các dòng dữ liệu vào tập dữ liệu mà giao tác kia quan tâm.

c. Các phương thức khóa cơ bản

Shared Lock (S):

- o Shared Lock □ Read Lock

- o Khi đọc 1 đơn vị dữ liệu, hệ quản trị tự động thiết lập Shared Lock trên đơn vị dữ liệu đó (trừ trường hợp sử dụng No Lock)

- o Shared Lock có thể được thiết lập trên 1 bảng, 1 trang, 1 khóa hay trên 1 dòng dữ liệu.

- o Nhiều giao tác có thể đồng thời giữ Shared Lock trên cùng 1 đơn vị dữ liệu.

- o Không thể thiết lập Exclusive Lock trên đơn vị dữ liệu đang có Shared Lock.

- o Shared Lock thường được giải phóng ngay sau khi sử dụng xong dữ liệu được đọc, trừ khi có thiết lập giữ shared lock cho đến hết giao tác.

Exclusive Lock (X):

- o Exclusive Lock □ Write Lock

- o Khi thực hiện thao tác ghi (insert, update, delete) trên 1 đơn vị dữ liệu, hệ quản trị tự động thiết lập Exclusive Lock trên đơn vị dữ liệu đó

- o Exclusive Lock luôn được giữ đến hết giao tác.

- o Tại 1 thời điểm, chỉ có tối đa 1 giao tác được quyền giữ Exclusive Lock trên 1 đơn vị dữ liệu.

- o Không thể thiết lập Exclusive Lock trên đơn vị dữ liệu đang có Shared Lock.

d. Mức cô lập**Read uncommitted:****❖ Đặc điểm:**

- o Không thiết lập Shared Lock trên những đơn vị dữ liệu cần đọc. Do đó không phải chờ khi đọc dữ liệu (kể cả khi dữ liệu đang bị lock bởi giao tác khác)

- o (Vẫn tạo Exclusive Lock trên đơn vị dữ liệu được ghi, Exclusive Lock được giữ cho đến hết giao tác).

❖ Ưu điểm

- o Tốc độ xử lý nhanh

- o Không cản trở những giao tác khác thực hiện việc cập nhật dữ liệu

❖ Khuyết điểm

Có khả năng xảy ra mọi vấn đề xử lý đồng thời:

- o Dirty Read
- o Non-repeatable Read
- o Phantom Read
- o Lost update

Read Committed:

❖ Đặc điểm

- o Đây là mức cô lập mặc định của Oracle/SQL Server
- o Tạo Shared Lock trên đơn vị dữ liệu được đọc, Shared Lock được giải phóng ngay sau khi đọc xong dữ liệu
- o Tạo Exclusive Lock trên đơn vị dữ liệu được ghi, Exclusive Lock được giữ cho đến hết giao tác

❖ Ưu điểm

- o Giải quyết vấn đề Dirty Reads
- o Shared Lock được giải phóng ngay, không cần phải giữ cho đến hết giao tác nên không cản trở nhiều đến thao tác cập nhật của giao tác khác

❖ Khuyết điểm

- o Chưa giải quyết được vấn đề Non-repeatable Read, Phantom Read, Lost Update
- o Phải chờ nếu đơn vị dữ liệu cần đọc đang được giữ khóa ghi (xlock)

Repeatable Read

❖ Đặc điểm

- o Tạo Shared Lock trên đơn vị dữ liệu được đọc và giữ shared lock này đến hết giao tác → các giao tác khác phải chờ đến khi giao tác này kết thúc nếu muốn cập nhật, thay đổi giá trị trên đơn vị dữ liệu này.

o (Repeatable Read = Read Committed + Giải quyết Non-repeatable Read)

o Tạo Exclusive Lock trên đơn vị dữ liệu được ghi, Exclusive Lock được giữ cho đến hết giao tác.

❖ Ưu điểm

o Giải quyết vấn đề Dirty Read và Non-repeatable Read

❖ Khuyết điểm

o Chưa giải quyết được vấn đề Phantom Read, do vẫn cho phép insert những dòng dữ liệu thỏa điều kiện thiết lập shared lock

o Phải chờ nếu đơn vị dữ liệu cần đọc đang được giữ khóa ghi (xlock)

o Shared lock được giữ đến hết giao tác → cản trở việc cập nhật dữ liệu của các giao tác khác

Serializable

❖ Đặc điểm

o Tạo Shared Lock trên đơn vị dữ liệu được đọc và giữ shared lock này đến hết giao tác → Các giao tác khác phải chờ đến khi giao tác này kết thúc nếu muốn cập nhật, thay đổi giá trị trên đơn vị dữ liệu này.

o Không cho phép Insert những dòng dữ liệu thỏa mãn điều kiện thiết lập Shared Lock (sử dụng Key Range Lock) → Serializable =

Repeatable Read + Giải quyết Phantom Read

o Tạo Exclusive Lock trên đơn vị dữ liệu được ghi, Exclusive Lock được giữ cho đến hết giao tác.

❖ Ưu điểm

o Giải quyết thêm được vấn đề Phantom Read

❖ Khuyết điểm

o Phải chờ nếu đơn vị dữ liệu cần đọc đang được giữ khóa ghi (xlock)

o Cản trở nhiều đến việc cập nhật dữ liệu của các thao tác khác

2. Mô tả kịch bản gây mất nhất quán dữ liệu, giải pháp đề ra trong đồ án môn học

a. Trường hợp Lost update

Tình huống 1

Mô tả tình huống: Khi bác sĩ A thực hiện giao dịch thêm một loại thuốc vào một chi tiết đơn thuốc mà chưa hoàn tất, cùng lúc bác sĩ B thực hiện hành động tương tự trên loại thuốc đó. Điều này dẫn đến việc mất dữ liệu.

Nguyên nhân: Khi bác sĩ A thực hiện giao dịch thêm một loại thuốc mà chưa toàn tất, số lượng tồn của loại thuốc đó đã bị thay đổi nhưng giao dịch của Bác sĩ B lại không nhìn thấy được. Dẫn đến khi Bác sĩ B thực hiện thêm tương tự loại thuốc đó, vì số lượng tồn của loại thuốc đã bị thay đổi nhưng chưa commit, nên bác sĩ B đọc được giá trị số lượng tồn ban đầu của loại thuốc thay vì giá trị mới được cập nhật từ giao dịch của bác sĩ A. Điều này dẫn tới giá trị số lượng tồn thực sự của loại thuốc bị ghi đè và mất đi.

Ở mức DBMS: sử dụng procedure để xử lý việc thêm một đơn thuốc mới và cập nhật số lượng tồn.

Procedure

```
CREATE PROCEDURE sp_ThemThuocChiTietDonThuoc
    @MaThuoc INT,
    @SoLuongCanLay INT,
    @MaToa INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @SoLuongTon INT;

    SELECT @SoLuongTon = SOLUONGTON FROM THUOC WHERE id =
    @MaThuoc;
    PRINT N'Số lượng tồn hiện tại: ' + CAST(@SoLuongTon AS NVARCHAR(20));

    WAITFOR DELAY '00:00:05';

    IF @SoLuongTon >= @SoLuongCanLay
    BEGIN
        UPDATE THUOC
        SET SOLUONGTON = @SoLuongTon - @SoLuongCanLay
        WHERE id = @MaThuoc;

    DECLARE @SoLuong INT;
    SELECT @SoLuong = SOLUONGTON FROM THUOC WHERE id = @MaThuoc;
    SET @SoLuongTon = @SoLuong;
```

```

PRINT N'Số lượng tồn sau khi cập nhật: ' + CAST(@SoLuong AS NVARCHAR(20));

    IF NOT EXISTS (SELECT 1 FROM CTDONTHUOC WHERE MATHUOC =
@MaThuoc AND MATOA = @MaToa)
    BEGIN
        INSERT INTO CTDONTHUOC (MATHUOC, MATOA, SOLUONG)
        VALUES (@MaThuoc, @MaToa, @SoLuongCanLay);
    END
    ELSE
    BEGIN
        UPDATE CTDONTHUOC
        SET SOLUONG = SOLUONG + @SoLuongCanLay
        WHERE MATHUOC = @MaThuoc AND MATOA = @MaToa;
    END

END
ELSE
BEGIN
    PRINT N'Số lượng tồn không đủ để thêm vào đơn thuốc.';
END
END;

```

Mô tả tình huống ở mức Hệ quản trị cơ sở dữ liệu SQL Server

T1 (Bác sĩ A thực hiện thêm loại thuốc Isulin, có id=7, số lượng 30 viên cho toa thuốc 1 vào bảng CTDONTHUOC)	T2 (Bác sĩ B thực hiện thêm loại thuốc Isulin, có id=7, số lượng 40 viên cho toa thuốc 2 vào bảng CTDONTHUOC)
<i>Bước 1</i> SET TRANSACTION ISOLATION LEVEL READ COMMITTED BEGIN TRANSACTION WAITFOR DELAY '00:00:05' EXEC sp_ThemThuocChiTietDonThuoc @MaThuoc = 7, @SoLuongCanLay = 30, @MaToa = '1'; COMMIT TRANSACTION;	<i>Bước 2</i> SET TRANSACTION ISOLATION LEVEL READ COMMITTED BEGIN TRANSACTION WAITFOR DELAY '00:00:06' EXEC sp_ThemThuocChiTietDonThuoc @MaThuoc = 7, @SoLuongCanLay = 40, @MaToa = '2'; COMMIT TRANSACTION;
<i>T1 hoàn thành trước T2</i>	

121 % Messages Số lượng tồn hiện tại: 50 Số lượng tồn sau khi cập nhật: 20 Completion time: 2024-05-29T20:52:24.5020334+07:00	
	121 % Messages Số lượng tồn hiện tại: 50 Số lượng tồn sau khi cập nhật: 10 Completion time: 2024-05-29T20:52:27.2248418+07:00

Chạy lần lượt các bước theo thứ tự: 1,2
Thu được kết quả như sau:

The screenshot displays the Microsoft SQL Server Management Studio interface. The left pane shows the 'Messages' window with the following text:

```

UPDATE THUOC SET SOLUONGTON = '50' WHERE id = 7
DELETE FROM CTDONTHUOC WHERE MATHUOC = 7
select * from thuoc

```

The right pane shows the 'Results' window with a table of drug inventory. The table has columns: id, TENTHUC, NUOCSX, DONGIA, HSD, SOLUONGTON, tinhTrang. The data is as follows:

id	TENTHUC	NUOCSX	DONGIA	HSD	SOLUONGTON	tinhTrang
1	Paracetamol	Việt Nam	10000.00	2025-05-10 00:00:00	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
3	Hydroxychl...	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
4	Meflomin	Hoa Kỳ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
5	Glipitine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
6	Sulfonyleure...	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
7	Insulin	Mỹ	20000.00	2025-06-15 00:00:00	10	Còn thuốc
8	Dexameth...	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc

The bottom status bar indicates 'Query executed successfully'.

Kết quả hiển thị cả 2 đơn thuốc đều được thêm vào bảng CTDONTHUOC nhưng ở cột SoLuongTon của bảng Thuoc là 10 => Mất dữ liệu.

Nguyên nhân: Khi transaction T1 thực hiện thêm 1 CTDONTHUOC với 1 loại thuốc nhưng không commit thì T2 vào thực hiện thêm 1 CTDONTHUOC với cùng loại thuốc đó nên dữ liệu thực sự bị T2 ghi đè. => Mất dữ liệu.

Mô tả tình huống ở mức chương trình

Ban đầu số lượng tồn thuốc của thuốc có id = 7 là 50.

QUẢN LÝ TỒN KHO THUỐC

Trang Chủ

Nhập tên thuốc

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Màn hình 1: Bác sĩ A thực hiện thêm loại thuốc Isulin, có id=7, số lượng 30 viên cho toa thuốc 1 vào bảng CTDONTHUOC).

Màn hình 2: Bác sĩ B thực hiện thêm loại thuốc Isulin, có id=7, số lượng 40 viên cho toa thuốc 2 vào bảng CTDONTHUOC

Kê Toa Thuốc

Mã Khám Bệnh: PKB00008
 Mã Bác Sĩ: 1
 Mã Bệnh Nhân: 3
 Tên phòng khám: Phòng điều trị tiểu đường
 Ngày tạo: June 1, 2024
 Chẩn đoán: Tiểu đường

PKB ID: 8

Mã Toa: 5

STT	Ma Thuoc	Ten Thuoc	So luong	Ghi Chu
0	7	Insulin	30	sáng 1 viên

Tổng hoá đơn: 600000.0

Kê Toa Thuốc

Mã Khám Bệnh: PKB00009
 Mã Bác Sĩ: 2
 Mã Bệnh Nhân: 4
 Tên phòng khám: Phòng điều trị tiểu đường
 Ngày tạo: June 1, 2024
 Chẩn đoán: Tiểu đường

PKB ID: 9

Mã Toa: 6

STT	Ma Thuoc	Ten Thuoc	So luong	Ghi Chu
0	7	Insulin	40	sáng 1 viên

Tổng hoá đơn: 800000.0

T1,T2 tiến hành kê toa thuốc đồng thời. Sau khi kê toa thì tiến hành thêm chi tiết đơn thuốc 1 cách đồng thời.

Phiếu khám bệnh ở T1 đã cập nhật chi tiết đơn thuốc thành công.

Mã Toa	Mã Thuốc	Tên Thuốc	Số Lượng	Đơn Giá
TOA00005	7	Insulin	30	20000.0

Phiếu khám bệnh ở T2 đã cập nhật chi tiết đơn thuốc thành công.

Mã Toa	Mã Thuốc	Tên Thuốc	Số Lượng	Đơn Giá
TOA00006	7	Insulin	40	20000.0

Nhưng kết quả cuối cùng của số lượng tồn của thuốc là 10 => Mất dữ liệu.

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	10	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Giải pháp cho tình huống Lost update: có thể dùng mức cô lập REPEATABLE READ, SNAPSHOT hoặc SERIALIZABLE để giải quyết.

Khi dùng các mức cô lập này thì hệ thống sẽ chọn 1 transaction làm nạn nhân cho deadlock, và transaction còn lại thực thi thành công.

T1 được chọn là nạn nhân cho deadlock nên phải dừng để cho T2 chạy.

Ở T1 sẽ xuất hiện lỗi

Msg 1205, Level 13, State 51, Line 13

Transaction (Process ID 57) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction.

T2 thì cập nhật số lượng tồn là 10 và thêm vào CTDONTHUOC thành công

Vì mức cô lập này chọn T2 thực thi trong vòng deadlock nên tránh được lost update.

Tình huống 2

Mô tả tình huống: Khi bác sĩ A thực hiện giao dịch thêm tất cả số thuốc còn lại của một loại thuốc trong kho vào một chi tiết đơn thuốc mà chưa hoàn tất, cùng lúc đó quản lý kho thực hiện hành động cập nhật số lượng thuốc tồn tương ứng với loại thuốc đó. Điều này dẫn đến việc mất dữ liệu.

Nguyên nhân: Khi bác sĩ A thực hiện giao dịch thêm tất cả số lượng còn lại của một loại thuốc vào chi tiết đơn thuốc mà chưa toàn tất, số lượng tồn của loại thuốc đó đã bị thay đổi nhưng giao dịch của quản lý kho lại không nhìn thấy được. Dẫn đến khi quản lý kho thực hiện thêm số lượng cho loại thuốc đó, vì số lượng tồn của loại thuốc đã bị thay đổi nhưng chưa commit bởi Bác sĩ A, nên quản lý thêm mới loại thuốc dựa trên số lượng tồn ban đầu. Điều này dẫn đến giá trị số lượng tồn thực sự của loại thuốc bị ghi đè và mất đi.

Ở mức DBMS: sử dụng procedure để xử lý việc thêm một đơn thuốc mới và cập nhật số lượng tồn.

PROCEDURE	
Procedure thêm chi tiết đơn thuốc ở T1	Procedure để update số lượng tồn ở T2
<pre> CREATE PROCEDURE sp_ThemThuocChiTietDonThuoc_1 @MaThuoc INT, @SoLuongCanLay INT null, @MaToa INT AS BEGIN SET NOCOUNT ON; DECLARE @SoLuongTon INT; SELECT @SoLuongTon = SOLUONGTON FROM THUOC WHERE id = @MaThuoc; PRINT N'Số lượng tồn hiện tại: ' + CAST(@SoLuongTon AS NVARCHAR(20)); --50 SET @SoLuongCanLay = @SoLuongTon PRINT N'Số lượng cần lấy: ' + CAST(@SoLuongCanLay AS NVARCHAR(20)); --50 WAITFOR DELAY '00:00:10'; IF NOT EXISTS (SELECT 1 FROM CTDONTHUOC WHERE MATHUOC = @MaThuoc AND MATOA = @MaToa) BEGIN INSERT INTO CTDONTHUOC (MATHUOC, MATOA, SOLUONG) </pre>	<pre> CREATE PROCEDURE sp_CapNhatSLT @id int, @SOLUONGTON int AS BEGIN DECLARE @SoLuong INT; SELECT @SoLuong = SOLUONGTON FROM THUOC WHERE id = @id; PRINT N'Số lượng tồn hiện tại: ' + CAST(@SoLuong AS NVARCHAR(20)); --WAITFOR DELAY '00:00:10' UPDATE THUOC SET SOLUONGTON = SOLUONGTON + @SOLUONGTON WHERE id = @id SELECT @SoLuong = SOLUONGTON FROM THUOC WHERE id = @id; PRINT N'Số lượng tồn hiện tại: ' + CAST(@SoLuong AS NVARCHAR(20)); END; </pre>

```
VALUES (@MaThuoc, @MaToa,
@SoLuongCanLay);
END
ELSE
BEGIN
UPDATE CTDONTHUOC
SET SOLUONG = SOLUONG +
@SoLuongCanLay
WHERE MATHUOC = @MaThuoc
AND MATOA = @MaToa;
END
END;
```

Mô tả tình huống ở mức Hệ quản trị cơ sở dữ liệu SQL Server

T1 (Bác sỹ A thêm toàn bộ số lượng thuốc có id = 7)	T2 (Quản lý kho thực hiện tăng số lượng của thuốc có id = 7)
<pre>SET TRANSACTION ISOLATION LEVEL READ COMMITTED BEGIN TRANSACTION EXEC sp_ThemThuocChiTietDonThuoc_1 '7', ', '2'; COMMIT TRANSACTION</pre>	<pre>SET TRANSACTION ISOLATION LEVEL READ COMMITTED BEGIN TRANSACTION EXEC sp_CapNhatSLT '7','30'; COMMIT TRANSACTION</pre>

Số lượng tồn ban đầu của thuốc có id = 7 là 50.

Thực hiện đồng thời hai transaction T1, T2 thu được kết quả như sau:

The screenshot displays the Microsoft SQL Server Management Studio interface. On the left, the Object Explorer shows the database structure. The main window shows two SQL queries being executed simultaneously. The left query (T1) updates the quantity of drug id=7 to 50. The right query (T2) updates the quantity of drug id=7 to 30. The Results pane at the bottom right shows the final state of the 'thuoc' table, where the quantity for drug id=7 is 50.

id	TENTHUCOC	NUOCSX	DONGIA	HSD	SOLUONGTON	TRANGTHAI
1	Paracetamol	Viet Nam	10000.00	2025-05-10 00:00:00	0	Đã hết thuốc
2	Ibuprofen	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
3	Hydroxychloroquine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
5	Glipizins	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
7	Insulin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc

T1 và T2 thực thi đồng thời và thành công.

SQLQuery5.sql - ANDREW\SQLEXPRESS.QuanLyThongTinBenhAn (sa (54))*

```

END;

EXEC sp_ThemThuocChiTietDonThuoc_1 '7', '50', '2'

COMMIT TRANSACTION

select * from ctdonthuoc
update thuoc set soluongton = 50 where id = 7

```

Results

	MATHUOC	MATOA	SOLUONG	GHICHU
1	1	2	10	Trúa 1 viên
2	1	11	50	NULL
3	2	2	10	Trúa 1 viên
4	2	7	30	NULL
5	2	8	40	NULL
6	3	2	10	Trúa 1 viên
7	4	1	10	Sáng 1 viên
8	5	1	10	Sáng 1 viên
9	6	1	10	Sáng 1 viên
10	7	2	50	NULL
11	7	5	30	NULL

SQLQuery4.sql - AN...inBenhAn (sa (54))*

```

BEGIN TRANSACTION
EXEC sp_CapNhatSLT '7','30';
COMMIT TRANSACTION

select * from thuoc

```

Results

	id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	TRANGTHAI
1	1	Paracetamol	Việt Nam	10000.00	2025-05-10 00:00:00	0	Đã hết thuốc
2	2	Ibuprofen	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
3	3	Hydroxychloroquine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
4	4	Metformin	Hoa Kỳ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
5	5	Gliptins	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
6	6	Sulfonylureas	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
7	7	Insulin	Mỹ	20000.00	2025-06-15 00:00:00	80	Còn thuốc
8	8	Dexamethasone	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
10	10	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
11	11	Loratadine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc

Kết quả hiển thị cả transaction đều thực thi thành công, T1 thực thi thành công thêm 1 CTDONTHUOC mới có MATHUOC = 7, cho MATOA = 2 với SOLUONG = 50, T2 thực thi thành công update SOLUONG = 30 cho THUOC có id = 7. Nhưng SOLUONG sau khi T2 update là 80=> Mất dữ liệu.

Nguyên nhân: Khi T1 thực hiện thêm một CTDONTHUOC với tất cả số lượng còn lại của thuốc đó nhưng chưa commit thì T2 cập nhật ghi đè làm mất dữ liệu.

Mô tả tình huống ở mức chương trình

Ban đầu số lượng tồn của thuốc với id = 7 có giá trị là 50.

Trang Chủ

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	0	Đã hết thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloroq...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	0	Đã hết thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Sau đó T1 thực hiện thêm CTDONTHUOC với thuốc có id = 7 và số lượng là 50.

Kê Toa Thuốc

Mã Khám Bệnh: PKB00009

Mã Bác Sĩ: 1

Mã Bệnh Nhân: 6

Tên phòng khám: Phòng điều trị tiểu đường

Ngày tạo: June 2, 2024

Chẩn đoán: Tiểu đường

Thêm chẩn đoán

PKB ID: 9

Tạo toa thuốc

Mã Toa: 16

Thêm chi tiết đơn thuốc

STT	Ma Thuoc	Ten Thuoc	So luong	Ghi Chu
0	7	Insulin	50	

Tạo CTToaThuoc

Tổng hoá đơn

Quay Lại

T2 thực thi đồng thời với T1 là cập nhật số lượng thuốc tăng 30 so với số lượng ban đầu

Kê Toa Thuốc

Mã Khám Bệnh: PKB00009

Mã Bác Sĩ: 1

Mã Bệnh Nhân: 6

Tên phòng khám: Phòng điều trị tiểu đường

Ngày tạo: June 2, 2024

Chẩn đoán: Tiểu đường

Thêm chẩn đoán

PKB ID: 9

Tạo toa thuốc

Mã Toa: 16

Thêm chi tiết đơn thuốc

STT	Ma Thuoc	Ten Thuoc	So luong	Ghi Chu
0	7	Insulin	50	

Tạo CTToaThuoc

Tổng hoá đơn

Quay Lại

SỬA THÔNG TIN THUỐC

ID: 7

Tên thuốc: Insulin

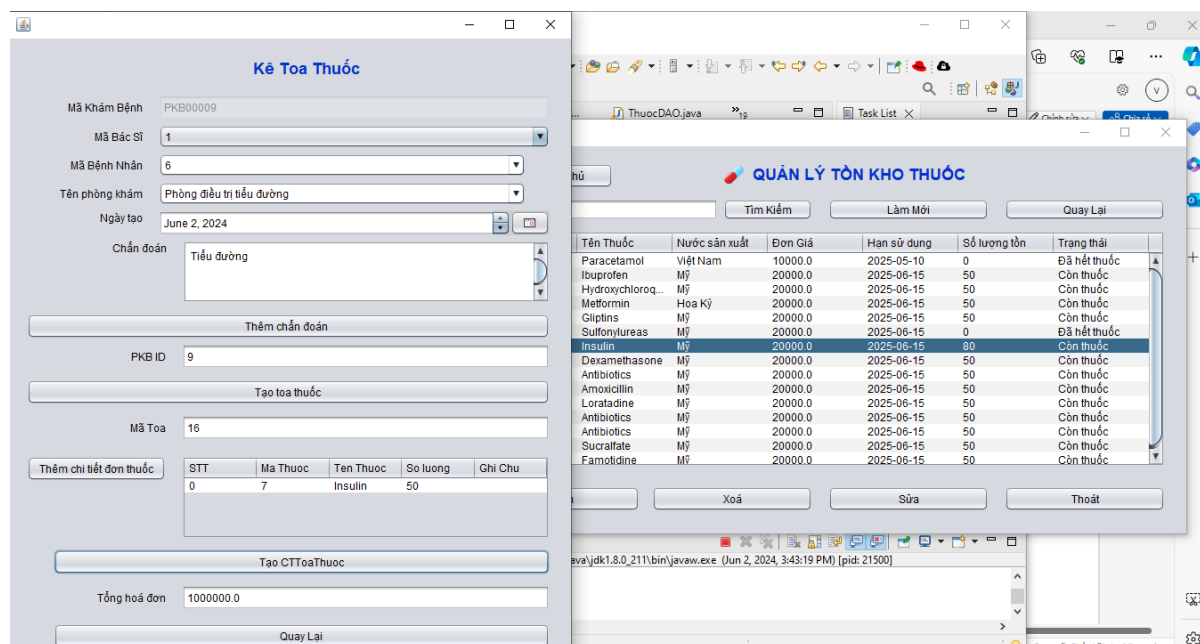
Nước sản xuất: Mỹ

Đơn giá: 20000.0

HSD: June 15, 2025

Số lượng tồn: 30

Lưu Quay Lại



Kết quả là 2 transactions đều thực thi thành công với từng giao tác của mình. Tuy nhiên số T1 đã thực hiện lấy số lượng tồn của thuốc 50 để kê toa nhưng số lượng tồn của thuốc sau khi T2 cập nhật là 80 => Mất dữ liệu.

Nguyên nhân: Nguyên nhân: Khi T1 thực hiện thêm một CTDONTHUOC với tất cả số lượng còn lại của thuốc đó nhưng chưa commit thì T2 cập nhật ghi đè làm mất dữ liệu.

b. Trường hợp Dirty read

Tình huống 1

Mô tả tình huống: Khi nhân viên thanh toán A đang thực hiện cập nhật giá tiền của một loại thuốc, cùng lúc đó nhân viên B thực hiện kiểm tra giá tiền của cùng loại thuốc đó. Dẫn đến việc giao dịch của nhân viên B đọc sai giá trị tiền thuốc.

Nguyên nhân: Khi giao dịch của nhân viên A đang thực hiện sửa đổi một đơn vị dữ liệu, giao dịch của nhân viên B thực hiện đọc trên cùng đơn vị dữ liệu đó. Điều này dẫn đến đọc sai dữ liệu vì giao dịch của nhân viên A chưa hoàn tất (commit).

Ở mức DBMS: sử dụng procedure thêm thuốc để tái sử dụng mã lệnh và tăng hiệu suất chương trình.

Procedure

```
CREATE PROCEDURE CapNhatGiaThuoc
    @TENTHUOC NVARCHAR(50),
    @NUOCSX NVARCHAR(50),
    @DONGIA MONEY
AS
BEGIN
```



```

SET NOCOUNT ON;

BEGIN TRANSACTION CapNhatGiaThuocTransaction;

BEGIN TRY
    IF NOT EXISTS (SELECT 1 FROM THUOC WHERE TENTHUOC =
@TENTHUOC AND NUOCSX = @NUOCSX)
        BEGIN
            RAISERROR('Loại thuốc không tồn tại.', 16, 1);
            RETURN;
        END;

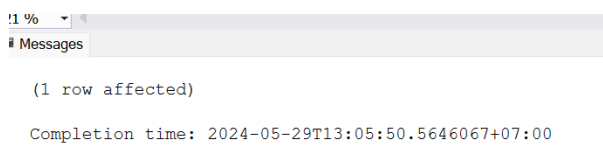
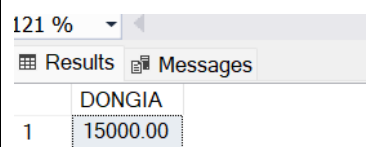
    UPDATE THUOC
    SET DONGIA = @DONGIA
    WHERE TENTHUOC = @TENTHUOC AND NUOCSX = @NUOCSX;

    COMMIT TRANSACTION CapNhatGiaThuocTransaction;
    PRINT N'Cập nhật giá thuốc thành công';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION CapNhatGiaThuocTransaction;
    PRINT N'Cập nhật giá thuốc thất bại. ' + ERROR_MESSAGE();
END CATCH;
END;

```

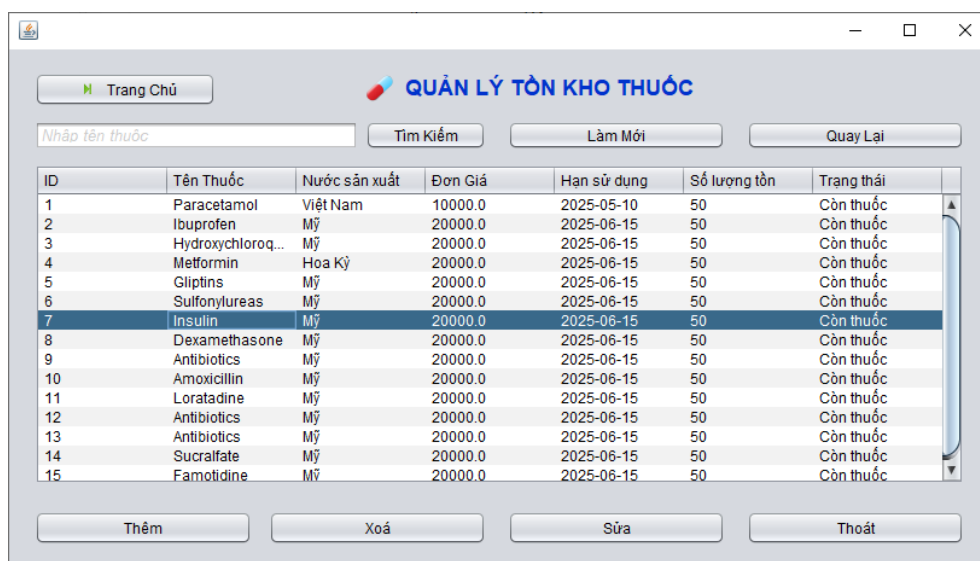
Mô tả tình huống trên hệ quản trị cơ sở dữ liệu SQL Server:

T1 (Nhân viên A thực hiện cập nhật giá của loại thuốc Isulin, có id = 7 trong bảng THUOC)	T2 (Nhân viên B thực hiện xem giá của thuốc Isulin, có id =7)
Bước 1 SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;	Bước 2 SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Bước 3 BEGIN TRANSACTION UPDATE THUOC SET DONGIA = 15000 WHERE id = 7; WAITFOR DELAY '00:00:10'	Bước 4 SELECT DONGIA FROM THUOC WHERE id = 7;

ROLLBACK TRANSACTION																																																																																	
Thực hiện chạy lần lượt theo thứ tự các bước 1, 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:10' để mô phỏng trạng thái chờ thay đổi từ T2																																																																																	
Kết quả ở T1: 	T2 thực hiện xong trước T1 																																																																																
Kết quả sau khi kết thúc T1, T2: <table border="1"> <thead> <tr> <th></th> <th>id</th> <th>TENTHUOC</th> <th>NUOCSX</th> <th>DONGIA</th> <th>HSD</th> <th>SOLUONGTON</th> <th>trinhTrang</th> </tr> </thead> <tbody> <tr><td>2</td><td>2</td><td>Ibuprofen</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> <tr><td>3</td><td>3</td><td>Hydroxychloroquine</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> <tr><td>4</td><td>4</td><td>Metformin</td><td>Hoa Kỳ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> <tr><td>5</td><td>5</td><td>Gliptins</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> <tr><td>6</td><td>6</td><td>Sulfonylureas</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> <tr><td>7</td><td>7</td><td>Insulin</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>10</td><td>Còn thuốc</td></tr> <tr><td>8</td><td>8</td><td>Dexamethasone</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> <tr><td>9</td><td>9</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> <tr><td>10</td><td>10</td><td>Amoxicillin</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr> </tbody> </table> Ban đầu loại thuốc Insulin có giá là 20000. Khi T1 thực hiện cập nhật giá thành 15000 thì T2 thực hiện đọc giá tiền của loại thuốc đó. Vì T1 thực hiện ROLLBACK nên quá trình cập nhật bị hủy bỏ, giá tiền thuốc không có sự thay đổi so với giá trị ban đầu. T2 đã đọc trong lúc T1 đang cập nhật dẫn đến việc đọc sai giá trị. Dirty read xảy ra			id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang	2	2	Ibuprofen	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	3	3	Hydroxychloroquine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	4	4	Metformin	Hoa Kỳ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	5	5	Gliptins	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	6	6	Sulfonylureas	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	7	7	Insulin	Mỹ	20000.00	2025-06-15 00:00:00	10	Còn thuốc	8	8	Dexamethasone	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	10	10	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
	id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang																																																																										
2	2	Ibuprofen	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
3	3	Hydroxychloroquine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
4	4	Metformin	Hoa Kỳ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
5	5	Gliptins	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
6	6	Sulfonylureas	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
7	7	Insulin	Mỹ	20000.00	2025-06-15 00:00:00	10	Còn thuốc																																																																										
8	8	Dexamethasone	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
10	10	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										

➔ Mô tả tình huống ở mức chương trình

Giá thuốc ban đầu của thuốc có id = 7 là 20000.



Màn hình 1: Nhân viên A thực hiện cập nhật giá của loại thuốc Insulin, có id = 7 trong bảng THUOC thành 15000

Trang Chủ

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc

Tìm Kiếm

Làm Mới

Quay Lại

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	15000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Thêm

Xoá

Sửa

Thoát

Cùng lúc đó thì T2 vào đọc cũng thấy được T1 đang update giá thuốc.

Trang Chủ

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc

Tìm Kiếm

Làm Mới

Quay Lại

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	15000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Thêm

Xoá

Sửa

Thoát

T1 thực hiện rollback. Nhưng T2 vẫn đang đọc giá thuốc cập nhật T1 ban đầu => dirty read.

Trang Chủ

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc

Tìm Kiếm

Làm Mới

Quay Lại

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	15000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Thêm

Xoá

Sửa

Thoát

Giải pháp cho trường hợp Dirty Read: Vì Dirty read chỉ xảy ra ở mức độ cô lập thấp nhất là READ UNCOMMITTED nên có thể sử dụng mức cô lập REPEATABLE READ, ngoài ra có thể dùng SNAPSHOT hoặc SERIALIZABLE để ngăn chặn dirty read.

Mô tả giải pháp

T1 (Nhân viên A thực hiện cập nhật giá của loại thuốc Insulin, có id = 7 trong bảng THUOC)	T2 (Nhân viên B thực hiện xem giá của thuốc Insulin, có id =7)																																																																																
Bước 1 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;	Bước 2 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;																																																																																
Bước 3 BEGIN TRANSACTION UPDATE THUOC SET DONGIA = 15000 WHERE id = 7; WAITFOR DELAY '00:00:10' ROLLBACK TRANSACTION	Bước 4 SELECT DONGIA FROM THUOC WHERE id = 7;																																																																																
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1 , 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:10' để mô phỏng trạng thái chờ thay đổi từ T2																																																																																	
Kết quả ở T1: 1 % Messages (1 row affected) Completion time: 2024-05-29T13:05:50.5646067+07:00	T2 thực hiện xong trước T1 121 % ResultsMessages <table><tr><td></td><td>DONGIA</td></tr><tr><td>1</td><td>20000.00</td></tr></table>		DONGIA	1	20000.00																																																																												
	DONGIA																																																																																
1	20000.00																																																																																
Kết quả sau khi kết thúc T1, T2: <table><tr><th></th><th>id</th><th>TENTHUOC</th><th>NUOCSX</th><th>DONGIA</th><th>HSD</th><th>SOLUONGTON</th><th>trinhTrang</th></tr><tr><td>2</td><td>2</td><td>Ibuprofen</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>3</td><td>3</td><td>Hydroxychloroquine</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>4</td><td>4</td><td>Metformin</td><td>Hoa Kỳ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>5</td><td>5</td><td>Gliptins</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>6</td><td>6</td><td>Sulfonylureas</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>7</td><td>7</td><td>Insulin</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>10</td><td>Còn thuốc</td></tr><tr><td>8</td><td>8</td><td>Dexamethasone</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>9</td><td>9</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>10</td><td>1...</td><td>Amoxicillin</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr></table>			id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang	2	2	Ibuprofen	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	3	3	Hydroxychloroquine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	4	4	Metformin	Hoa Kỳ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	5	5	Gliptins	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	6	6	Sulfonylureas	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	7	7	Insulin	Mỹ	20000.00	2025-06-15 00:00:00	10	Còn thuốc	8	8	Dexamethasone	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	10	1...	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc
	id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang																																																																										
2	2	Ibuprofen	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
3	3	Hydroxychloroquine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
4	4	Metformin	Hoa Kỳ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
5	5	Gliptins	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
6	6	Sulfonylureas	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
7	7	Insulin	Mỹ	20000.00	2025-06-15 00:00:00	10	Còn thuốc																																																																										
8	8	Dexamethasone	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										
10	1...	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																										

Ở mức cô lập **SERIALIZABLE** cho phép khóa exclusive đặt trên T1 khi thực hiện UPDATE được giữ nguyên đến khi giao dịch T1 hoàn tất (rollback), điều này ngăn chặn T2 thực hiện đặt khóa shared để đọc dữ liệu.
Dirty read được giải quyết

c. Trường hợp Non-repeatable read

Tình huống 1

Mô tả tình huống: Khi bác sỹ A đang thực hiện kiểm tra số lượng tồn của một loại thuốc trong hệ thống thì bác sỹ B đồng thời thực hiện thêm loại thuốc đó vào chi tiết đơn thuốc mới, làm cho số lượng tồn bị thay đổi trong quá trình kiểm tra. Điều này dẫn đến việc bác sỹ A đọc sai dữ liệu so với số lượng tồn ban đầu.

Nguyên nhân: Khi bác sỹ A thực hiện tra cứu số lượng một loại thuốc thì bác sỹ B thực hiện thêm loại thuốc đó vào chi tiết đơn thuốc mới, lúc này trigger trong hệ thống được kích hoạt để tự động cập nhật lại số lượng tồn mỗi khi một chi tiết đơn thuốc mới được tạo ra. Điều này dẫn đến việc số lượng tồn thực sự bị ghi đè và mất đi, bác sỹ A đọc được dữ liệu sai vì thay đổi của bác sỹ B.

Ở mức DBMS: Sử dụng procedure và trigger để xử lý, tái sử dụng mã lệnh và tự động hóa các thao tác có tính lặp lại nhiều lần:

Tăng hiệu suất vì procedure thực thi ở DBMS nhanh hơn so với việc gửi nhiều lệnh SQL từ máy khách.

Đảm bảo tính nhất quán dữ liệu vì trigger giúp kiểm soát và ngăn ngừa các lỗi không mong muốn xảy ra.

Procedure	Trigger
<pre>CREATE PROCEDURE ThemCTDonThuoc @MATHUOC INT, @MATOA INT, @SOLUONG INT AS BEGIN SET NOCOUNT ON; BEGIN TRANSACTION; BEGIN TRY</pre>	<pre>CREATE TRIGGER TRIG_UPDATE_THUOC ON CTDONTHUOC AFTER INSERT AS BEGIN SET NOCOUNT ON; DECLARE @MATHUOC INT, @SOLUONG INT;</pre>

```

IF NOT EXISTS (SELECT 1
FROM THUOC WHERE id =
@MATHUOC)
BEGIN
    RAISERROR('Mã thuốc không
tồn tại trong bảng THUOC.', 16, 1);
    RETURN;
END;
IF NOT EXISTS (SELECT 1
FROM TOATHUOC WHERE id =
@MATOA)
BEGIN
    RAISERROR('Mã toa thuốc
không tồn tại trong bảng TOATHUOC.',
16, 1);
    RETURN;
END;
IF EXISTS (SELECT 1 FROM
CTDONTHUOC WHERE MATHUOC
= @MATHUOC AND MATOA =
@MATOA AND SOLUONG =
@SOLUONG)
BEGIN
    RAISERROR ('Chi tiết đơn
thuốc đã tồn tại.', 16, 1);
    RETURN;
END;
-- Chèn dữ liệu vào bảng
CTDONTHUOC
INSERT INTO CTDONTHUOC
(MATHUOC, MATOA, SOLUONG)
VALUES (@MATHUOC,
@MATOA, @SOLUONG);
-- Commit transaction nếu mọi thứ
thành công
COMMIT TRANSACTION;
PRINT N'Thêm chi tiết đơn thuốc
thành công.';
END TRY
BEGIN CATCH
    -- Rollback transaction nếu có lỗi
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    -- In thông báo lỗi chi tiết
    PRINT N'Thêm chi tiết đơn thuốc
thất bại. ' + ERROR_MESSAGE();

```

```

SELECT @MATHUOC =
i.MATHUOC, @SOLUONG =
i.SOLUONG
FROM inserted i;

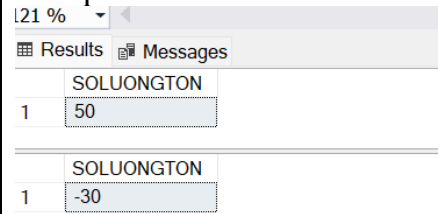
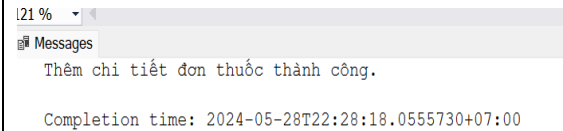
UPDATE THUOC
SET SOLUONGTON =
SOLUONGTON - @SOLUONG
WHERE id = @MATHUOC;

IF EXISTS (
    SELECT 1
    FROM THUOC
    WHERE id = @MATHUOC AND
    SOLUONGTON <= 0
)
BEGIN
    UPDATE THUOC
    SET tinhTrang = N'Đã hết thuốc'
    WHERE id = @MATHUOC;
END
END;

```

END CATCH; END;	
--------------------	--

Mô tả tình huống trên hệ quản trị cơ sở dữ liệu SQL Server:

T1 (Bác sỹ A thực hiện kiểm tra số lượng tồn của loại thuốc có id = 7, tên thuốc là Insulin trong bảng THUOC)	T2 (Bác sỹ B thực hiện thêm loại thuốc Insulin, id = 7 cho toa thuốc là 1 vào bảng CTDONTHUOC)
Bước 1 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Bước 2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
Bước 3 BEGIN TRANSACTION SELECT SOLUONGTON FROM THUOC WHERE id=7 WAITFOR DELAY '00:00:05' SELECT SOLUONGTON FROM THUOC WHERE id=7 COMMIT TRANSACTION	Bước 4 EXEC ThemCTDonThuoc '7', '1', '40';
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1 , 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:05' để mô phỏng trạng thái chờ thay đổi từ T2	
Kết quả ở T1: 	T2 thực hiện xong trước T1 
Lần đọc đầu tiên ở T1, giá trị số lượng tồn ban đầu là 50 Thực hiện chờ 5s để đợi T2 thay đổi. Lần đọc thứ hai ở T1, giá trị số lượng tồn bị thay đổi thành -30 Non-repeatable read xảy ra	

Mô tả tình huống ở mức chương trình

Màn hình 1: Bác sỹ A thực hiện kiểm tra số lượng tồn của thuốc Insulin, có id = 7 trong bảng THUOC

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Glipitins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Màn hình 2: Bác sỹ B thực hiện thêm loại thuốc Insulin có id = 7 đó cho toa thuốc có id = 1, với số lượng 40 viên vào bảng CTDONTHUOC

Mã Khám Bệnh: PKB00007

Mã Bác Sĩ: 1

Mã Bệnh Nhân: 1

Tên phòng khám: Phòng tiêu hóa

Ngày tạo: June 1, 2024

Chẩn đoán: Ăn không tiêu

Thêm chẩn đoán

PKB ID: 7

Tạo toa thuốc

Mã Toa: 7

STT	Ma Thuoc	Ten Thuoc	So luong	Ghi Chu
0	7	Insulin	40	

Tạo CTToaThuoc

Tổng hoá đơn: 800000.0

Quay Lại

Màn hình 1: Bác sỹ A thực hiện kiểm tra lại số lượng tồn của thuốc Insulin, có id = 7 trong bảng THUOC một lần nữa

Giải pháp cho trường hợp Non-repeatable Read: Sử dụng mức cô lập REPEATABLE READ, ngoài ra có thể dùng SNAPSHOT hoặc SERIALIZABLE.

Mở một cửa sổ truy vấn khác, thực hiện khôi phục dữ liệu bằng các lệnh sau:

```
SELECT * FROM CTDONTHUOC
```

```
SELECT * FROM THUOC
```

```
UPDATE THUOC SET SOLUONGTON = 50, tinhTrang = N'Còn thuốc'
```

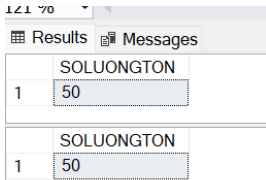
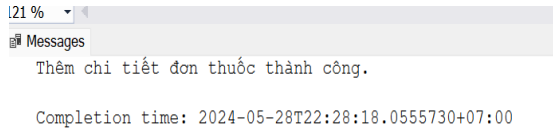
```
WHERE id = 7;
```

```
DELETE FROM CTDONTHUOC WHERE MATHUOC = 7;
```

Sau đó thực hiện thay đổi mức cô lập thành REPEATABLE READ

Mô tả tình huống trên hệ quản trị cơ sở dữ liệu SQL Server:

T1 (Bác sỹ A thực hiện kiểm tra số lượng tồn của loại thuốc có id = 7, tên thuốc là Insulin trong bảng THUOC)	T2 (Bác sỹ B thực hiện thêm loại thuốc Insulin, id = 7 cho toa thuốc là 1 vào bảng CTDONTHUOC)
Bước 1 SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;	Bước 2 SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Bước 3	Bước 4

BEGIN TRANSACTION SELECT SOLUONGTON FROM THUOC WHERE id=7 WAITFOR DELAY '00:00:05' SELECT SOLUONGTON FROM THUOC WHERE id=7 COMMIT TRANSACTION	EXEC ThemCTDonThuoc '7', '1', '40';
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1 , 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:05' để mô phỏng trạng thái chờ thay đổi từ T2	
Kết quả ở T1: 	T2 thực hiện xong trước T1 
Lần đọc đầu tiên, giá trị số lượng tồn ban đầu là 50. Thực hiện chờ 5s để đợi T2 thay đổi, vì mức độ cô lập REPEATABLE READ cho phép giữ nguyên khóa shared (S) đến khi giao dịch ở T1 hoàn tất (commit hoặc rollback), điều này ngăn cản T2 thực hiện cập nhật (hoặc xóa) trên cùng một hàng, dẫn đến việc ở lần đọc thứ hai giá trị số lượng tồn vẫn được bảo toàn là 50. Non-repeatable read được giải quyết	

Tình huống 2

Mô tả tình huống: Khi nhân viên thanh toán A đang kiểm tra số tiền khám của một biên lai, cùng lúc nhân viên thanh toán B cập nhật số tiền khám của biên lai đó. Điều này dẫn đến việc nhân viên A đọc sai dữ liệu so với số tiền khám ban đầu.

Nguyên nhân: Nhân viên A đọc một dòng dữ liệu hai lần và thấy dữ liệu khác nhau sau 2 lần đọc vì hệ thống đã cập nhật số tiền khám của biên lai từ thay đổi của nhân viên B.

Ở mức DBMS: Sử dụng procedure cập nhật biên lai để tăng hiệu suất vì procedure thực thi ở DBMS nhanh hơn so với việc gửi nhiều lệnh SQL từ máy khách.

Procedure
GO CREATE PROCEDURE PROC_UPDATE_BIENLAI @MABL VARCHAR(7), @TONGTIENKHAM MONEY NULL AS

```

BEGIN
    SET NOCOUNT ON;

    BEGIN TRANSACTION ;

    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM BIENLAI WHERE MABL =
@MABL)
            BEGIN
                RAISERROR('Bien lai không tồn tại.', 16, 1);
                RETURN;
            END;

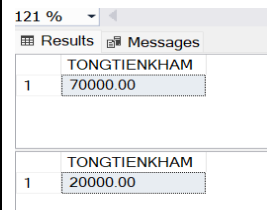
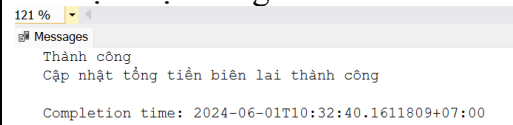
        UPDATE BIENLAI
            SET TONGTIENKHAM = @TONGTIENKHAM
            WHERE MABL = @MABL

        COMMIT TRANSACTION ;
        PRINT N'Cập nhật tổng tiền biên lai thành công';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION ;
        PRINT N'Cập nhật tổng tiền thất bại. ' + ERROR_MESSAGE();
    END CATCH;
END;

```

Mô tả tình huống trên hệ quản trị cơ sở dữ liệu SQL Server:

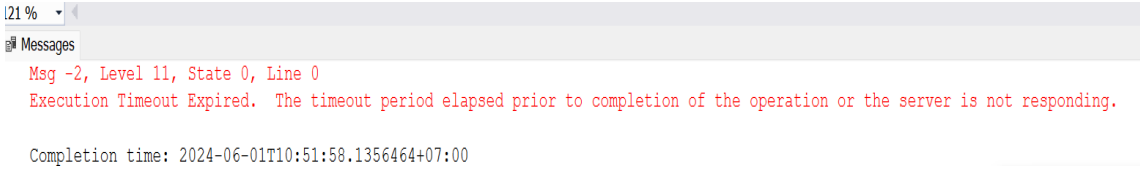
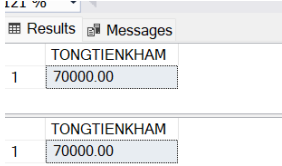
T1 (Nhân viên A đọc số tiền khám của biên lai BL00004 hai lần)	T2 (Nhân viên B cập nhật số tiền khám của BL00004)
Bước 1 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Bước 2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
Bước 3 BEGIN TRANSACTION	Bước 4 EXEC PROC_UPDATE_BIENLAI 'BL00004', '20000';

<pre>SELECT TONGTIENKHAM FROM BIENLAI WHERE MABL = 'BL00004' WAITFOR DELAY '00:00:10' SELECT TONGTIENKHAM FROM BIENLAI WHERE MABL = 'BL00004' COMMIT TRANSACTION</pre>	
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1 , 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:10' để mô phỏng trạng thái chờ thay đổi từ T2	
Kết quả ở T1: 	T2 thực hiện xong trước T1 
Lần đọc đầu tiên là giá trị số lượng tồn ban đầu là 70 Thực hiện chờ 10s để đợi T2 thay đổi Lần đọc thứ hai là giá trị số lượng tồn bị thay đổi thành 20 Non-repeatable read xảy ra	

Sử dụng mức cô lập SERIALIZABLE để giải quyết Non-repeatable read

Mô tả tình huống trên hệ quản trị cơ sở dữ liệu SQL Server:

T1	T2
Bước 1 <pre>SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;</pre>	Bước 2 <pre>SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;</pre>
Bước 3 <pre>BEGIN TRANSACTION SELECT TONGTIENKHAM FROM BIENLAI WHERE MABL = 'BL00004' WAITFOR DELAY '00:00:10' SELECT TONGTIENKHAM FROM BIENLAI WHERE MABL = 'BL00004'</pre>	Bước 4 <pre>EXEC PROC_UPDATE_BIENLAI 'BL00004', '20000';</pre>

COMMIT TRANSACTION	
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1 , 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:05' để mô phỏng trạng thái chờ thay đổi từ T2	
T2 thực hiện xong trước T1	
	
Kết quả ở T1:  Lần đọc đầu tiên, giá trị số lượng tồn ban đầu là 70. Thực hiện chờ 10s để đợi T2 thay đổi, vì mức độ cô lập SERIALIZABLE cho phép giữ nguyên khóa shared (S) đến khi giao dịch ở T1 hoàn tất (commit hoặc rollback). Nếu T2 muốn thay đổi thì phải chờ cho đến khi T1 hoàn tất và giải phóng khóa đọc. Do đó, T1 sẽ thấy cùng một giá trị của D cả hai lần đọc là 70. Non-repeatable read được giải quyết	

d. Trường hợp Phantom read

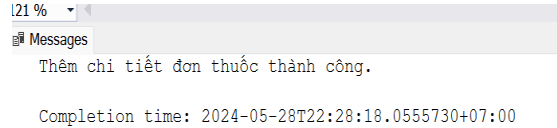
Tình huống 1

Mô tả tình huống: Khi bác sỹ A đang thực hiện kiểm tra toàn bộ chi tiết đơn thuốc tồn tại trong hệ thống, cùng lúc bác sỹ B đồng thời thực hiện thêm một chi tiết đơn thuốc mới vào hệ thống. Điều này dẫn đến việc bác sỹ A kiểm tra lại lần nữa thì thấy số lượng bản ghi của bảng CTDONTHUOC tăng thêm một dòng.

Nguyên nhân: Khi bác sỹ A thực hiện kiểm tra số lượng CTDONTHUOC lần đầu trong trạng thái giao dịch chưa hoàn tất, bác sỹ B đồng thời thêm một bản ghi mới vào CTDONTHUOC, lúc này bác sỹ A hoàn tất giao dịch bằng việc commit dữ liệu thì bản ghi đã được thêm vào hệ thống khiến kết quả trả về nhiều hơn một dòng so với dữ liệu cũ.

Ở mức DBMS: sử dụng procedure thêm chi tiết đơn thuốc để tái sử dụng mã lệnh và tăng hiệu suất chương trình.

Mô tả tình huống trên Hệ quản trị cơ sở dữ liệu SQL Server:

T1 (Bác sỹ A thực hiện kiểm tra tất cả bản ghi CTDONTHUOC tồn tại trong hệ thống)	T2 (Bác sỹ B thực hiện thêm loại thuốc Isulin, id = 7, số lượng là 40 vào bảng CTDONTHUOC làm cho số lượng bản ghi tăng thêm một dòng)
Bước 1 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Bước 2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
Bước 3 BEGIN TRANSACTION SELECT * FROM CTDONTHUOC WAITFOR DELAY '00:00:10' SELECT * FROM CTDONTHUOC COMMIT TRANSACTION	Bước 4 EXEC ThemCTDonThuoc '7', '1', '40';
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1 , 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:05' để mô phỏng trạng thái chờ thay đổi từ T2	
	T2 thực hiện xong trước T1 
Kết quả ở T1:	

121 %

Results Messages

	MATHUOC	MATOA	SOLUONG	GHICHU
1	2	2	10	Trưa 1 viên
2	3	2	10	Trưa 1 viên
3	4	1	10	Sáng 1 viên
4	5	1	10	Sáng 1 viên
5	6	1	10	Sáng 1 viên
6	8	3	10	Chiều 1 viên
7	9	3	10	Chiều 1 viên
8	12	4	10	Chiều 1 viên

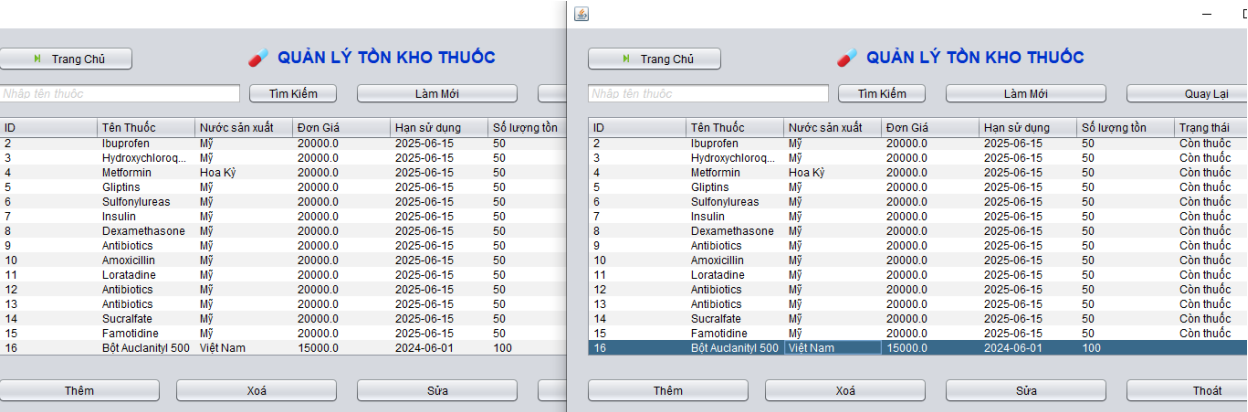
	MATHUOC	MATOA	SOLUONG	GHICHU
1	2	2	10	Trưa 1 viên
2	3	2	10	Trưa 1 viên
3	4	1	10	Sáng 1 viên
4	5	1	10	Sáng 1 viên
5	6	1	10	Sáng 1 viên
6	7	1	40	NULL
7	8	3	10	Chiều 1 vi...
8	9	3	10	Chiều 1 vi...

Lần đọc đầu tiên là toàn bộ bản ghi hiện có trong CTDONTHUOC.
5s để đợi T2 thay đổi
Lần đọc thứ hai số lượng bản ghi tăng thêm một dòng.
Phantom read xảy ra

Mô tả tình huống ở mức chương trình

Màn hình 1: Bác sỹ A thực hiện kiểm tra tất cả bản ghi THUOC tồn tại trong hệ thống

Chụp ảnh demo



Trang Chủ

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

THÊM MỚI THUỐC

ID

Tên thuốc

Nước sản xuất

Đơn giá

HSD

Số lượng

Màn hình 2: Bác sỹ B thực hiện thêm loại thuốc Bột Auclanityl có id = 16 với số lượng 100 viên vào bảng THUOC làm cho số lượng bản ghi tăng thêm một dòng.

Trang Chủ

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc: Tìm Kiếm

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
16	Bột Aucanilityl 500	Việt Nam	15000.0	2024-06-01	100	Còn thuốc

Chụp ảnh demo

Màn hình 1: Bác sỹ A thực hiện kiểm tra lại số lượng tồn của thuốc Isulin, có id = 7 trong bảng THUOC một lần nữa

Chụp ảnh demo

Trang Chủ

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc: Tìm Kiếm

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
16	Bột Aucanilityl 500	Việt Nam	15000.0	2024-06-01	100	Còn thuốc

Giải pháp cho trường hợp Phantom Read: Sử dụng mức cô lập SERIALIZABLE để ngăn chặn một transaction khác thực hiện chèn dữ liệu trong lúc một transaction đang thực thi.

Mở một cửa sổ truy vấn khác, thực hiện khôi phục dữ liệu bằng các lệnh sau:

```
SELECT * FROM CTDONTHUOC
```

```
SELECT * FROM THUOC
```

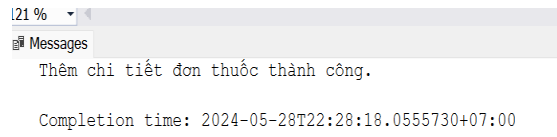
```
UPDATE THUOC SET SOLUONGTON = 50, tinhTrang = N'Còn thuốc'
```

```
WHERE id = 7;
```

```
DELETE FROM CTDONTHUOC WHERE MATHUOC = 7;
```

Sau đó thực hiện thay đổi mức cô lập thành SERIALIZABLE.

Mô tả tình huống trên Hệ quản trị cơ sở dữ liệu SQL Server:

T1 (Bác sỹ A thực hiện kiểm tra tất cả bản ghi CTDONTHUOC tồn tại trong hệ thống)	T2 (Bác sỹ B thực hiện thêm loại thuốc Insulin, id = 7, số lượng là 40 vào bảng CTDONTHUOC làm cho số lượng bản ghi tăng thêm một dòng)
Bước 1 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Bước 2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
Bước 3 BEGIN TRANSACTION SELECT * FROM CTDONTHUOC WAITFOR DELAY '00:00:10' SELECT * FROM CTDONTHUOC COMMIT TRANSACTION	Bước 4 EXEC ThemCTDonThuoc '7', '1', '40';
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1 , 2, 3, và 4. Câu lệnh WAITFOR DELAY '00:00:05' để mô phỏng trạng thái chờ thay đổi từ T2	
	T2 thực hiện xong trước T1 
Kết quả ở T1:	

	MATHUOC	MATOA	SOLUONG	GHICHU
1	2	2	10	Trưa 1 viên
2	3	2	10	Trưa 1 viên
3	4	1	10	Sáng 1 viên
4	5	1	10	Sáng 1 viên
5	6	1	10	Sáng 1 viên
6	8	3	10	Chiều 1 viên
7	9	3	10	Chiều 1 viên
8	12	4	10	Chiều 1 viên

Lần đọc đầu tiên là toàn bộ bản ghi hiện có trong CTDONTHUOC. T1 chờ 5s để đợi T2 thay đổi, ở mức cô lập SERIALIZABLE cho phép T1 đặt các khóa phạm vi (range locks) trên toàn bộ tập hợp dữ liệu CTDONTHUOC, T2 sẽ bị chặn khi cố gắng chèn hàng mới trong phạm vi đã khóa nên ở lần đọc thứ hai số lượng bản ghi được bảo tồn

Phantom read được giải quyết

Tình huống 2

Mô tả tình huống: Khi bác sĩ A đang thực hiện kiểm tra các loại thuốc tồn tại trong hệ thống, cùng lúc bác sĩ B đồng thời thực hiện thêm một loại thuốc mới vào hệ thống. Điều này dẫn đến việc bác sĩ A kiểm tra lại lần nữa thì thấy số lượng bản ghi của bảng THUOC tăng thêm một dòng.

Nguyên nhân: Khi bác sĩ A thực hiện kiểm tra số bản ghi THUOC lần đầu trong trạng thái giao dịch chưa hoàn tất, bác sĩ B đồng thời thêm một bản ghi mới vào THUOC, lúc này bác sĩ A hoàn tất giao dịch bằng việc commit dữ liệu thì bản ghi đã được thêm vào hệ thống khiến kết quả trả về nhiều hơn một dòng so với dữ liệu cũ.

Ở mức DBMS: sử dụng procedure thêm chi tiết đơn thuốc để tái sử dụng mã lệnh và tăng hiệu suất chương trình.

Mô tả tình huống trên Hệ quản trị cơ sở dữ liệu SQL Server:

T1 (Bác sĩ A thực hiện kiểm tra tất cả bản ghi THUOC tồn tại trong hệ thống)	T2 (Bác sĩ B thực hiện thêm loại thuốc Insulin 2 vào làm cho số lượng bản ghi tăng thêm một dòng)																																																																							
Bước 1 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Bước 2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;																																																																							
Bước 3 BEGIN TRANSACTION SELECT * FROM THUOC WAITFOR DELAY '00:00:05' SELECT * FROM THUOC COMMIT TRANSACTION	Bước 4 EXEC ThemThuoc N'Insulin 8', N'Mỹ', '200000', '2024-06-01', '50', N'Còn thuốc';																																																																							
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1, 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:05' để mô phỏng trạng thái chờ thay đổi từ T2																																																																								
Kết quả ở T1: 121 % Results Messages <table><tr><th>id</th><th>TENTHUOC</th><th>NUOCSX</th><th>DONGIA</th><th>HSD</th><th>SOLUONGTON</th><th>trinhTrang</th></tr><tr><td>9</td><td>9</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>10</td><td>10</td><td>Amoxicillin</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>11</td><td>11</td><td>Loratadine</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>12</td><td>12</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>13</td><td>13</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>14</td><td>14</td><td>Sucralfate</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>15</td><td>15</td><td>Famotidine</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>16</td><td>36</td><td>Insulin 7</td><td>Mỹ</td><td>200000.00</td><td>2024-06-01 00:00:00</td><td>50</td><td>Còn thuốc</td></tr></table>	id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang	9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	10	10	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	11	11	Loratadine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	12	12	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	13	13	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	14	14	Sucralfate	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	15	15	Famotidine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	16	36	Insulin 7	Mỹ	200000.00	2024-06-01 00:00:00	50	Còn thuốc	T2 thực hiện xong trước T1 121 % Messages Thêm thuốc thành công Completion time: 2024-06-01T13:34:39.9413022+07:00
id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang																																																																		
9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																	
10	10	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																	
11	11	Loratadine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																	
12	12	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																	
13	13	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																	
14	14	Sucralfate	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																	
15	15	Famotidine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																	
16	36	Insulin 7	Mỹ	200000.00	2024-06-01 00:00:00	50	Còn thuốc																																																																	
Lần đọc đầu tiên là toàn bộ bản ghi hiện có trong THUOC. 5s để đợi T2 thay đổi Lần đọc thứ hai số lượng bản ghi tăng thêm một dòng. Phantom read xảy ra																																																																								

Sử dụng mức cô lập SERIALIZABLE để giải quyết

T1 (Bác sĩ A thực hiện kiểm tra tất cả bản ghi THUOC tồn tại trong hệ thống)	T2 (Bác sĩ B thực hiện thêm loại thuốc Insulin 2 vào làm cho số lượng bản ghi tăng thêm một dòng)
Bước 1	Bước 2

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;	SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;																																																																								
Bước 3 BEGIN TRANSACTION SELECT * FROM THUOC WAITFOR DELAY '00:00:05' SELECT * FROM THUOC COMMIT TRANSACTION	Bước 4 EXEC ThemThuoc N'Insulin 8', N'Mỹ', '200000', '2024-06-01', '50', N'Còn thuốc';																																																																								
Thực hiện chạy lần lượt theo thứ tự liên tục từng bước 1, 2, 3, và 4 Câu lệnh WAITFOR DELAY '00:00:05' để mô phỏng trạng thái chờ thay đổi từ T2																																																																									
Kết quả ở T1: 121 % Results Messages <table><tr><th></th><th>id</th><th>TENTHUOC</th><th>NUOCSX</th><th>DONGIA</th><th>HSD</th><th>SOLUONGTON</th><th>trinhTrang</th></tr><tr><td>9</td><td>9</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>10</td><td>10</td><td>Amoxicillin</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>11</td><td>11</td><td>Loratadine</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>12</td><td>12</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>13</td><td>13</td><td>Antibiotics</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>14</td><td>14</td><td>Sucralfate</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>15</td><td>15</td><td>Famotidine</td><td>Mỹ</td><td>20000.00</td><td>2025-06-15 00:00:00</td><td>50</td><td>Còn thuốc</td></tr><tr><td>16</td><td>36</td><td>Insulin 7</td><td>Mỹ</td><td>200000.00</td><td>2024-06-01 00:00:00</td><td>50</td><td>Còn thuốc</td></tr></table>		id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang	9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	10	10	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	11	11	Loratadine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	12	12	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	13	13	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	14	14	Sucralfate	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	15	15	Famotidine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc	16	36	Insulin 7	Mỹ	200000.00	2024-06-01 00:00:00	50	Còn thuốc	T2 thực hiện xong trước T1 121 % Messages Thêm thuốc thành công Completion time: 2024-06-01T13:34:39.9413022+07:00
	id	TENTHUOC	NUOCSX	DONGIA	HSD	SOLUONGTON	trinhTrang																																																																		
9	9	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																		
10	10	Amoxicillin	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																		
11	11	Loratadine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																		
12	12	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																		
13	13	Antibiotics	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																		
14	14	Sucralfate	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																		
15	15	Famotidine	Mỹ	20000.00	2025-06-15 00:00:00	50	Còn thuốc																																																																		
16	36	Insulin 7	Mỹ	200000.00	2024-06-01 00:00:00	50	Còn thuốc																																																																		
Lần đọc đầu tiên là toàn bộ bản ghi hiện có trong THUOC. T1 chờ 5s để đợi T2 thay đổi, ở mức cô lập SERIALIZABLE cho phép T1 đặt các khóa phạm vi (range locks) trên toàn bộ tập hợp dữ liệu THUOC, T2 sẽ bị chặn khi cố gắng chèn hàng mới trong phạm vi đã khóa nên ở lần đọc thứ hai số lượng bản ghi được bảo tồn. Sau khi T1 thực hiện đọc xong thì T2 mới thêm thành công bản ghi mới. Phantom read được giải quyết.																																																																									

e. Deadlock

Tình huống 1

Mô tả tình huống: Hai giao dịch T1, T2 được thực hiện đồng thời như sau:

- + T1: Bắt đầu bằng cách khóa bảng TaiKhoan để cập nhật thông tin của một tài khoản. Sau đó cố gắng khóa bảng NhanVien để cập nhật thông tin nhân viên liên quan đến tài khoản đó.
- + Bắt đầu bằng cách khóa bảng NhanVien để cập nhật thông tin của một nhân viên.

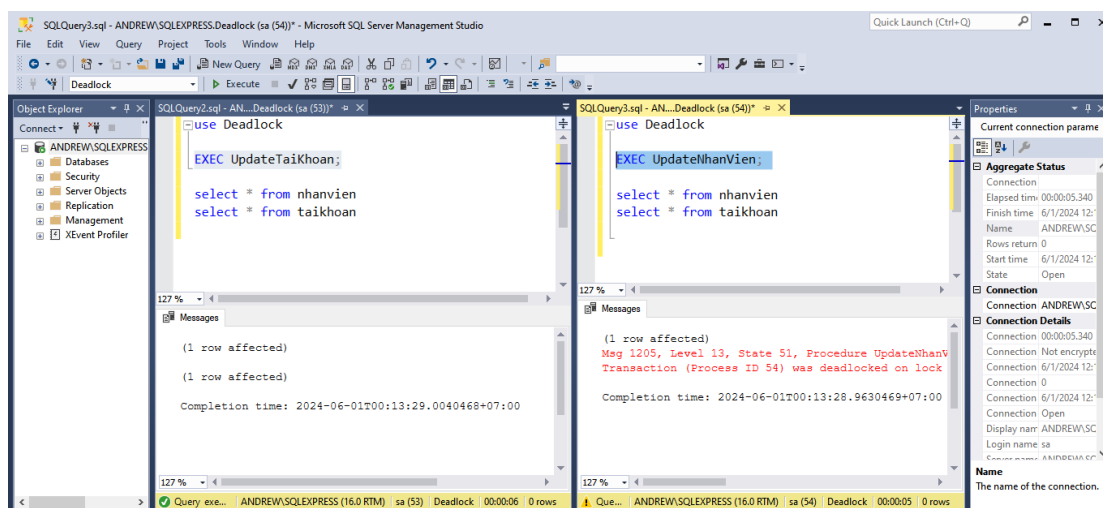
Sau đó cố gắng khóa bảng TaiKhoan để cập nhật thông tin tài khoản liên quan đến nhân viên đó.

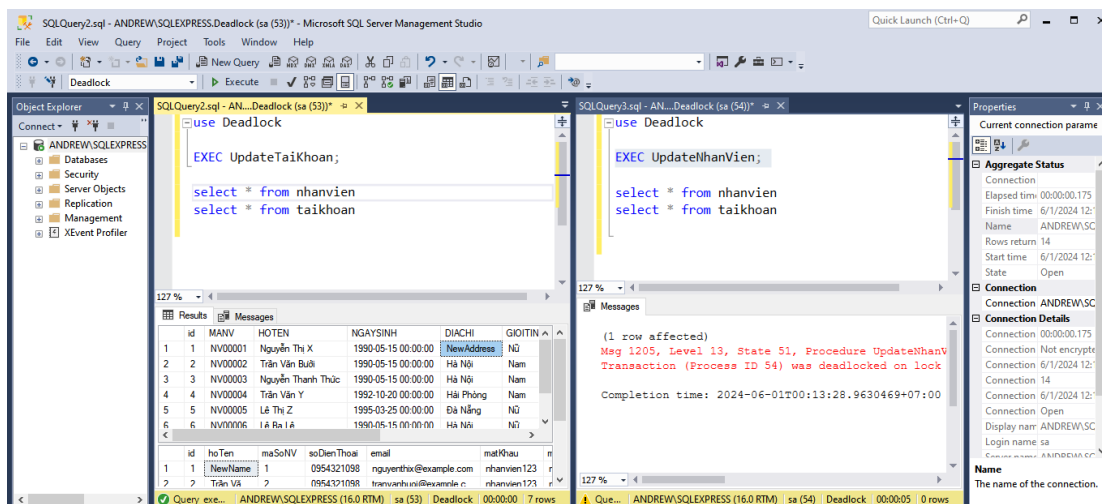
Cả hai giao dịch đều bắt đầu bằng cách khóa một bảng và sau đó cố gắng khóa bảng kia, nhưng bảng thứ hai đã bị khóa bởi giao dịch kia. Điều này dẫn đến deadlock.

Mô tả tình huống ở mức Hệ quản trị SQL Server

<pre>CREATE PROCEDURE UpdateTaiKhoan AS BEGIN BEGIN TRANSACTION; -- Khóa bảng TaiKhoan UPDATE TaiKhoan SET hoTen = 'NewName' WHERE id = 1; -- Chờ khóa bảng NhanVien WAITFOR DELAY '00:00:05'; -- Giả lập thời gian chờ UPDATE NhanVien SET diaChi = 'NewAddress' WHERE id = 1; COMMIT; END;</pre>	<pre>CREATE PROCEDURE UpdateNhanVien AS BEGIN BEGIN TRANSACTION; -- Khóa bảng NhanVien UPDATE NhanVien SET diaChi = 'NewAddress1' WHERE id = 1; -- Chờ khóa bảng TaiKhoan WAITFOR DELAY '00:00:05'; -- Giả lập thời gian chờ UPDATE TaiKhoan SET hoTen = 'NewName1' WHERE id = 1; COMMIT; END;</pre>
Bước 1	EXEC UpdateTaiKhoan;
Bước 2	EXEC UpdateNhanVien;
Bước 3	<pre>select * from nhanvien select * from taikhoan</pre>

Chạy đồng thời 2 transaction thì deadlock xảy ra => hệ thống sẽ chọn 1 transaction để làm nạn nhân cho deadlock, buộc transaction đó phải dừng thực thi và giải phóng khóa cho các transaction khác.





Cách giải quyết.

Màn hình transaction 1 thực hiện thao tác trên TAIKHOAN. Sau đó chờ 5s thì thao tác trên bảng NHANVIEN

Danh Sách Đăng Ký							
Menu							
Danh sách tài khoản							
	Tìm kiếm	Làm mới	Trang Chủ				
Họ tên	Mã nhân viên	Sdt	Email	Mật khẩu	Mật khẩu xác nhận	Vai trò	Trạng thái
Nguyễn Thị X	NV00001	0954321098	nguyenthix@exa...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Trần Văn Bưởi	NV00002	0954321098	tranvanbui@exa...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Nguyễn Thanh Thức	NV00003	0954321099	nguyenthathuc...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Trần Văn Y	NV00004	0954321099	tranvany@examp...	nhanvien123	nhanvien123	NV Tiếp Nhận	Hiệu lực
Le Thi Z	NV00005	0954321010	lethiz@example.com	nhanvien123	nhanvien123	NV Thanh Toán	Hiệu lực
Le Ba Lê	NV00006	0954321098	lebelat@example...	nhanvien123	nhanvien123	Admin	Hiệu lực
Trần Bảo Ngọc	NV00007	0954321098	tranbaongoc@exa...	nhanvien123	nhanvien123	Quản lý kho	Hiệu lực
Thêm		Xóa		Sửa		Thoát	

Màn hình transaction 2 thực hiện thao tác trên NHANVIEN. Sau đó chờ 5s thì thao tác trên bảng TAIKHOAN

Quản lý thông tin nhân viên

Menu

 **TRANG QUẢN LÝ THÔNG TIN NHÂN VIÊN**

ID	Mã nhân viên	Họ tên	Ngày sinh	Địa chỉ	Giới tính	Ngày vào làm	Vai trò	Trạng thái
1	NV00001	Nguyễn Thị X	1990-05-15	Hà Nội	Nữ	2010-01-01	Bác sĩ	Đang tồn tại
2	NV00002	Trần Văn Bưởi	1990-05-15	Hà Nội	Nam	2010-01-15	Bác sĩ	Đang tồn tại
3	NV00003	Nguyễn Thanh...	1990-05-15	Hà Nội	Nam	2011-01-01	Bác sĩ	Đang tồn tại
4	NV00004	Trần Văn Y	1992-10-20	Hải Phòng	Nam	2012-03-15	NV Tiếp nhận	Đang tồn tại
5	NV00005	Lê Thị Z	1995-03-25	Đà Nẵng	Nữ	2015-07-20	NV Thanh toán	Đang tồn tại
6	NV00006	Lê Ba Lê	1990-05-15	Hà Nội	Nữ	2010-01-01	Admin	Đang tồn tại
7	NV00007	Trần Bảo Ngọc	1990-05-15	Hà Nội	Nam	2010-01-15	Quản lý kho	Đang tồn tại

Danh Sách Đăng Ký

Menu

Danh sách tài khoản

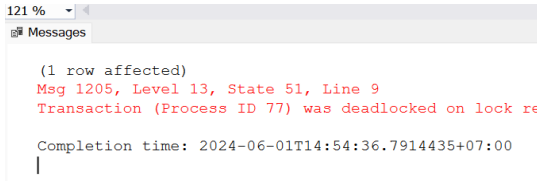
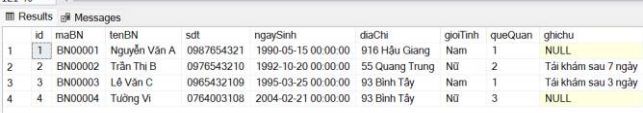
Họ tên	Mã nhân viên	Sdt	Email	Mật khẩu	Mật khẩu xác nhận	Vai trò	Trạng thái
Nguyễn Thị X	NV00001	0954321098	nguyenthix@exa...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Trần Văn Bưởi	NV00002	0954321098	tranvanbui@exa...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Nguyễn Thanh Thức	NV00003	0954321099	nguyenthanhthuc...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Trần Văn Y	NV00004	0954321099	tranvany@exampl...	nhanvien123	nhanvien123	NV Tiếp Nhận	Hiệu lực
Le Thị Z	NV00005	0954321010	lethiz@example.com	nhanvien123	nhanvien123	NV Thanh Toán	Hiệu lực
Le Ba Lê	NV00006	0954321098	lebel@exaample.com	nhanvien123	nhanvien123	Admin	Hiệu lực
Trần Bảo Ngọc	NV00007	0954321098	tranbaongoc@exa...	nhanvien123	nhanvien123	Quản lý kho	Hiệu lực

Tình huống 2

Mô tả tình huống: Hai giao dịch T1, T2 đang cố gắng đồng thời cập nhật dữ liệu trên cùng một bảng BenhNhan. Mỗi giao dịch đều đang thực hiện hai bước cập nhật khác nhau trên các bản ghi khác nhau của bảng này, dẫn đến việc T1 cần một khóa mà T2 đang giữ và ngược lại, tạo ra một vòng lặp chờ đợi không thể giải quyết được gọi là deadlock. Cả hai giao dịch sẽ bị treo vô thời hạn cho đến khi hệ thống phát hiện và giải quyết deadlock.

Mô tả tình huống trên Hệ quản trị cơ sở dữ liệu SQL Server:

T1	T2
SET TRANSACTION ISOLATION LEVEL READ COMMITTED	SET TRANSACTION ISOLATION LEVEL READ COMMITTED
BEGIN TRANSACTION	BEGIN TRANSACTION
UPDATE BenhNhan SET ghichu = N'Đang khám' WHERE id = 2	UPDATE BenhNhan SET ghichu = N'Tái khám sau 3 ngày' WHERE id = 3

WAITFOR DELAY '00:00:05' UPDATE BenhNhan SET ghichu = N'Khám xong' WHERE id = 3 COMMIT TRANSACTION	WAITFOR DELAY '00:00:05' UPDATE BenhNhan SET ghichu = N'Tái khám sau 7 ngày' WHERE id = 2 COMMIT TRANSACTION
Kết quả ở T1: 	T2 thực hiện xong trước T1 
Nguyên nhân: T1 đang giữ khóa trên id = 2 và chờ khóa trên id = 3. T2 đang giữ khóa trên id = 3 và chờ khóa trên id = 2. Điều này dẫn đến deadlock giữa hai giao dịch. Hệ thống DBMS có cơ chế tự động phát hiện deadlock và chọn một trong hai transaction làm nạn nhân để giải phóng khóa. Ở đây hệ thống chọn T1 làm nạn nhân, dừng T1 giải phóng khóa cho T2 thực thi thành công cập nhật dữ liệu trong bảng.	

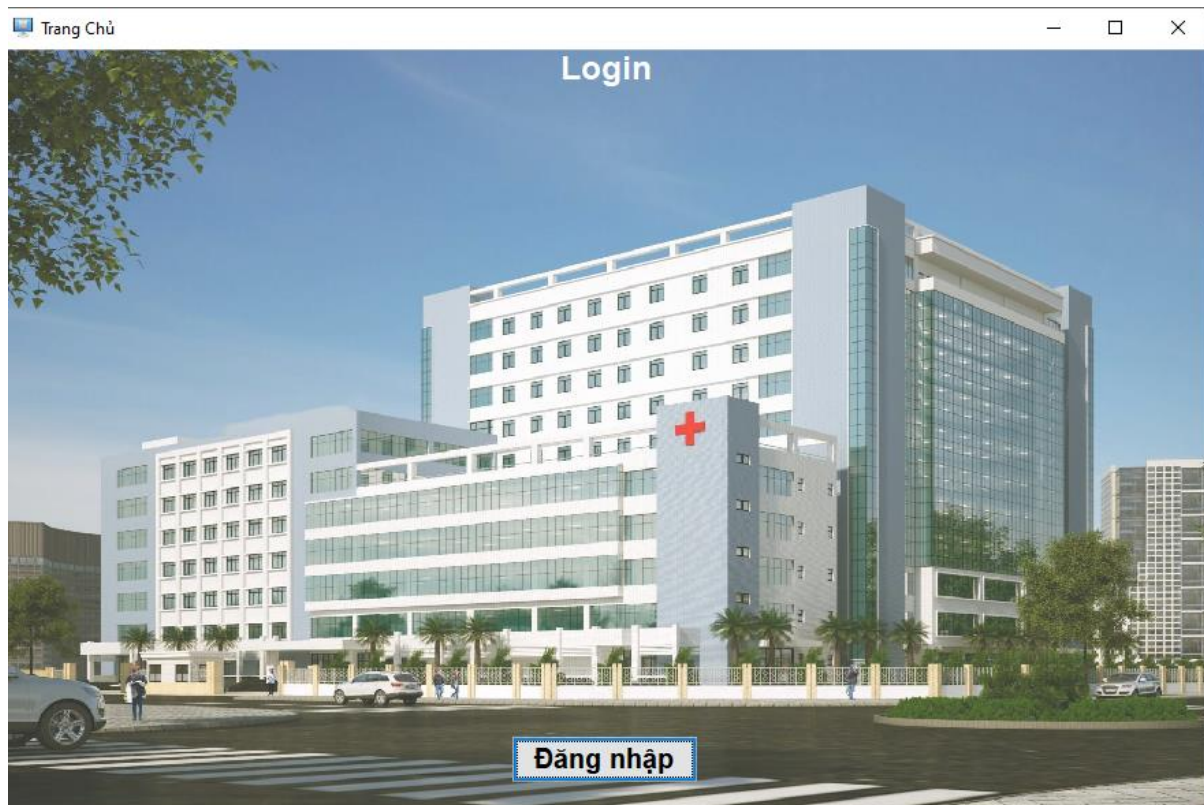
PHẦN IV. THIẾT KẾ GIAO DIỆN

1. Thiết kế giao diện

1.1. Danh sách các giao diện

STT	Giao Diện	Chức Năng
1	Trang chủ đăng nhập	Hiện thị trang chủ và chức năng đăng nhập
2	Đăng nhập	Đăng nhập hệ thống
3	Đăng ký	Đăng ký tài khoản, Sửa thông tin tài khoản
4	Quản lý thông tin khám bệnh	Hiện thị thông tin của các phiếu khám bệnh
5	Kê toa thuốc	Kê toa thuốc
6	Thêm chi tiết thuốc	Chọn thuốc để kê vào toa
7	Phiếu khám bệnh	Hiện thị thông tin chi tiết của một phiếu khám bệnh
8	Quản lý thông tin biên lai	Hiện thị thông tin các biên lai
9	Thêm biên lai	Thêm một biên lai mới

10	Thông tin chi tiết biên lai	Hiển thị thông tin chi tiết biên lai của một biên lai
11	Trang chủ admin	Trang chủ chứa các chức năng của admin
12	Trang quản lý thông tin nhân viên	Hiển thị thông tin tất cả các thông tin của các nhân viên
13	Thông tin nhân viên	Thêm, sửa nhân viên
14	Danh sách tài khoản	Hiển thị tất cả thông tin của các tài khoản
15	Thông tin tài khoản	Thêm, sửa tài khoản
16	Thống kê quē quán của bệnh nhân	Thống kê số lượng quē quán có bệnh nhân đến bệnh viện
17	Thống kê doanh thu theo năm	Thống kê doanh thu qua các năm
18	Thống kê doanh thu theo tháng của năm	Thống kê doanh thu qu từng tháng của năm
19	Quản lý tồn kho thuốc	Hiển thị tất cả thông tin của thuốc
20	Thêm thông tin thuốc	Thêm thông tin thuốc
21	Sửa thông tin thuốc	Sửa thông tin thuốc
22	Trang chủ nhân viên tiếp nhận	Trang chủ chứa các chức năng của nhân viên tiếp nhận
23	Trang quản lý thông tin bệnh nhân	Hiển thị tất cả các thông tin của bệnh nhân
24	Thông tin bệnh nhân	Thêm, sửa thông tin bệnh nhân
25	Thêm mới phiếu khám bệnh	Thêm một phiếu khám bệnh mới.
26	Xuất phiếu khám bệnh	Phiếu khám bệnh thể hiện phòng khám và stt cho bệnh nhân cần khám
27	Xuất chi tiết phiếu khám bệnh	Chi tiết PKB gồm thông tin bệnh nhân, bác sĩ , chẩn đoán , toa thuốc
28	Xuất biên lai	Biên lai PDF



Hình 1 Màn hình giao diện trang chủ tổng quan của nhân viên

1.2. Màn hình giao diện Trang chủ

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Đăng nhập	JBUTTON	Thực hiện thao tác chuyển đến giao diện đăng nhập

Hình 2 Màn hình đăng nhập của nhân viên

1.3 Màn hình giao diện Đăng nhập

Mô tả giao diện.

STT	Tên	Kiểu	Chức năng
1	Tài khoản	JTextField	Nhập email
2	Mật khẩu	JTextField	Nhập mật khẩu
3	Bác sĩ	JRadioButton	Đăng nhập với vai trò là bác sĩ
4	NV Tiếp Nhận	JRadioButton	Đăng nhập với vai trò là nhân viên tiếp nhận
5	Quản lý kho	JRadioButton	Đăng nhập với vai trò là quản lý
6	NV Thanh Toán	JRadioButton	Đăng nhập với vai trò là Admin
7	Admin	JRadioButton	Đăng nhập với vai trò là nhân viên thanh toán

8	Đăng ký tài khoản	JRadioButton	Chọn chức năng đăng ký
9	Đăng nhập	JButton	Thực hiện thao tác đăng nhập
10	Hủy	JButton	Thực hiện thao tác hủy đăng nhập
11	Đăng ký	JButton	Thực hiện thao tác đăng ký

1.4 . Màn hình giao diện đăng ký.

Đăng Ký

Họ và tên: Phan Thị Tường Vi

Mã số: NV00009

Điện thoại: 0764003108

Email: phantuongvi2102@gmail.com

Mật khẩu: Vihaha

Gõ lại mật khẩu: Vihaha

☐ NV Tiếp Nhận
 ☒ NV Thanh Toán
 ☐ Bác Sĩ
 ☐ Admin
 ☐ Quản Lý Kho

Đăng Ký
 Lưu
 Hủy

Hình 3 Đăng ký

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Họ và tên	JTextField	Nhập họ tên tài khoản
2	Mã số	JTextField	Nhập mã số nhân viên
3	Điện thoại	JTextField	Nhập điện thoại
4	Email	JTextField	Nhập email đăng ký
5	Mật khẩu	JTextField	Nhập mật khẩu
6	Gõ lại mật khẩu	JTextField	Nhập lại mật khẩu

7	NV Tiếp Nhận	JRadioButton	Nhấn để chọn vai trò là nhân viên tiếp nhận
8	NV Thanh Toán	JRadioButton	Nhấn để chọn vai trò là nhân viên thanh toán
9	Bác sĩ	JRadioButton	Nhấn để chọn vai trò là bác sĩ
10	Admin	JRadioButton	Nhấn để chọn vai trò là Admin
11	Quản lý kho	JButton	Nhấn để chọn vai trò là quản lý kho
12	Đăng ký	JButton	Nhấn để đăng ký
13	Lưu	JButton	Nhấn để lưu thông tin tài khoản sau khi sửa
14	Hủy	JButton	Nhấn để hủy đăng ký tài khoản

1.5. Màn hình giao diện tương tác với bác sĩ.

QUẢN LÝ THÔNG TIN KHÁM BỆNH

Nhập tên bệnh nhân

ID	MaPKB	MaBS	maBN	maPK	Ngày Tạo	Chan Doan
1	PKB00001	NV00001	BN00001	PK00001	2024-05-13	Tiểu đường
2	PKB00002	NV00001	BN00002	PK00006	2024-05-13	Thấp khớp
3	PKB00003	NV00002	BN00003	PK00003	2024-05-13	Viêm mắt
4	PKB00004	NV00003	BN00004	PK00005	2024-05-13	Dạ dày

Hình 4 Quản lý thông tin khám bệnh

Mô tả giao diện

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Trang chủ	JButton	Nhấn để quay về trang chủ đăng nhập của hệ thống
2	Tên bệnh nhân	TextField	Nhập tên bệnh nhân để tìm kiếm
3	Tìm kiếm	JButton	Nhấn để tìm kiếm phiếu khám bệnh của bệnh nhân
4	Làm mới	JButton	Hiện thị lại tất cả thông tin phiếu khám bệnh
5	Danh sách các phiếu khám bệnh	JTable	Hiện thị tất cả thông tin phiếu khám bệnh
6	Thêm	JButton	Tạo phiếu khám bệnh cho bệnh nhân
7	Xóa	JButton	Xóa phiếu khám bệnh
8	Sửa	JButton	Sửa thông tin phiếu khám bệnh cho bệnh nhân
9	Thoát	JButton	Thoát hệ thống

Kê Toa Thuốc

Mã Khám Bệnh

PKB00004

Mã Bác Sĩ

▼

1

Mã Bệnh Nhân

▼

4

Tên phòng khám

▼

Phòng tiêu hóa

Ngày tạo

May 13, 2024

▲▼

Chẩn đoán

Dạ dày

▲▼

Thêm chẩn đoán

PKB ID

Tạo toa thuốc

Mã Toa

Thêm Toa Thuốc

Title 1	Title 2	Title 3	Title 4

Tạo CTToaThuoc

Tổng hoá đơn

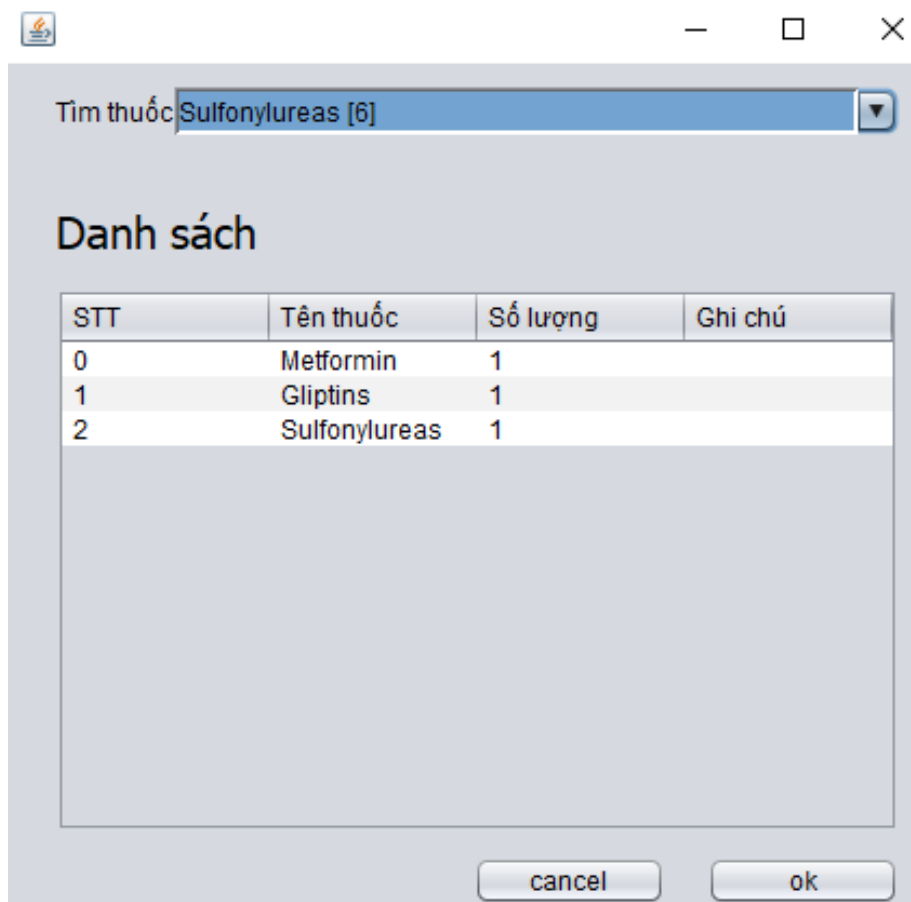
Quay Lại

Hình 5 Kê toa thuốc

Mô tả giao diện.

STT	Tên	Kiểu	Chức năng
-----	-----	------	-----------

1	Mã khám bệnh	TextField	Bị khóa
2	Mã bác sĩ	ComboBox	Chọn mã bác sĩ đang khám
3	Mã bệnh nhân	ComboBox	Chọn mã bệnh nhân đang khám
4	Mã phòng khám	ComboBox	Chọn mã phòng khám đang khám
5	Ngày tạo	DateChooser	Chọn ngày tạo phiếu khám bệnh
6	Chẩn đoán	TextField	Ghi chẩn đoán bệnh của bệnh nhân
7	Thêm PKB	Button	Thêm phiếu khám bệnh
8	Tạo toa thuốc	Button	Nhấn để tạo toa thuốc
8	Mã toa	TextField	Hiện thị mã toa mới nhất



Hình 6 Thêm chi tiết đơn thuốc

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Tìm thuốc	JComboBox	Chọn thuốc để kê đơn
2	Danh sách các thuốc được kê	JTable	Hiển thị thông tin trong chi tiết đơn thuốc
3	Cancel	JButton	Hủy tạo đơn thuốc
	OK	JButton	Thêm thuốc vào chi tiết đơn thuốc

Phiếu Khám Bệnh

Bác Sĩ Khám

Nguyễn Thị X

Ngày Khám

2024-05-13

Tên Bệnh Nhân

Nguyễn Văn A

Yêu cầu khám

Phòng điều trị tiểu đường

Chẩn đoán

Tiểu đường

Toa Thuốc

Mã Toa	Mã Thuốc	Tên Thuốc	Số Lượng	Đơn Giá
TOA00001	4	Metformin	10	20000.0
TOA00001	5	Gliptins	10	20000.0
TOA00001	6	Sulfonylureas	10	20000.0

Hình 7 Phiếu khám bệnh

Mô tả giao diện.

STT	Tên	Kiểu	Chức năng
1	Bác sĩ khám	JTextField	Hiển thị tên bác sĩ khám
2	Tên bệnh nhân	JTextField	Hiển thị tên bệnh nhân
3	Chẩn đoán	JTextField	Hiển thị chẩn đoán bệnh
4	Ngày khám	JTextField	Hiển thị ngày khám
5	Yêu cầu khám	JTextField	Hiển thị tên phòng khám
6	Toa thuốc	JTable	Hiển thị thông tin các thuốc đã được kê đơn

1.6. Màn hình giao diện tương tác của nhân viên thanh toán.



Hình 8 Trang quản lý thông tin biên lai

STT	Tên	Kiểu	Chức năng
1	Menu	JMenuBar	Chức các chức năng con
2	Open File	JMenuItem	Mở file đã lưu từ máy tính
3	Save File	JMenuItem	Save file từ hệ thống vào máy tính
4	Xuất biên lai	JMenuItem/JPopupMenu	Xuất PDF thông tin biên lai đã chọn

5	Mã biên lai	TextField	Nhập mã biên lai để tìm kiếm
6	Tìm kiếm theo mã biên lai	Button	Nhấn để tìm kiếm thông tin biên lai của bệnh nhân
7	Hiển thị tất cả	Button	Hiển thị lại tất cả thông tin phiếu khám bệnh
9	Danh sách thông tin các biên lai	Table	Hiển thị tất cả thông tin biên lai của các bệnh nhân
10	Thêm	Button	Tạo biên lai thanh toán cho bệnh nhân
11	Xóa	Button	Xóa biên lai thanh toán
13	Thoát	Button	Thoát hệ thống
14	Trang chủ	Button	Quay về trang chủ
15	Hiển thị chi tiết	PopupMenu	Hiển thị GUI chi tiết biên lai của bệnh nhân

 Thêm biên lai
 — □ ✕



THÔNG TIN BIÊN LAI

Mã toa:

TOA00001

Tổng tiền(đồng):

900000.0

Thêm

Hình 9 . Thêm biên lai

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
-----	-----	------	-----------

1	Mã toa	TextField	Nhập mã toa cần thanh toán
2	Tổng tiền	TextField	Hiện thị tổng tiền khám của bệnh nhân
3	Thêm	Button	Thêm biên lai

 Chi tiết biên lai

THÔNG TIN BIÊN LAI

ID: 1

Mã biên lai: BL00001

Mã NVTT: NV00005

Mã toa: TOA00001

Ngày tạo: 2024-05-26

Tổng tiền khám: 900000.0

Hình 10 Thông tin chi tiết biên lai

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	ID	TextField	Số thứ tự của biên lai
2	Mã biên lai	TextField	Hiện thị mã biên lai

3	Mã NVTT	TextField	Hiện thị mã của nhân viên thanh toán biên lai đó
4	Mã toa	TextField	Hiện thị mã toa thuốc của biên lai
5	Ngày tạo	TextField	Ngày tạo biên lai
6	Tổng tiền khám	TextField	Hiện thị tổng tiền khám bệnh nhân cần thanh toán

1.7. Màn hình giao diện tương tác với admin.



Hình 11 Trang chủ admin

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Danh sách tài khoản	JButton	Hiện thị danh sách các tài khoản của nhân viên
2	Danh sách tài khoản	JButton	Hiện thị danh sách các thông tin của nhân viên
3	Thống kê quỹ quán	JButton	Hiện thị thống kê quỹ quán

4	Thống kê doanh thu	JButton	Hiển thị thống kê doanh thu

 Quản lý thông tin nhân viên

Menu


TRANG QUẢN LÝ THÔNG TIN NHÂN VIÊN

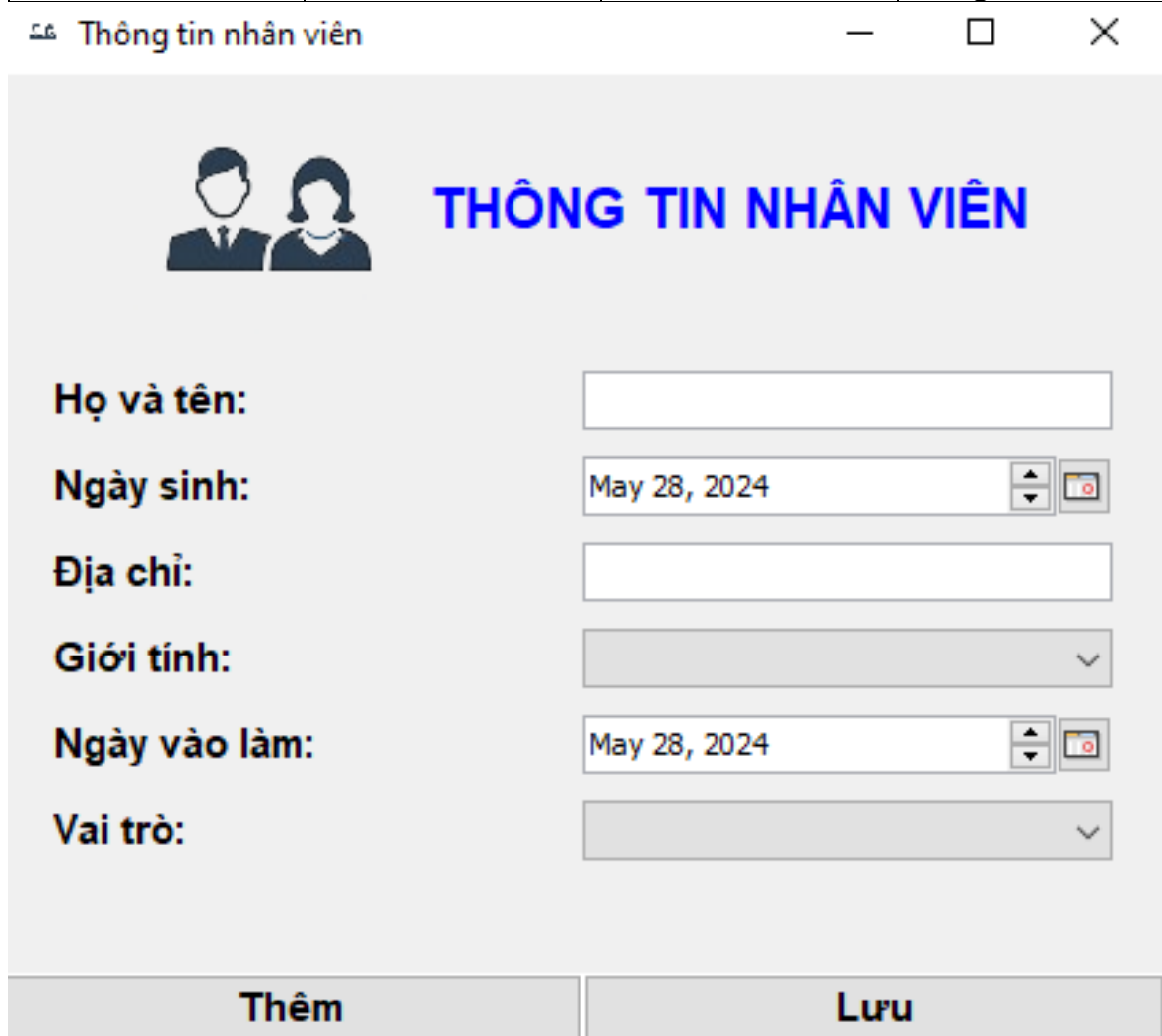
ID	Mã nhân viên	Họ tên	Ngày sinh	Địa chỉ	Giới tính	Ngày vào làm	Vai trò	Trạng thái
1	NV00001	Nguyễn Thị X	1990-05-15	Hà Nội	Nữ	2010-01-01	Bác sĩ	Đang tồn tại
2	NV00002	Trần Văn Bưởi	1990-05-15	Hà Nội	Nam	2010-01-15	Bác sĩ	Đang tồn tại
3	NV00003	Nguyễn Than...	1990-05-15	Hà Nội	Nam	2011-01-01	Bác sĩ	Đang tồn tại
4	NV00004	Trần Văn Y	1992-10-20	Hải Phòng	Nam	2012-03-15	NV Tiếp nhận	Đang tồn tại
5	NV00005	Lê Thị Z	1995-03-25	Đà Nẵng	Nữ	2015-07-20	NV Thanh toán	Đang tồn tại
6	NV00006	Lê Ba Lê	1990-05-15	Hà Nội	Nữ	2010-01-01	Admin	Đang tồn tại
7	NV00007	Trần Bảo Ngọc	1990-05-15	Hà Nội	Nam	2010-01-15	Quản lý kho	Đang tồn tại
8	NV00008	Trần Tinh Tú	1990-05-15	Hà Nội	Nam	2010-01-01	Bác sĩ	Đang tồn tại
9	NV00009	Phan Thị Trữ...	2024-05-28	916 Hải Giang	Nữ	2024-05-28	NV Thanh toán	Đang tồn tại

Hình 12 Trang quản lý thông tin nhân viên

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Mã nhân viên	JTextField	Nhập mã nhân viên
2	Tìm kiếm theo mã nhân viên	JButton	Tìm kiếm thông tin nhân viên
3	Hiển thị tất cả	JButton	Hiển thị lại tất cả dữ liệu sau tìm kiếm
4	Trang chủ	JButton	Quay lại trang chủ
5	Danh sách thông tin nhân viên	JTable	Hiển thị thông tin tất cả nhân viên
6	Thêm	JButton	Nhấn để thêm thông tin nhân viên mới
7	Xóa	JButton	Nhấn để xóa thông tin nhân viên mới

8	Sửa	JButton	Nhấn để sửa thông tin nhân viên mới
9	Thoát	JButton	Nhấn để thoát hệ thống



Thông tin nhân viên

 **THÔNG TIN NHÂN VIÊN**

Họ và tên:

Ngày sinh: 

Địa chỉ:

Giới tính:

Ngày vào làm: 

Vai trò:

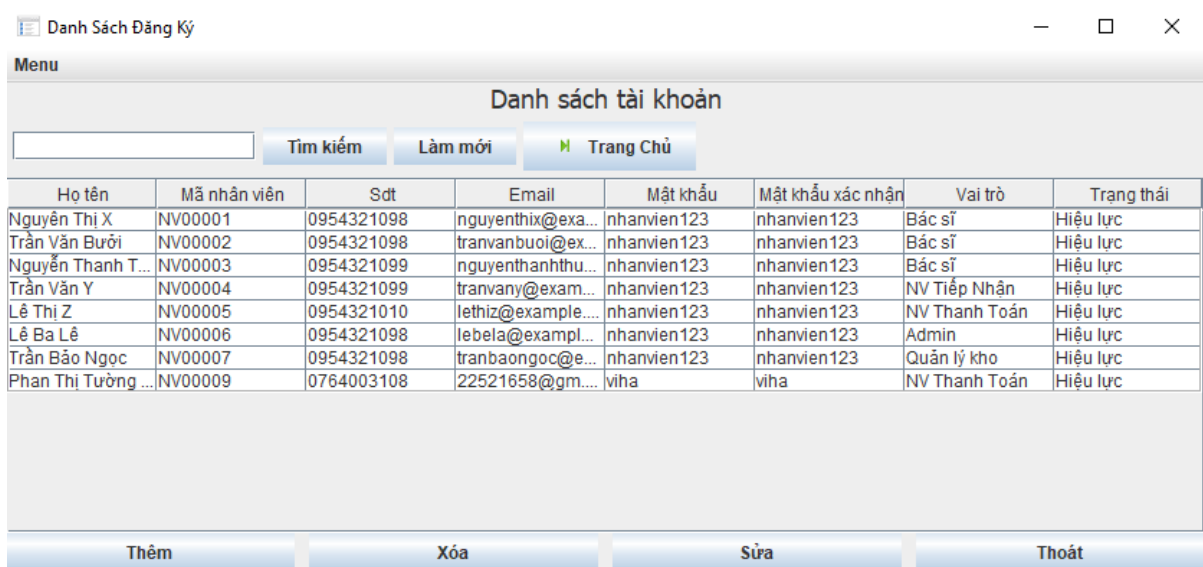
Thêm **Lưu**

Hình 13 Thông tin nhân viên

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Họ và tên	JTextField	Nhập họ tên nhân viên
2	Ngày sinh	JDateChooser	Chọn ngày sinh của nhân viên
3	Địa chỉ	JTextField	Nhập địa chỉ nhân viên

4	Giới tính	JComboBox	Chọn giới tính
5	Danh sách thông tin nhân viên	JTable	Hiển thị thông tin tất cả nhân viên
6	Ngày vào làm	JDateChooser	Chọn ngày vào làm của nhân viên
7	Vai trò	JCombobox	Chọn vai trò của nhân viên



Họ tên	Mã nhân viên	Sdt	Email	Mật khẩu	Mật khẩu xác nhận	Vai trò	Trạng thái
Nguyễn Thị X	NV00001	0954321098	nguyenthix@exa...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Trần Văn Bưởi	NV00002	0954321098	tranvanbui@ex...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Nguyễn Thanh T...	NV00003	0954321099	nguyenthanhthu...	nhanvien123	nhanvien123	Bác sĩ	Hiệu lực
Trần Văn Y	NV00004	0954321099	tranvany@exam...	nhanvien123	nhanvien123	NV Tiếp Nhận	Hiệu lực
Lê Thị Z	NV00005	0954321010	lethiz@example...	nhanvien123	nhanvien123	NV Thanh Toán	Hiệu lực
Lê Ba Lê	NV00006	0954321098	lebel@exampl...	nhanvien123	nhanvien123	Admin	Hiệu lực
Trần Bảo Ngọc	NV00007	0954321098	tranbaongoc@e...	nhanvien123	nhanvien123	Quản lý kho	Hiệu lực
Phan Thị Tường ...	NV00009	0764003108	22521658@gm...	viha	viha	NV Thanh Toán	Hiệu lực

Hình 14 Danh sách tài khoản

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Số điện thoại	JTextField	Nhập số điện thoại của nhân viên cần tìm
2	Tìm kiếm	JButton	Tìm kiếm thông tin nhân viên
3	Làm mới	JButton	Hiển thị lại tất cả dữ liệu sau tìm kiếm
4	Trang chủ	JButton	Quay lại trang chủ
5	Danh sách thông tin nhân viên	JTable	Hiển thị thông tin tất cả tài khoản

6	Thêm	JButton	Nhấn để thêm thông tin tài khoản mới
7	Xóa	JButton	Nhấn để xóa thông tin tài khoản
8	Sửa	JButton	Nhấn để sửa thông tin tài khoản
9	Thoát	JButton	Nhấn để thoát hệ thống

Đăng Ký

Họ và tên: Trần Bảo Ngọc

Mã số: 7

Điện thoại: 0954321098

Email: tranbaongoc@example.com

Mật khẩu: nhanvien123

Gõ lại mật khẩu: nhanvien123

☐ NV Tiếp Nhận
 ☐ NV Thanh Toán
 ☐ Bác Sĩ
 ☒ Quản Lý Kho
 ☐ Admin

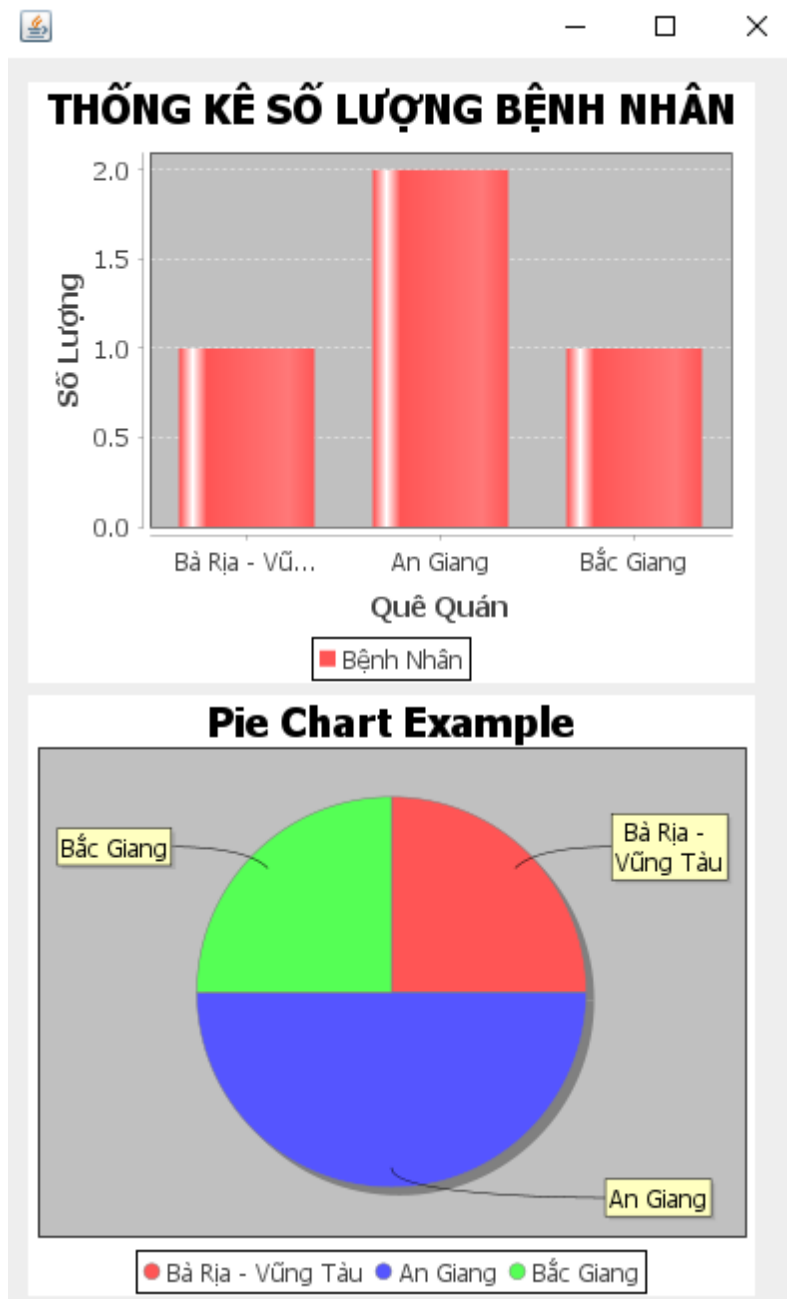
Đăng Ký
 Lưu
 Hủy

Hình 15 Thông tin tài khoản

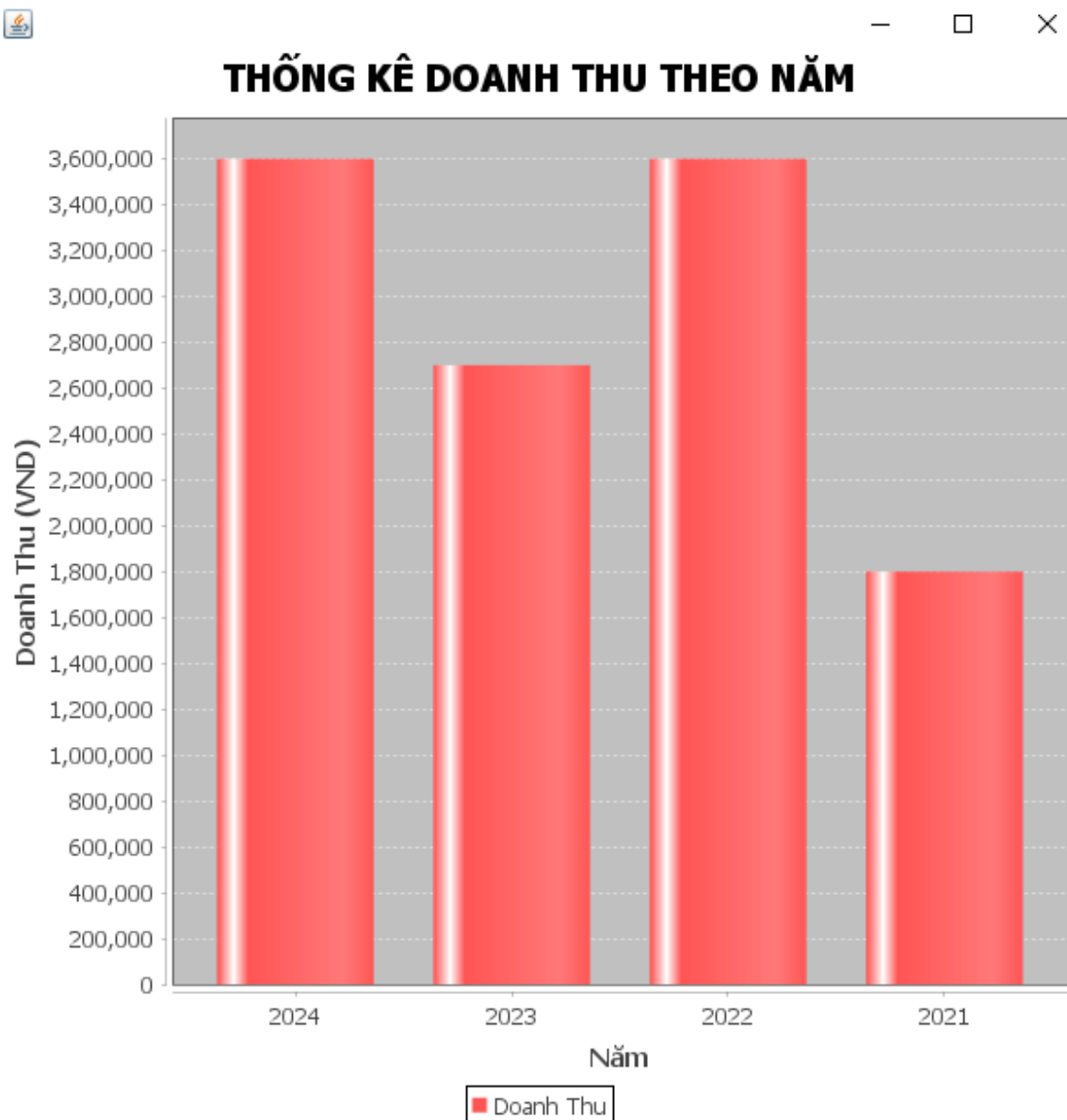
Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Họ và tên	JTextField	Hiện thị họ tên tài khoản
2	Mã số	JTextField	Hiện thị mã số nhân viên
3	Điện thoại	JTextField	Hiện thị điện thoại
4	Email	JTextField	Hiện thị email đăng ký
5	Mật khẩu	JTextField	Hiện thị mật khẩu
6	Gõ lại mật khẩu	JTextField	Hiện thị mật khẩu xác nhận

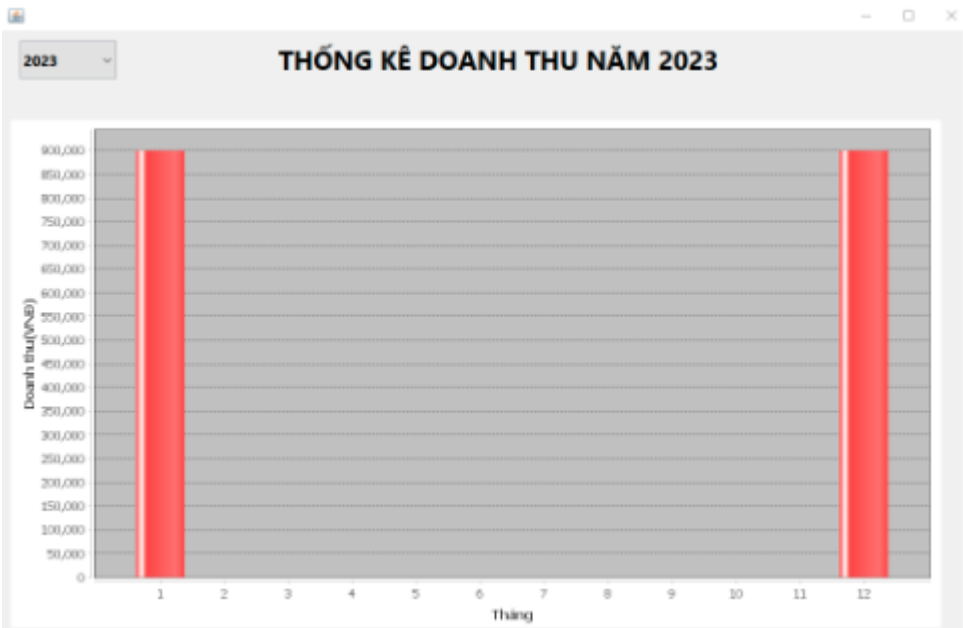
7	Các JButton	JRadioButton	Hiển thị vai trò đã chọn
---	-------------	--------------	--------------------------



Hình 16 Thống kê quê quán của bệnh nhân



Hình 17 Thống kê doanh thu qua các năm



Hình 18 Thống kê doanh thu theo tháng của năm

1.8. Màn hình giao diện tương tác với quản lý kho

QUẢN LÝ TỒN KHO THUỐC

Nhập tên thuốc: Tìm Kiếm

ID	Tên Thuốc	Nước sản xuất	Đơn Giá	Hạn sử dụng	Số lượng tồn	Trạng thái
1	Paracetamol	Việt Nam	10000.0	2025-05-10	50	Còn thuốc
2	Ibuprofen	Mỹ	20000.0	2025-06-15	50	Còn thuốc
3	Hydroxychloro...	Mỹ	20000.0	2025-06-15	50	Còn thuốc
4	Metformin	Hoa Kỳ	20000.0	2025-06-15	50	Còn thuốc
5	Gliptins	Mỹ	20000.0	2025-06-15	50	Còn thuốc
6	Sulfonylureas	Mỹ	20000.0	2025-06-15	50	Còn thuốc
7	Insulin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
8	Dexamethasone	Mỹ	20000.0	2025-06-15	50	Còn thuốc
9	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
10	Amoxicillin	Mỹ	20000.0	2025-06-15	50	Còn thuốc
11	Loratadine	Mỹ	20000.0	2025-06-15	50	Còn thuốc
12	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
13	Antibiotics	Mỹ	20000.0	2025-06-15	50	Còn thuốc
14	Sucralfate	Mỹ	20000.0	2025-06-15	50	Còn thuốc
15	Famotidine	Mỹ	20000.0	2025-06-15	50	Còn thuốc

Hình 19 Quản lý tồn kho thuốc

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Tên thuốc	JTextField	Nhập tên thuốc cần tìm thông tin
2	Tìm kiếm	JButton	Tìm kiếm thông tin thuốc
3	Làm mới	JButton	Hiển thị lại tất cả dữ liệu sau tìm kiếm
4	Trang chủ	JButton	Quay lại trang chủ
5	Danh sách thông tin thuốc	JTable	Hiển thị thông tin tất cả thuốc
6	Thêm	JButton	Nhấn để thêm thông tin thuốc mới
7	Xóa	JButton	Nhấn để xóa thông tin thuốc
8	Sửa	JButton	Nhấn để sửa thông tin thuốc
9	Thoát	JButton	Nhấn để thoát hệ thống



—

□

×

THÊM MỚI THUỐC

ID

Tên thuốc

Nước sản xuất

Đơn giá

HSD

May 28, 2024

▲

▼



Số lượng

Thêm

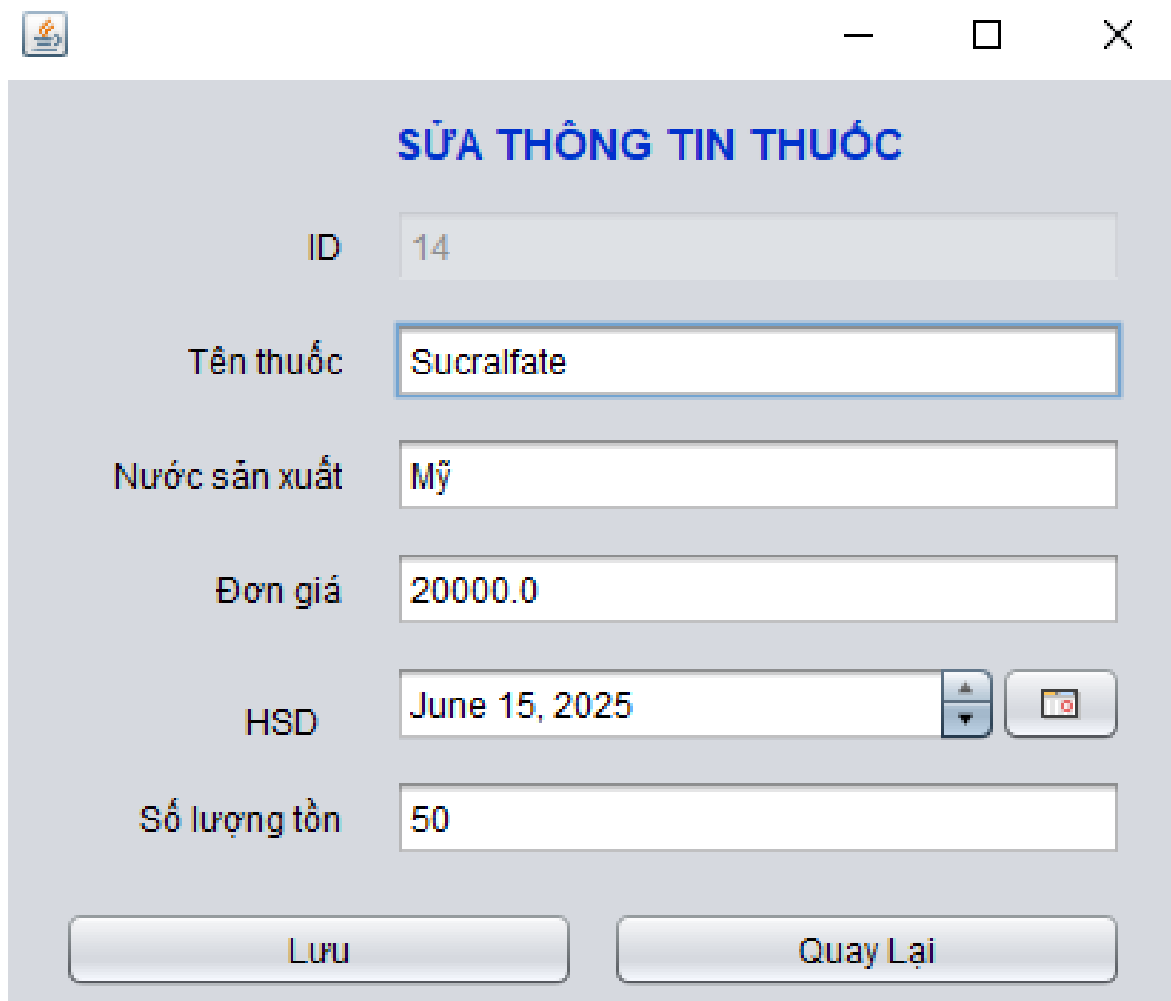
Quay Lại

Hình 20 Thêm thông tin thuốc

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	ID	TextField	Bị khóa (tự động cập nhật thứ tự thuốc trong danh sách
2	Tên thuốc	TextField	Nhập tên thuốc
3	Nhà sản xuất	TextField	Nhập nhà sản xuất thuốc
4	Đơn giá	TextField	Nhập đơn giá cho thuốc
5	HSD	DateChooser	Chọn hạn sử dụng của thuốc

6	Số lượng	TextField	Nhập số lượng nhập thuốc
7	Thêm	Button	Nhấn để thêm thuốc vào danh sách
8	Quay lại	Button	Hủy thêm thuốc và quay lại trang quản lý kho



SỬA THÔNG TIN THUỐC

ID: 14

Tên thuốc: Sucralfate

Nước sản xuất: Mỹ

Đơn giá: 20000.0

HSD: June 15, 2025

Số lượng tồn: 50

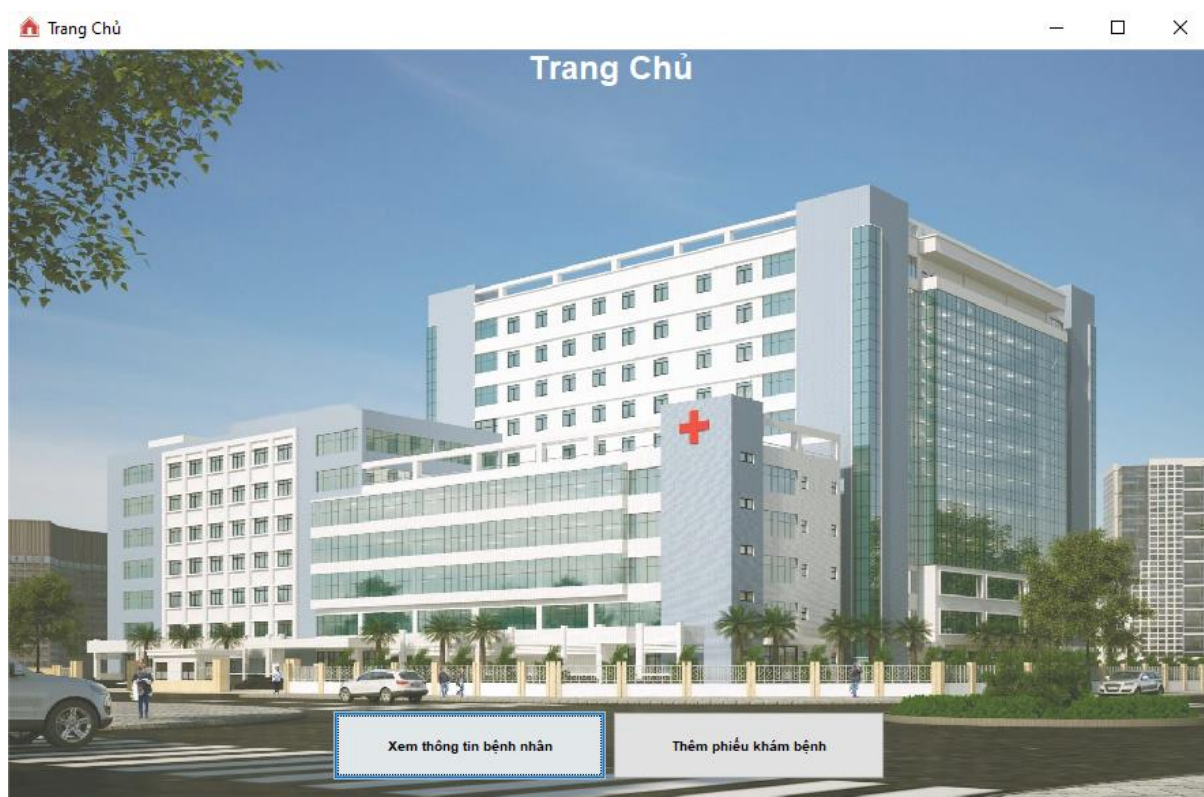
Lưu Quay Lại

Hình 21 Sửa thông tin thuốc

STT	Tên	Kiểu	Chức năng
1	ID	TextField	Hiển thị thứ tự của thuốc trong danh sách

2	Tên thuốc	TextField	Hiển thị tên thuốc
3	Nhà sản xuất	TextField	Hiển thị nhà sản xuất thuốc
4	Đơn giá	TextField	Hiển thị giá tiền của thuốc
5	HSD	Chooser	Chọn hạn sử dụng của thuốc
6	Số lượng	TextField	Hiển thị số lượng nhập thuốc
7	Lưu	Button	Nhấn để lưu thông tin thuốc vừa sửa vào danh sách
8	Quay lại	Button	Hủy thêm thuốc và quay lại trang quản lý kho

1.9. Màn hình giao diện tương tác với nhân viên tiếp nhận



Hình 22 Trang chủ nhân viên tiếp nhận

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Xem thông tin bệnh nhân	JButton	Chuyển tới giao diện Quản lý thông tin bệnh nhân
2	Thêm phiếu khám bệnh	JButton	Chuyển tới giao diện Quản lý kho thuốc



Hình 23 Trang quản lý thông tin bệnh nhân

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Menu	JMenuBar	Chức các chức năng con
2	Open File	JMenuItem	Mở file đã lưu từ máy tính
3	Save File	JMenuItem	Save file từ hệ thống vào máy tính

4	Xuất danh sách	JMenuItem	Xuất danh sách PDF thông tin tất cả bệnh nhân
5	Số điện thoại	JTextField	Nhập số điện thoại bệnh nhân để tìm kiếm
6	Tìm kiếm	JButton	Nhấn để tìm kiếm thông tin của bệnh nhân
7	Làm mới	JButton	Hiển thị lại tất cả thông tin phiếu khám bệnh
8	Quê quán	JCombobox	Chọn quê quán muốn lọc
9	Lọc	JButton	Lọc ra những bệnh nhân có quê quán đã chọn
9	Danh sách thông tin bệnh nhân	JTable	Hiển thị tất cả thông tin của các bệnh nhân
10	Thêm	JButton	Tạo phiếu khám bệnh cho bệnh nhân
11	Xóa	JButton	Xóa phiếu khám bệnh
12	Sửa	JButton	Sửa thông tin phiếu khám bệnh cho bệnh nhân
13	Thoát	JButton	Thoát hệ thống
14	Trang chủ	JButton	Quay về trang chủ

Thông tin bệnh nhân

THÔNG TIN BỆNH NHÂN



Tên bệnh nhân

Số điện thoại

Ngày sinh

Địa chỉ

Quê quán

Giới tính

Ghi chú

Thêm

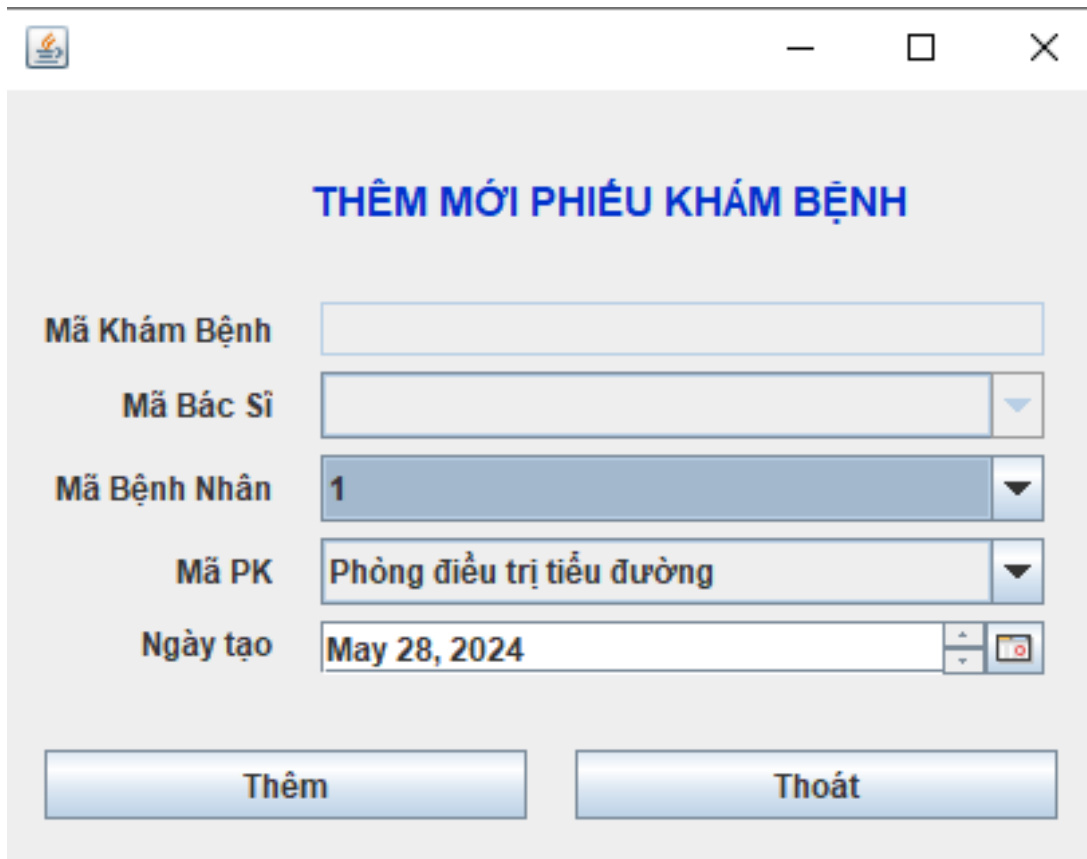
Lưu

Hình 24 Thông tin bệnh nhân

Mô tả giao diện.

STT	Tên	Kiểu	Chức năng
1	Tên bệnh nhân	JTextField	Nhập tên bệnh nhân
2	Số điện thoại	JTextField	Nhập số điện thoại
3	Ngày sinh	JDateChooser	Chọn ngày sinh của bệnh nhân
4	Địa chỉ	JTextField	Nhập địa chỉ bệnh nhân
5	Quê quán	JCombobox	Chọn quê quán của bệnh nhân
6	Giới tính	JCombobox	Chọn giới tính của bệnh nhân
7	Ghi chú	JTextField	Ghi chú các thông tin khác nếu có
10	Thêm	JButton	Tạo hồ sơ cho bệnh nhân mới
11	Lưu	JButton	Lưu thông tin mới của bệnh nhân sau khi sửa

pg. 162



THÊM MỚI PHIẾU KHÁM BỆNH

Mã Khám Bệnh

Mã Bác Sĩ

Mã Bệnh Nhân

Mã PK

Ngày tạo

Thêm **Thoát**

Hình 25 . Thêm mới phiếu khám bệnh

BỘ Y TẾ
BỆNH VIỆN QUỐC TẾ
Mã ĐVQHNS: 3569

PHIẾU KHÁM BỆNH

Mã Phiếu: PKB00008
Tên bệnh nhân: Nguyễn Thị Lành
Năm sinh: 2014
Tuổi: 10
Địa chỉ: 764 Bình Tây

Yêu cầu khám Phòng điều trị tiểu đường

Ngày 03 tháng 06 năm 2024

Hình 26 phiếu khám bệnh PDF

BỘ Y TẾ
BỆNH VIỆN QUỐC TẾ
Mã ĐVQHNS: 3569

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

BIÊN LAI THU TIỀN VIỆN PHÍ

Mã biên lai: BL00005

Mã toa: TOA00008

Tên bệnh nhân: Nguyễn Thị Lành

Năm sinh: 2014

Tuổi: 10

Địa chỉ: 764 Bình Tây

Quê quán: Bạc Liêu

Tổng tiền khám: 1,100,000 VNĐ

NGƯỜI NỘP TIỀN
(Ký và ghi rõ họ tên)

Ngày 03 tháng 06 năm 2024
TÊN NHÂN VIÊN THANH TOÁN
(Ký và ghi rõ họ tên)

Hình 27 Biên lai PDF

BỘ Y TẾ
BỆNH VIỆN QUỐC TẾ
Mã ĐVQHNS: 3569

PHIẾU KẾT QUẢ KHÁM BỆNH

Họ tên: Nguyễn Thị Lành

Năm sinh: 2014

Giới tính: Nữ

Địa chỉ: 764 Bình Tây

Yêu cầu khám: Phòng điều trị tiểu đường

Bác sĩ điều trị: Nguyễn Thanh Thức

Chẩn đoán: Tiểu đường

Tòa thuốc

Mã Toa	Mã Thuốc	Tên Thuốc	Số Lượng	Đơn Giá	Ghi chú
TOA00008	1	Paracetamol	20	10000.0	Ăn xong uống
TOA00008	7	Insulin	30	20000.0	Sáng 1 viên

Ngày 03 tháng 06 năm 2024

Hình 28 Phiếu khám bệnh chi tiết PDF

pg. 165

Mô tả giao diện

STT	Tên	Kiểu	Chức năng
1	Mã khám bệnh	JTextField	Bị khóa(Tự động cập nhật lấy mã khám bệnh mới nhất)
2	Mã bác sĩ	JComboBox	Chọn mã bác sĩ đang khám
3	Mã bệnh nhân	JComboBox	Chọn mã bệnh nhân đang khám
4	Tên phòng khám	JComboBox	Chọn tên phòng khám đang khám
5	Ngày tạo	JDateChooser	Chọn ngày tạo phiếu khám bệnh
7	Thêm PKB	JButton	Thêm phiếu khám bệnh
12	Thoát	JButton	Nhấn để quay lại trang chủ của nhân viên tiếp nhận.

PHẦN V. KẾT LUẬN

1. Bảng phân công công việc

		Hồng Tuyết	Tường Vi	Khôi Nguyên	Ánh Xuân
Bảng tóm tắt nội dung bằng tiếng anh				x	
Phát biểu bài toán			x		
Xác định yêu cầu			x		
Phân tích thiết kế		x	x	x	x
Mô tả ràng buộc toàn vẹn			x		
Xây dựng giao tác					
Phần II.2 TRIGGER	2.1 → 2.4			x	
	2.5 → 2.7				x
	2.8 → 2.10	x			
	2.11 → 2.14		x		
Xây dựng giao tác Procedure, Function		x	x	x	x
Xử lý đồng thời					
Lost update	TH1		x		x

2. Môi

	TH2		x		x
Dirty read	TH1		x		x
	TH2		x		x
Non-Repeatable Read	TH1		x		x
	TH2		x		x
Phantom Read	TH1		x		x
	TH2		x		x
Deadlock	TH1		x		x
	TH2		x		x
Giao diện					
Thiết kế giao diện		x	x		
Viết chương trình Java		x	x		
Tổng kết					
Kết luận báo cáo		x	x	x	x
File word báo cáo		x	x	x	x
File ppt báo cáo		x	x	x	x

trường phát triển và môi trường triển khai**2.1. Môi trường phát triển**

- Hệ điều hành: Microsoft Windows 10
- Hệ quản trị cơ sở dữ liệu: SQL SERVER
- Công cụ xây dựng ứng dụng: NetBean , Eclipse
- Ngôn ngữ: Java

2.2. Môi trường triển khai

- Hệ điều hành: Microsoft Windows 7 trở lên
- Cần cài đặt hệ quản trị cơ sở dữ liệu SQL SERVER

Kết quả đạt được**3.1. Kết quả đạt được**

Có khả năng thiết kế và xây dựng cơ sở dữ liệu một cách hiệu quả.

Thành thạo trong việc tạo Trigger, Store Procedure, và Function.

Hiểu biết về các tình huống có thể gây ra mất nhất quán dữ liệu và có thể đưa ra giải pháp để khắc phục.

Có kỹ năng phát triển ứng dụng, giao diện người dùng, và tạo các báo cáo.

Xây dựng hệ thống với các chức năng bao gồm:

- Quản lý thông tin bệnh nhân.

- Quản lý thông tin nhân viên.
- Quản lý thông tin tài khoản.
- Quản lý thuốc.
- Quản lý thông tin khám bệnh.
- Quản lý thông tin biên lai
- Xuất/Save file.
- Thống kê số liệu
- Xuất PDF

3.2. Khó khăn

- Do thời gian thực hiện đề tài còn hạn chế, kết quả của đề tài vẫn còn một số hạn chế:
- Chưa triển khai được chức năng thống kê xét nghiệm cho bệnh nhân
- Kỹ năng và kinh nghiệm làm đồ án còn hạn chế
- Chưa đạt được mức độ hoàn thiện cao nhất

4. Hướng phát triển

Cải thiện bảo mật và quyền riêng tư dữ liệu.

Cải thiện giao diện người dùng: Thiết kế giao diện trực quan và dễ sử dụng hơn.

5. Tài liệu tham khảo

- Đồ án xây dựng hệ thống rạp chiếu phim (Đồ án tham khảo)
- Slide bài giảng chương 3 Quản lý giao tác, chương 4 Điều khiển đồng thời, chương 5 Khôi phục sự số.
- Đồ án mẫu "Xây dựng hệ thống quản lý nhà sách hải an"
- Đồ án mẫu "Quản lý trung tâm tiếng anh"
- Bài báo Thông tư 46/2018/TT-BYT về hồ sơ bệnh án điện tử, đăng bởi trang LuậtVietnam-Thành viên INCOM Communications ., JSC
- Bài đăng: <https://hatex.vn/chao-ban/phan-mem-quan-ly-kho-ho-so-benh-an-ehis-42577.html>
- Bài báo: <https://accgroup.vn/benh-an-ngoai-tru>
- Website: <https://viblo.asia/p/transaction-o-muc-do-co-lap-isolation-level-1ZnbRIWNv2Xo>
- Website: <https://dogy.io/2021/01/01/transaction-isolation-part-2/>
- Website: <https://rabiloo.com/vi/blog/mat-du-lieu-cap-nhat-lost-update-la-gi-va-cach-xu-ly>

